HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO TỔNG HỢP AGENT 02

MÔN HỌC: PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

HỆ ĐA TÁC TỬ, REINFORCEMENT LEARNING
TUTORIAL – PAPER – ASSIGNMENT

Giảng viên hướng dẫn	:	PGS.TS. Trần Đình Quế
Lớp	:	D23CTPM01
Nhóm học phần	:	CT01
Sinh viên thực hiện	:	Nguyễn Nam Hải
Mã sinh viên	:	B23DCCN277

Hà Nội – Năm 2026

MỤC LỤC

Danh mục thuật ngữ **vii**

I TUTORIAL: PHÁT TRIỂN HỆ ĐA TÁC TỬ 1

1 Mục tiêu, lộ trình và bản đồ thực hành **1**
1.1 Chuẩn bị môi trường và chạy nhanh 1

2 Unit 1 — Từ mô hình AI đến hệ đa tác tử **2**

3 Unit 2 — Chọn bài toán và xác định lý do dùng MAS **2**

4 Unit 3 — PEAS, tính hợp lý và đặc tả nhiệm vụ **2**

5 Unit 4 — Môi trường, mục tiêu và kiến trúc hybrid **3**

6 Unit 5 — Kiến trúc giao tiếp và sơ đồ vận hành **4**

7 Unit 6 — Quy trình kỹ thuật từ yêu cầu đến kiểm thử **4**

8 Unit 7 — Python cơ bản và hợp đồng đầu vào **5**

9 Unit 8 — Catalog và recommendation **5**

10 Unit 9 — Coordinator và bất biến đầu ra **5**

11 Unit 10 — PyTorch: từ mạng chưa train đến classifier thật **6**
11.1 Ngưỡng tin cậy, từ chối và lỗi ngoài miền 6

12 Unit 11 — Timeout, fallback và giám sát **7**

13 Unit 12 — Case study thương mại điện tử **7**

14 Unit 13 — Case study du lịch **8**

15 Unit 14 — Case study hỗ trợ hành chính y tế **8**

16 Unit 15 — So sánh ba miền và quyền sở hữu ràng buộc **8**

17 Unit 16 — Thiết kế benchmark có đối chứng **9**

18 Unit 17 — Hoàn thành năm bài thực hành **9**
18.1 Exercise 1: Đặc tả tác tử và PEAS 9
18.2 Exercise 2: Mở rộng shopping system 9
18.3 Exercise 3: Huấn luyện intent classifier 10

18.4 Exercise 4: Failure injection . . . 10
18.5 Exercise 5: So sánh kiến trúc . . . 10

19 Unit 18 — Kiểm thử, rubric và lỗi thường gặp 11

20 Unit 19 — Kiến trúc tham chiếu và hướng phát triển 11

21 Unit 20 — Tổng kết, demo và câu hỏi bảo vệ 11

22 Tutorial thực hành: xây dựng và chạy hệ đa tác tử bằng Python 12
22.1 Chuẩn bị và tổ chức tệp . . . 12
22.2 Bài 1: định nghĩa hợp đồng thông điệp và môi trường . . . 12
22.3 Bài 2: coordinator, luật và xác minh tồn kho . . . 13
22.4 Bài 3: huấn luyện PyTorch và đánh giá . . . 14
22.5 Bài 4: chạy các ca bình thường và đưa lỗi vào hệ thống . . . 15
22.6 Bài 5: so sánh kiến trúc và chạy kiểm thử . . . 15

23 Phụ lục mã nguồn đầy đủ của tutorial 16
23.1 Mã gốc: hệ cơ bản theo slide . . . 16
23.2 Mã mở rộng: shopping multi-agent system . . . 18
23.3 Mã huấn luyện và đánh giá mô hình PyTorch . . . 20
23.4 Mã chạy ví dụ thực hành tổng hợp . . . 22
23.5 Mã benchmark B0–B3 . . . 24
23.6 Mã kiểm thử tutorial . . . 25

24 Kết quả thực hành và bằng chứng chạy 26
24.1 Kiến trúc B0–B3 . . . 26
24.2 Chỉ số classifier trên held-out test . . . 26
24.3 Ngoài miền: những lỗi cần giữ lại để phân tích . . . 26
24.4 Kết quả failure injection và test . . . 27
24.5 Mã thực thi cần đọc khi bảo vệ . . . 27

II ĐỌC VÀ PHÂN TÍCH HAI PAPER 29

25 Mở đầu: Hai paper trong một khung nghiên cứu 29
25.1 Cấu trúc và quy ước bằng chứng . . . 29

26 Paper 1: Cơ chế cộng tác đa tác tử dựa trên LLM 30
26.1 Hồ sơ tài liệu, phạm vi và cách tiếp cận . . . 30
26.2 Vấn đề nghiên cứu và động lực . . . 30
26.3 Đóng góp và vị trí của survey . . . 31
26.4 Nền tảng: MAS, LLM và collaborative AI . . . 31
26.5 Mô hình hóa tác tử và hệ cộng tác . . . 32
26.6 Cộng tác sớm, giữa và muộn . . . 32

26.7 Ba loại quan hệ cộng tác . . . 33
26.8 Ba chiến lược: rule-based, role-based, model-based . . . 34
26.9 Cấu trúc giao tiếp và chi phí . . . 35
26.10 Điều phối tĩnh và động . . . 36
26.11 Bỏ phiếu, tranh luận và kiểm chứng . . . 36
26.12 Ứng dụng được survey phân tích . . . 37
26.13 Vấn đề mở và đánh giá phê bình . . . 37
26.14 Thiết kế nghiên cứu tiếp nối do báo cáo đề xuất . . . 38
26.15 Kết luận riêng cho Paper 1 . . . 39

27 Paper 2: Học tăng cường và ứng dụng trong các lĩnh vực **40**
27.1 Hồ sơ bài báo và mục đích đọc . . . 40
27.2 RL khác học có giám sát và học không giám sát . . . 40
27.3 Mô hình MDP và giả định Markov . . . 40
27.4 Những thành phần cơ bản của môi trường . . . 41
27.5 Return, hệ số chiết khấu và tiêu chí kinh doanh . . . 41
27.6 Chính sách, kế hoạch và hàm giá trị . . . 42
27.7 Phương trình Bellman . . . 42
27.8 Dynamic programming: biết mô hình để lập kế hoạch . . . 43
27.9 Monte Carlo: học từ return hoàn chỉnh . . . 44
27.10 Temporal difference và ý nghĩa bootstrapping . . . 44
27.11 SARSA và Q-learning . . . 44
27.12 Epsilon-greedy và khám phá . . . 45
27.13 Learning rate và điều kiện hội tụ . . . 45
27.14 Actor–critic và policy gradient . . . 46
27.15 Bayesian methods và bất định . . . 46
27.16 Deep Q-Network và các thành phần ổn định . . . 47
27.17 So sánh các họ phương pháp . . . 47
27.18 Ứng dụng trong paper: đọc bằng cấu trúc bài toán . . . 48
27.19 Các thách thức lớn và đọc phản biện . . . 49
27.20 Kiểm tra giả định trên chính môi trường assignment . . . 50
27.21 Phân tích lợi nhuận kỳ vọng và chính sách mốc . . . 51
27.22 Cầu nối với MAS và phương pháp nghiên cứu chung . . . 51
27.23 Kết luận riêng cho Paper 2 . . . 52

28 Chuyên đề toán học và kỹ thuật bổ sung **53**
28.1 1. Từ collaboration channel đến hợp đồng có thể kiểm . . . 53
28.2 2. State machine cho workflow thay vì vòng chat không giới hạn . . . 53
28.3 3. Complexity của giao tiếp và đường găng . . . 54
28.4 4. Mô hình lỗi dây chuyền và giá trị kiểm chứng . . . 54
28.5 5. Đa số và tương quan sai số . . . 54
28.6 6. Confusion matrix, macro-F1 và selective prediction . . . 55
28.7 7. Vì sao chia theo nhóm dữ liệu quan trọng . . . 55

28.8 8. Chứng minh observation baseline chưa đủ Markov . . . 55
28.9 9. Hữu hạn chân trời và terminal aliasing . . . 56
28.1010. Quy hoạch động lùi: một ví dụ hai bước . . . 56
28.1111. Giá trị kỳ vọng của policy bằng forward occupancy . . . 57
28.1212. Tách lỗi học khỏi nhiễu evaluation bằng exact values . . . 57
28.1313. Kỳ vọng và phương sai profit ở một bước . . . 57
28.1414. Confidence interval và đơn vị mẫu . . . 58
28.1515. Robustness: zero-shot shift khác retraining . . . 58
28.1616. Đối thủ cố định, self-play và cân bằng . . . 58
28.1717. Từ thuật toán sang thiết kế có thể tái lập . . . 59
28.1818. Phạm vi đóng góp của bộ báo cáo . . . 59

29 Tổng hợp, phản biện và thiết kế nối hai hướng nghiên cứu 60
29.1 Bảng đối chiếu mục tiêu và loại bằng chứng . . . 60
29.2 Hai cấp độ thích nghi không nên gộp . . . 60
29.3 Thiết kế tham chiếu: trợ lý kinh doanh có tầng quyết định và tầng kiểm chứng 60
29.4 Thiết kế thí nghiệm để tách lợi ích chia vai khỏi tăng tính toán . . . 61
29.5 Đề xuất kiểm tra độ bền nhiều tầng . . . 61
29.6 Những luận điểm cần tránh sau khi đọc hai paper . . . 61

30 Bằng chứng minh họa liên hệ hai paper 62

31 Phụ lục đọc paper: bản đồ nội dung và câu hỏi kiểm tra 63
31.1 Đối chiếu Paper 1 với tài liệu gốc . . . 63
31.2 Đối chiếu Paper 2 với tài liệu gốc . . . 64
31.3 Câu hỏi phản biện và lời giải định hướng . . . 65

32 Kết luận chung của báo cáo paper 65

III ASSIGNMENT: RL VÀ RULE-BASED AGENTS 66

33 Giới thiệu và mục tiêu thí nghiệm 66

34 Mô tả môi trường và hai tác tử 66
34.1 Mô hình thị trường . . . 66
34.2 Agent A: Q-learning . . . 67
34.3 Agent B: luật cố định . . . 67
34.4 Hai vấn đề trạng thái cần lưu ý . . . 67

35 Mã nguồn và thay đổi so với tài liệu tham chiếu 67

36 Thiết lập thực nghiệm và protocol 68

37 Thí nghiệm A — Baseline và policy đã học 69

38 Thí nghiệm B — Quá trình học **69**

39 Thí nghiệm C — Learning rate **70**

40 Thí nghiệm D — Khám phá **70**

41 Thí nghiệm E — So sánh policy công bằng **70**

42 Trả lời đầy đủ mười câu hỏi **71**
42.1 Câu 1 — Mô hình tác tử và môi trường . . . 71
42.2 Câu 2 — Quyết định học và theo luật . . . 71
42.3 Câu 3 — Tính cập nhật Q-learning . . . 71
42.4 Câu 4 — Khám phá và khai thác . . . 72
42.5 Câu 5 — Reward và mục tiêu kinh doanh . . . 72
42.6 Câu 6 — Hiệu năng và thống kê . . . 73
42.7 Câu 7 — Learning rate và ổn định . . . 73
42.8 Câu 8 — Tương tác và môi trường không dừng . . . 73
42.9 Câu 9 — Giới hạn và biến đổi đã thực hiện . . . 74
42.10 Câu 10 — Thiết kế agent tốt hơn . . . 74

43 Phân tích bổ sung: lợi nhuận, oracle và độ nhạy **75**
43.1 Bảng lợi nhuận kỳ vọng . . . 75
43.2 Giải thích mốc 35 và quy hoạch động . . . 75
43.3 Ngưỡng discount có lợi tức thời . . . 76

44 Thảo luận về độ tin cậy, giới hạn và diễn giải **76**

45 Kiểm tra độc lập bằng đánh giá policy chính xác **76**

46 Kết luận của assignment **77**

47 Phụ lục kết quả đo và bằng chứng thực thi **77**
47.1 Bảng tổng hợp thí nghiệm alpha và epsilon . . . 77
47.2 So sánh công bằng với cùng đối thủ . . . 77
47.3 Thí nghiệm biến đổi discount 40% . . . 78
47.4 Q-table minh họa seed 42 . . . 78
47.5 Từng seed của baseline . . . 78
47.6 Cặp learner trong self-play . . . 78
47.7 Đối chiếu bản gốc và bản mở rộng seed 42 . . . 79
47.8 Mã Python cốt lõi . . . 81
47.9 Bằng chứng kiểm thử . . . 82

Tài liệu tham khảo **83**

DANH MỤC HÌNH VẼ

1 Kiến trúc cục bộ: các quyết định có vai trò riêng, kết quả đi qua coordinator. 4
2 Loss train/validation và confusion matrix trên 108 câu test tổng hợp. 27
3 Minh họa phân biệt chất lượng trong phân phối và đánh giá ngoài miền; đây là dữ liệu tự thực thi. 62
4 Độ nhạy learning rate trong mô hình bán hàng toy; vùng tô là một SD qua năm seed. 63
5 Đánh giá policy đóng băng và mốc analytic không giảm giá. 63
6 Learning curve baseline: moving average 100 episode, mean và một SD qua năm seed. 79
7 So sánh ba learning rate với cùng epsilon schedule. 79
8 So sánh ba cấu hình epsilon; mọi cấu hình đánh giá với epsilon bằng 0. . . . 80
9 Policy đóng băng: mean và SD giữa seed; đường 35 là kỳ vọng lý thuyết. . . 80
10 Q-table seed 42 và độ nhạy lợi nhuận tức thời theo discount fraction. 80

DANH MỤC THUẬT NGỮ

MAS	Multi-Agent System – Hệ đa tác tử
LLM	Large Language Model – Mô hình ngôn ngữ lớn
RL / MARL	Reinforcement Learning / Multi-Agent RL
MDP / POMDP	Quá trình quyết định Markov / quan sát không đầy đủ
TD / Q	Sai phân thời gian / Giá trị hành động
DP	Dynamic Programming – Quy hoạch động
OOD	Out-of-distribution – Ngoài phân phối/ngoài miền
PEAS	Performance, Environment, Actuators, Sensors
DAG	Directed Acyclic Graph – Đồ thị có hướng không chu trình
SD / CI	Độ lệch chuẩn mẫu / Khoảng tin cậy
P50 / P95	Phân vị 50 / 95 của thời gian phản hồi
TTL	Time to Live – Giới hạn độ mới của dữ liệu

Quy ước đọc báo cáo

Dữ liệu thực hành là tổng hợp hoặc mô phỏng. Số đo được lấy từ mã đã chạy và lưu kèm; phép tính lý thuyết và đề xuất chưa thực hiện được gọi rõ. Thông tin bìa, logo và phong cách trình bày kế thừa báo cáo trước. Bản tổng hợp cập nhật ngày 29/10/2026.

Phần I

TUTORIAL: PHÁT TRIỂN HỆ ĐA TÁC TỬ

1 Mục tiêu, lộ trình và bản đồ thực hành

Tutorial này phát triển đầy đủ 20 chủ đề trong slide *Developing Multiagent Systems: A Practical Tutorial with Python and PyTorch*, đồng thời hoàn thành năm bài thực hành ở Unit 17. Bản mở rộng thêm hợp đồng thông điệp, timeout thực sự, kiểm tra độ mới tồn kho, dữ liệu huấn luyện có chia nhóm, kiểm thử tự động, ablation và bảng đo thực nghiệm. [1]

Kết quả cuối là một trợ lý mua hàng cục bộ có CatalogAgent, RecommendationAgent, InventoryAgent, PolicyAgent và Coordinator; một bộ định tuyến intent bằng PyTorch đã train; một baseline một thành phần; và bộ ca lỗi dùng để kiểm tra phối hợp. Dữ liệu catalog, kho và câu yêu cầu đều **tổng hợp phục vụ học tập**. Không có thanh toán, đặt mua, dữ liệu bệnh nhân hay kết nối dịch vụ ngoài.

Lộ trình đề xuất gồm ba buổi: buổi 1 học khái niệm và chạy luồng mua hàng; buổi 2 đọc hợp đồng, thêm kiểm chứng và thử lỗi; buổi 3 train classifier, chạy benchmark và phân tích kết quả. Người học nên chạy ví dụ nhỏ trước rồi mới đọc các mở rộng. Các lệnh bên dưới chạy từ thư mục `solution/`; script tự tìm thư mục kết quả theo vị trí tệp nên cũng có thể gọi bằng đường dẫn tuyệt đối.

1.1 Chuẩn bị môi trường và chạy nhanh

Mã đã được kiểm tra bằng Python 3.9.6, NumPy và PyTorch CPU trong máy hiện tại; với môi trường mới nên dùng Python 3.10 trở lên có bản PyTorch phù hợp. Hệ MAS cơ bản chỉ dùng thư viện chuẩn; classifier và đồ thị cần các gói ghi trong `requirements.txt`. Các phiên bản thực tế được lưu trong JSON và tệp môi trường, giúp người đọc phân biệt môi trường đã thử với khuyến nghị cài đặt.

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
python3 tutorial/slide_baseline.py
python3 tutorial/mas_system.py
python3 tutorial/train_intent.py
python3 tutorial/evaluate_mas.py
python3 -m unittest discover -s tutorial -p 'test_*.py'
```

Các bước huấn luyện và đánh giá được tách tệp. Nếu chỉ muốn xem demo MAS, chưa cần train classifier. Nếu muốn chạy B2 hoặc kiểm thử liên quan router, cần tạo checkpoint trước, hoặc dùng checkpoint đã kèm trong `results/tutorial/`. Khi dùng Jupyter, mở notebook tutorial, chọn kernel cùng môi trường rồi Run All; đầu trang notebook ghi rõ phạm vi thực thi.

2 Unit 1 — Từ mô hình AI đến hệ đa tác tử

Một agent tiếp nhận thông tin, lựa chọn hành động và chịu trách nhiệm một phần mục tiêu. MAS có nhiều thành viên với quyết định tương tác qua môi trường hoặc thông điệp. Một nhóm hàm thuần túy không tự động chứng minh cần MAS; phải có ranh giới nhiệm vụ có ý nghĩa, chính sách hoặc tài nguyên riêng và cơ chế phối hợp có thể quan sát.

Trong ví dụ mua hàng, mô hình xếp hạng chỉ trả điểm. Hệ thông minh còn cần truy hồi đúng sản phẩm, kiểm tra ngân sách, xác nhận kho và phản hồi khi không đủ bằng chứng. Chia thành agent giúp xác định ai sở hữu mỗi quyết định. Mã chạy chung một tiến trình; "đa tác tử" ở đây nói về trách nhiệm và chính sách, không phải nhiều máy hay nhiều LLM.

Thực hành: chạy `mas_system.py`, quan sát đầu ra của kịch bản normal, timeout, malformed, stale và forged. Với normal, danh sách là p2 giá 25 và p4 giá 55. Đây là kết quả của lọc dữ liệu, xếp hạng và kiểm chứng kho, không có suy luận ngôn ngữ được giả lập. Ghi lại thành phần nào tạo ra từng trường trong kết quả.

Câu hỏi tự kiểm tra: nếu thay tất cả agent bằng một hàm lớn nhưng giữ hành vi, lợi ích và chi phí thiết kế thay đổi thế nào? Câu trả lời cần nói đến khả năng thay thế thành phần, kiểm thử, logging và overhead, thay vì khẳng định số lượng agent quyết định trí thông minh.

3 Unit 2 — Chọn bài toán và xác định lý do dùng MAS

Các miền e-commerce, tourism và healthcare support đều có thể phân rã thành chuyên môn, nhưng yêu cầu dữ liệu và mức rủi ro khác nhau. Mua hàng cần catalog, giá và tồn kho; lịch trình cần giờ mở cửa, thời gian đi và chi phí; hỗ trợ hành chính y tế cần nội dung đã duyệt và lịch hẹn. Cùng một sơ đồ agent không có nghĩa dùng chung tiêu chí nghiệm thu.

Tutorial chọn e-commerce vì có thể tạo một môi trường nhỏ với ground truth rõ. Mỗi sản phẩm có ID, giá, category và tags; kho có số lượng theo ID. Điều kiện thành công là trả đúng danh sách hợp lệ hoặc trả đúng trạng thái không thể xác minh. Đây là lựa chọn giúp kiểm chứng đầu cuối mà không cần API key.

Chỉ nên tách agent nếu ranh giới giúp quyết định, phân quyền hoặc thử nghiệm. CatalogAgent và InventoryAgent dùng hai loại dữ kiện khác nhau; PolicyAgent kiểm tra điều kiện bắt buộc; RecommendationAgent có thể thay bằng learned ranker sau này. Với một catalog năm sản phẩm, MAS không có lý do để được giả định nhanh hơn baseline đơn; benchmark sẽ đo thay vì suy đoán.

4 Unit 3 — PEAS, tính hợp lý và đặc tả nhiệm vụ

PEAS gồm Performance, Environment, Actuators và Sensors. Với trợ lý mua hàng: Performance là đúng ràng buộc, truy hồi đúng và không khẳng định tồn kho cũ; Environment là catalog và kho tổng hợp; Actuators là trả danh sách hoặc trạng thái lỗi; Sensors là truy vấn, ngân sách và kết quả agent. Không có actuator thanh toán.

Tính hợp lý là chọn hành động tốt theo thông tin và nguồn lực hiện có. Khi kho timeout, "không xác minh được" là kết quả hợp lý dù không đưa ra khuyến nghị hoàn chỉnh. Tạo một câu trả lời tự tin không làm nhiệm vụ thành công nếu thiếu dữ kiện bắt buộc. Do đó success phải bao gồm cả những ca từ chối hoặc fallback đúng.

Thành viên	Đầu vào	Đầu ra	Quyền	Tiêu chí kiểm tra
Catalog	Query	Danh sách ID	Đọc catalog	Tag khớp, query rỗng không trả toàn bộ
Recommender	Candidate ID, budget	Tối đa 3 ID	Đọc thuộc tính	Chỉ candidate hợp lệ, giá không vượt budget
Inventory	ID	Stock và tuổi dữ liệu	Đọc kho	Có đủ ID, stock không âm, dữ liệu mới
Policy	ID, candidate, budget	ID đã xác nhận	Chỉ kiểm chứng	Không cho ID lạ hoặc vượt giá
Coordinator	Yêu cầu	Status và items	Điều phối	Envelope đúng, deadline, đầu ra được kiểm

Mỗi hàng là một hợp đồng thiết kế. Khi một agent xử lý thêm trách nhiệm ngoài hàng của mình, nên giải thích vì sao và bổ sung test. Ví dụ recommender có thể lọc budget để tiết kiệm, nhưng PolicyAgent vẫn kiểm lại vì output thành phần có thể sai. Lặp kiểm tra ở ranh giới tin cậy là chủ đích, không phải hai định nghĩa nghiệp vụ khác nhau.

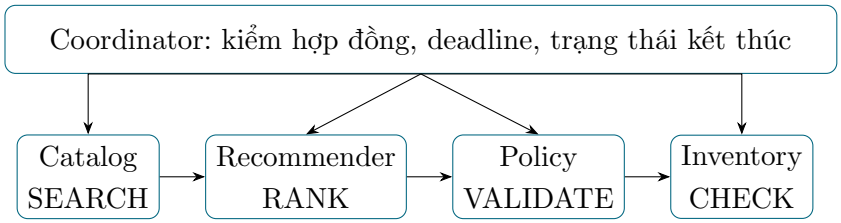
5 Unit 4 — Môi trường, mục tiêu và kiến trúc hybrid

Môi trường catalog nhỏ gần như tĩnh trong một lần gọi, nhưng dữ liệu kho có tuổi và có thể trở nên cũ. Một đề xuất dựa trên dữ liệu vừa đọc chỉ có hiệu lực tại thời điểm đó. Dữ liệu cũ một giờ được tạo trong kịch bản stale nhằm buộc hệ thống phân biệt "có số lượng trong bộ nhớ" với "đã xác minh hiện tại".

Kiến trúc hybrid dùng luật cho ràng buộc cứng và mô hình học cho quyết định dự đoán. Trong tutorial, intent classifier có thể chọn luồng, còn budget và provenance không giao cho mạng tự quyết. Một lỗi phân loại cần được ghi là lỗi định tuyến; một sản phẩm vượt budget cần bị chặn cho dù score mô hình cao đến đâu.

Khi mở rộng, có thể dùng BDI để tổ chức niềm tin, mục tiêu và ý định, hoặc utility để cân bằng sở thích mềm. Tuy nhiên, bất biến như "không trả ID không tồn tại" vẫn cần kiểm chứng trực tiếp. Không nên biến điều kiện bắt buộc thành một trọng số nhỏ trong score rồi hy vọng mô hình tự ưu tiên.

6 Unit 5 — Kiến trúc giao tiếp và sơ đồ vận hành



Hình 1: Kiến trúc cục bộ: các quyết định có vai trò riêng, kết quả đi qua coordinator.

Thông điệp có task ID, sender, receiver, kind và payload. Task ID liên kết các bước thuộc cùng yêu cầu; kind xác định loại thao tác; payload chứa dữ liệu có cấu trúc. Timestamp cho độ mới được mô hình hóa bằng tuổi dữ liệu kho, tránh lẫn với thời gian đo latency. Trace riêng lưu thời lượng thực thi từng lần gọi.

```
@dataclass(frozen=True)
class Message:
    task_id: str
    sender: str
    receiver: str
    kind: str
    payload: dict[str, Any]
```

Mỗi agent xác nhận người nhận và loại yêu cầu; coordinator kiểm sender, receiver, task ID và loại phản hồi. Cùng một payload nhưng task ID sai bị từ chối vì có thể là kết quả của công việc khác. Hệ hiện tại là in-process nên chưa có chữ ký hay xác thực mạng; schema validation không nên được mô tả thành cơ chế bảo mật phân tán hoàn chỉnh.

7 Unit 6 — Quy trình kỹ thuật từ yêu cầu đến kiểm thử

Bắt đầu bằng câu người dùng: “tìm đồ portable không quá 100”. Chuyển thành điều kiện kiểm chứng: query không rỗng, chỉ trả sản phẩm từ catalog, giá không quá 100, stock dương, dữ liệu kho không quá 60 giây. Xác định trạng thái dừng cho không kết quả và lỗi. Sau đó mới phân vai và viết message.

Bản end-to-end tối thiểu cần chạy được trước khi thêm ML. Nó cung cấp nơi kiểm tra lỗi đầu vào, lọc dữ liệu và tính đúng của logic. Khi thêm classifier, giữ nguyên bộ test để phát hiện mô hình mới làm hỏng ràng buộc cũ. Tách các thay đổi giúp biết lỗi đến từ orchestration hay learning.

Tệp `mas_traces.json` là bằng chứng hữu ích khi debug: mỗi case có kết quả cùng chuỗi lời gọi. Nếu kết quả trống, kiểm tra đó là no_results, timeout, availability_unverified hay contract_error. Các trạng thái này có ý nghĩa khác nhau với người dùng và không nên bị gom thành “AI không trả lời”.

8 Unit 7 — Python cơ bản và hợp đồng đầu vào

Catalog, kho và sản phẩm được lưu trong bộ nhớ để ví dụ dễ chạy. `Product` và `Message` dùng dataclass đóng băng, giúp hạn chế sửa vô ý ở cấu trúc ngoài. Lưu ý frozen dataclass không tự làm các dict lồng bên trong bất biến; code tránh sửa payload sau khi gửi và không quảng cáo cấu trúc này là deep immutable.

Coordinator kiểm query là chuỗi, budget là số hữu hạn và không âm. Các ca NaN cần được chặn vì so sánh với NaN có hành vi dễ gây lọt ràng buộc. Tệp test có cả budget âm và NaN. Một kiểm tra kiểu chung chung như "có thể chuyển thành float" chưa đủ để xác nhận ý nghĩa nghiệp vụ.

Thực hành sửa lỗi: thay query bằng chuỗi rỗng và quan sát no_results; thay budget bằng -1 và NaN để quan sát invalid_input. Sau đó đọc `Message.validate` và `Coordinator.call`. Chỉ mở rộng kiểu dữ liệu được chấp nhận khi có lý do và test phù hợp.

9 Unit 8 — Catalog và recommendation

CatalogAgent tìm chuỗi truy vấn trong tên, category hoặc tags của sản phẩm. Truy vấn được strip và chuyển lower. Query rỗng không được xem là khớp mọi chuỗi. Cách tìm này minh bạch, nhưng không hỗ trợ lỗi chính tả, đồng nghĩa, truy vấn nhiều điều kiện hoặc ngữ nghĩa tiếng Việt; đó là giới hạn có chủ đích của tầng retrieval.

RecommendationAgent lọc candidate có trong catalog và giá không quá budget, sắp theo giá tăng rồi lấy tối đa ba sản phẩm. Nó không phải một recommender cá nhân hóa được học từ người dùng. Tên "recommendation" mô tả chức năng đưa gợi ý, không hàm ý đã train mô hình xếp hạng.

Với query portable và budget 100, laptop 850 và tablet 150 bị loại; mouse 25 và backpack 55 còn hợp lệ. Với query sports, giày có giá 70 nhưng stock 0 nên danh sách cuối trống. Hai ca này phân biệt lọc theo giá và xác minh tồn kho, tránh nhầm retrieval có kết quả với nhiệm vụ đã hoàn tất.

10 Unit 9 — Coordinator và bất biến đầu ra

Coordinator thực hiện SEARCH → RANK → VALIDATE → CHECK. Kết quả được kiểm ở mỗi ranh giới. Một message đúng kiểu nhưng sai task ID vẫn thất bại. PolicyAgent đối chiếu cả catalog, candidate ban đầu và budget, giúp chặn một ranker đưa vào ID tự tạo hoặc sản phẩm ngoài truy vấn.

Đầu ra cuối có `status` và `items`; từng item có ID, giá lấy từ catalog và availability. Chỉ trạng thái đã xác minh mới có item được gắn verified. Nếu kho cũ hoặc lỗi, hệ không tái sử dụng một kết luận availability từ bước trước. Đây là tiêu chí cụ thể có thể kiểm bằng failure injection.

Một vấn đề chưa được giải quyết trong prototype là race condition giữa kiểm kho và mua thật. Ở đây không có thao tác mua nên kết quả chỉ là snapshot gợi ý. Nếu thêm checkout,

cần reservation, transaction, idempotency và kiểm tra lại trước xác nhận. Không thể coi việc đã CHECK là đã khóa được hàng.

11 Unit 10 — PyTorch: từ mạng chưa train đến classifier thật

Slide minh họa một mạng nhận hai đặc trưng số nhưng chưa huấn luyện. Bản mở rộng thay bằng dữ liệu câu tiếng Việt có nhãn và bag-of-words. Quy trình chuẩn hóa bỏ dấu, chuyển chữ thường và lấy token a–z; vocabulary chỉ xây từ train. Vector đầu vào có 79 chiều trong lần chạy hiện tại, mạng gồm Linear → ReLU → Linear với hidden size 32 và ba logits đầu ra.

Dữ liệu gồm 432 câu: ba lớp, mỗi lớp tám họ động từ, sáu cụm đối tượng và ba hậu tố. Chia theo họ động từ: năm họ train, một validation và hai test trong mỗi lớp; tương ứng 270/54/108 câu. Không có câu trùng chính xác giữa các split và group ID không giao nhau. Tuy vậy, từ chỉ đối tượng vẫn chia sẻ giữa split, nên đây là kiểm tra tổng hợp tương đối dễ.

Hàm mất mát CrossEntropyLoss nhận logits và nhãn số nguyên; không softmax trước loss. Adam dùng learning rate 0.02, 120 epoch, seed 42 và một CPU thread. Checkpoint được chọn theo validation loss thấp nhất; test chỉ dùng để báo cáo cuối. Tài liệu PyTorch chính thức xác nhận giao diện logits/target này. [10]

```
model.train()
optimizer.zero_grad()
logits = model(train_x)
loss = criterion(logits, train_y)
loss.backward()
optimizer.step()
model.eval()
with torch.inference_mode():
    probabilities = model(test_x).softmax(dim=1)
```

Classifier đạt accuracy và macro-F1 bằng 1 trên 108 câu test, mỗi lớp 36 câu. Đây là số đo thật trên tập tổng hợp, không chứng minh hiểu ngôn ngữ rộng. Không có người dùng thật hoặc dữ liệu bệnh viện trong tập. Mục tiêu của bài thực hành là thực hiện đúng vòng train–validation–test và quan sát giới hạn ngoài phân phối.

11.1 Ngưỡng tin cậy, từ chối và lỗi ngoài miền

Ngưỡng được chọn trên validation để giữ coverage cao với không có lỗi quan sát được; kết quả là 0.5. Router còn yêu cầu câu có ít nhất một token thuộc lexicon miền. Hai lớp kiểm tra này vẫn không bảo đảm phát hiện OOD. Softmax là độ tự tin nội bộ của mô hình, không phải xác suất được hiệu chỉnh rằng yêu cầu thuộc miền.

Sáu câu OOD minh họa được đánh giá sau chọn ngưỡng. Bốn câu được trả clarify, nhưng "thời tiết hôm nay thế nào" bị route sang ecommerce và "giải phương trình" sang tourism. Nguyên nhân có thể liên quan token trùng và độ tự tin cao ngoài miền; chưa có ablation đủ để kết luận định lượng vai trò từng token. Báo cáo giữ nguyên các lỗi để cho thấy accuracy test tổng hợp chưa đủ.

Một cải tiến tiếp theo là tạo tập development OOD và test OOD độc lập, định nghĩa reject class hoặc huấn luyện phát hiện ngoài miền, rồi đo false acceptance và coverage–risk. Không được sửa lexicon theo đúng sáu câu đã xem rồi báo rằng hệ đã tổng quát hóa. Nếu dùng temperature scaling, nhiệt độ cũng phải chọn bằng validation riêng.

12 Unit 11 — Timeout, fallback và giám sát

`asyncio.wait_for` áp deadline thực sự lên coroutine gọi agent. Inventory chậm được mô phỏng bằng sleep 50 ms, trong khi deadline mặc định là 10 ms. Khi quá hạn, lời gọi bị hủy và coordinator trả timeout cùng danh sách trống. Đây là điểm mạnh hơn đoạn `try/except` chỉ bắt TimeoutError mà không hề thiết lập thời hạn.

Tính tương thích Python cần chú ý: ở một số phiên bản, `asyncio.TimeoutError` và built-in `TimeoutError` không hoàn toàn cùng lớp. Mã bắt cả hai; test timeout đã được thực thi trong môi trường hiện tại. Trường status của trace được khởi tạo trước try để ngay cả lỗi vẫn ghi nhận được, tránh lỗi logging che mất lỗi gốc.

Deadline ở mức coroutine không tương đương khả năng dừng an toàn mọi thư viện blocking hoặc một tiến trình ngoài. Với HTTP, cần timeout của client; với CPU job, có thể cần process riêng; với thao tác ghi, timeout còn để lại trạng thái không biết đã hoàn tất chưa. Prototype này chỉ đọc dữ liệu nên không có retry giao dịch hoặc duplicate purchase.

Các chỉ số theo dõi gồm success, violation, P50/P95 latency và số message. Một request–response được tính hai message. Trace hiện lưu task ID và thời lượng, không chứa khóa API hay dữ liệu cá nhân. Cấu trúc này đủ để minh họa quan sát vận hành mà chưa phải hệ tracing phân tán đầy đủ.

13 Unit 12 — Case study thương mại điện tử

Thiết kế hoàn chỉnh có thể thêm intent, preference, catalog, ranker, inventory và policy. Tuy nhiên, số lượng vai trò nên tăng khi có trách nhiệm rõ. Với dữ liệu hiện tại, preference chưa có nguồn người dùng, vì vậy không tạo hồ sơ giả rồi gọi là cá nhân hóa. Score giá tăng là một baseline minh bạch.

Một bộ dữ liệu thật có thể đánh giá retrieval Recall@k, ranking NDCG@k và tỷ lệ vi phạm điều kiện. Các chỉ số này cần relevance label hoặc ground truth phù hợp. Bộ benchmark hiện dùng ID mong đợi và status, không báo NDCG vì chưa có nhãn graded relevance. Phân biệt chỉ số chưa đo giúp báo cáo giữ đúng phạm vi bằng chứng.

Gợi ý triển khai tiếp: thêm catalog adapter chỉ đọc, snapshot có version và stock TTL theo loại hàng. Trước khi đánh giá, đóng băng tập yêu cầu và catalog snapshot để mọi kiến trúc thấy cùng dữ liệu. Khi dùng thông tin online, cần ghi timestamp để biết sai do mô hình hay do dữ liệu thay đổi.

14 Unit 13 — Case study du lịch

Một tourism MAS có thể gồm preference, attraction, route và budget agent. Trạng thái lịch trình chứa điểm đã chọn, thời điểm đến, thời gian tham quan, chi phí và ràng buộc. RouteAgent phải tính cả di chuyển, không chỉ cộng thời lượng tham quan. BudgetAgent phải bao gồm phí và vận chuyển, không chỉ vé vào cửa.

Ví dụ tổng hợp: người dùng có 6 giờ và ngân sách 500 đơn vị; ba điểm cần lần lượt 120, 90, 150 phút; tổng di chuyển 80 phút. Chọn cả ba cần 440 phút, vượt 360 phút dù từng điểm riêng lẻ đều phù hợp. Coordinator phải bỏ điểm hoặc xin thay đổi ưu tiên mềm, không tạo lịch trình bất khả thi.

Bộ mã hiện không triển khai tourism planner và không dùng thông tin giờ mở cửa thực tế. Đây là bài thiết kế chuyển miền hoàn chỉnh về vai trò và ràng buộc, phù hợp với Unit 13; phần thực thi năm exercise được chọn ở e-commerce. Nếu phát triển planner thật, dữ liệu mở cửa và thời gian di chuyển phải lấy từ nguồn hiện hành có thể kiểm tra.

15 Unit 14 — Case study hỗ trợ hành chính y tế

Phạm vi phù hợp cho prototype sinh viên là chuẩn bị giấy tờ, hướng dẫn quầy tiếp nhận và hỗ trợ lịch hẹn. IntakeAgent lấy yêu cầu; InformationAgent chỉ đọc nguồn đã duyệt; SchedulingAgent đọc lịch; PreparationAgent tạo danh sách việc cần chuẩn bị; Coordinator quản lý quyền và xác nhận thao tác nếu có thay đổi lịch thật.

Bài này không triển khai phân loại cấp cứu, chẩn đoán hoặc điều trị. Lớp hỗ trợ hành chính y tế trong classifier chỉ là nhãn tác vụ hành chính tổng hợp. Do vậy accuracy của lớp này không được gọi là độ chính xác y khoa. Từ "healthcare" trong một mô hình routing không tự mang ý nghĩa clinical validation.

Đánh giá hành chính nên kiểm tính đúng thông tin, độ đầy đủ và bảo vệ dữ liệu. Khi không có nguồn xác nhận, hệ phải trình bày chưa biết. Cách xử lý inventory_unverified ở e-commerce minh họa nguyên tắc tương tự, nhưng không đủ để chứng nhận an toàn cho miền y tế.

16 Unit 15 — So sánh ba miền và quyền sở hữu ràng buộc

Miền	Dữ liệu có thẩm quyền	Ràng buộc chính	Khi thiếu dữ liệu
E-commerce	Catalog, price, inventory	Budget, ID hợp lệ, stock mới	Không khẳng định availability
Tourism	Điểm đến, giờ mở cửa, thời gian đi	Khả thi thời gian, budget	Trả phương án chưa xác minh hoặc sửa lịch
Healthcare administration	Tài liệu cơ sở và lịch hẹn	Đúng thông tin, quyền, tối thiểu dữ liệu	Dừng thao tác và chuyển kiểm tra phù hợp

Ở cả ba miền, mỗi ràng buộc cần người sở hữu. Nếu "ai cũng kiểm" nhưng không có hợp đồng, có thể không thành viên nào thực hiện đầy đủ. Coordinator nên có một checklist kiểm được bằng máy cho điều kiện bắt buộc, còn các đánh giá mềm để agent chuyên môn cung cấp.

Phân biệt hành động đề xuất với hành động ghi là cần thiết khi chuyển prototype sang ứng dụng. Trong bài hiện tại, mọi actuator chỉ trả kết quả nên có thể chạy tự động. Nếu thêm đặt hàng hoặc thay lịch, phải thiết kế thao tác có khả năng xem trước và xác nhận rõ, cùng idempotency và kiểm tra kết quả dịch vụ.

17 Unit 16 — Thiết kế benchmark có đối chứng

B0 là pipeline một thành phần có cùng kiểm chứng factual. B1 là MAS theo luật có coordinator validation. B2 thêm classifier intent trước MAS. B3 tắt policy/freshness validation để đo ảnh hưởng của việc bỏ kiểm. Đây là bốn cấu hình có mục đích khác nhau; B3 là ablation chẩn đoán, không phải một sản phẩm được đề xuất sử dụng.

Tập đánh giá có 11 ca, mỗi ca lặp 20 lần, tổng 220 yêu cầu mỗi kiến trúc. Các ca gồm bình thường, query không biết, query rỗng, budget 0, hết hàng, budget âm, timeout, malformed, stale và forged. Repetition dùng để ước lượng latency cục bộ; nó không biến 11 ca thành 220 tác vụ độc lập về nội dung.

Success yêu cầu đồng thời status và danh sách ID bằng ground truth. Violation kiểm item ngoài catalog, vượt budget hoặc khẳng định availability từ dữ liệu stale. B0/B1/B2 đạt đủ các ca; B3 chỉ đạt 8/11 loại ca và có vi phạm. Số đo cụ thể được chèn từ CSV ở phần kết quả, không viết tay.

Có hai giới hạn so sánh quan trọng. B0 mô phỏng trạng thái timeout đồng bộ nên không chờ deadline thật; không thể dùng P95 của nó để kết luận một hệ dịch vụ đơn luôn nhanh hơn MAS cùng tỷ lệ đó. B2 dùng cùng một câu mua laptop để định tuyến trước mọi shopping case; benchmark này đo overhead tích hợp classifier, **không** đo routing đa miền. Chất lượng routing được đánh giá riêng trên tập 108 câu và OOD.

18 Unit 17 — Hoàn thành năm bài thực hành

18.1 Exercise 1: Đặc tả tác tử và PEAS

Ứng dụng chọn là shopping assistant. Bảng đặc tả ở Unit 3 có bốn worker và coordinator, đầy đủ input, output, quyền và tiêu chí thành công. PEAS bao gồm điều kiện từ chối đúng, không chỉ số lượng sản phẩm. Sơ đồ Unit 5 thể hiện nguồn quyết định và thứ tự truyền dữ liệu. Đó là bằng chứng cho yêu cầu tối thiểu ba agent.

18.2 Exercise 2: Mở rộng shopping system

Tệp `slide_baseline.py` chép lại luồng bốn sản phẩm từ Unit 7–9 và đã chạy năm kiểm tra: truy hồi tag, không vượt budget, unknown query, empty query và chỉ trả candidate. Output

được lưu trong `slide_baseline.json`. Từ baseline hai agent đó, bản mở rộng bổ sung inventory và policy; B0 trong benchmark là một baseline đơn có cùng điều kiện kiểm chứng để so công bằng, khác bản slide ban đầu. Policy kiểm ID nằm trong catalog/candidate và không vượt budget; inventory kiểm stock; coordinator kiểm response envelope và TTL. Kịch bản portable/100 chứng minh không trả laptop vượt ngân sách, còn sports/100 chứng minh không trả sản phẩm stock 0.

Để tự mở rộng, thêm PreferenceAgent nhận danh sách ID cùng sở thích có schema. Hãy giữ budget ở lớp filter và dùng sở thích chỉ xếp hạng trong miền hợp lệ. Viết test rằng một sản phẩm có score cao nhưng vượt budget vẫn không xuất hiện. Đây là cách kiểm tra integration, thay vì chỉ thử riêng hàm chấm điểm.

18.3 Exercise 3: Huấn luyện intent classifier

`train_intent.py` tạo dữ liệu, chia nhóm, huấn luyện, chọn checkpoint, chọn threshold và xuất per-class metrics. `intent_model.pt` chứa state_dict, vocabulary và threshold. Không tải một checkpoint không rõ nguồn. Sau chạy, xem confusion matrix, loss train/validation và các câu OOD, rồi giải thích vì sao kết quả 100% vẫn không đủ cho triển khai.

Bài đã có accuracy, precision, recall và F1 từng lớp trên test, đúng yêu cầu của slide. Phần confidence threshold là cơ chế abstention có thể chạy, đồng thời báo cáo lỗi false acceptance. Muốn cải thiện nên tạo tập dữ liệu phong phú hơn và giữ test độc lập sau khi chỉnh quy tắc.

18.4 Exercise 4: Failure injection

Bốn lỗi bắt buộc là timeout, empty result, malformed message và stale availability đều có code và test. Bổ sung forged recommendation, sai correlation ID, budget NaN và budget âm. Người học có thể chạy `test_tutorial.py` để xem ca nào thất bại khi cố ý bỏ validation.

Ví dụ thực hành: đổi `Coordinator(validate=False)` rồi chạy stale và forged. Kết quả vẫn có thể mang status ok nhưng vi phạm factual. Sau đó bật validation, quan sát availability_unverified hoặc contract_error. Bài tập cho thấy đánh giá an toàn phải xét nội dung và provenance, không chỉ việc hàm trả về mà không ném exception.

18.5 Exercise 5: So sánh kiến trúc

Bảng benchmark đi kèm cho thấy B0 và B1 có cùng tỷ lệ success trên bộ ca. Vì vậy kết quả **không chứng minh chia nhiều agent tăng chất lượng** ở catalog toy. B1 có thêm message và latency; lợi ích là ranh giới trách nhiệm, trace và khả năng thay thành phần. B3 cho thấy bỏ kiểm chứng làm giảm chất lượng dù vẫn có nhiều agent.

Một so sánh tiếp theo nên tăng quy mô dữ liệu và độc lập nguồn lỗi, giữ cùng tác vụ, lặp theo nhiều tải và dùng cùng cơ chế timeout cho cả B0/B1. Nếu muốn chứng minh lợi ích chuyên môn hóa, cần tác vụ mà các agent có dữ liệu hoặc công cụ khác nhau thực sự, và baseline đơn được cấp nguồn lực tương xứng.

19 Unit 18 — Kiểm thử, rubric và lỗi thường gặp

Có 17 test tự động cho tutorial, bao gồm đường bình thường, ngân sách, empty/unknown, hết hàng, input sai, timeout, message malformed, stale, forged, correlation, trace, chia nhóm dữ liệu, OOD đơn giản và tương đương baseline. Test cuối đối chiếu 20 tổ hợp query/budget bình thường. Các test lỗi kiểm hành vi tại ranh giới, không chỉ gọi lại công thức trong implementation.

Rubric của slide được đối chiếu bằng sản phẩm: PEAS và vai trò có trong báo cáo; nguồn Python và README có sẵn; timeout thực và validation có test; classifier đã huấn luyện; benchmark có dữ liệu thô; phạm vi synthetic được ghi rõ; demo và slide trình bày có kèm. Các yêu cầu này được gắn vào manifest để người đọc kiểm tra nhanh.

Những lỗi cần tránh: gọi mạng chưa train là predictor hợp lệ; dùng cùng tập để chọn ngưỡng và báo test; lấy output giả làm bằng chứng chạy; gọi nhiều hàm là chuyên gia mà không có trách nhiệm; xem response ok là thành công dù có ID bịa; và suy ra readiness triển khai từ vài dữ liệu tổng hợp.

20 Unit 19 — Kiến trúc tham chiếu và hướng phát triển

Nếu mở rộng thành dịch vụ, nên thêm adapter dữ liệu có version, cấu hình timeout theo loại công cụ, retry cho thao tác đọc có giới hạn và logging không chứa thông tin nhạy cảm. Những thao tác ghi cần idempotency key, đọc lại xác nhận và mô hình trạng thái giao dịch. Không nên tự thêm một vòng retry vô hạn khi không hiểu nguyên nhân lỗi.

Nếu mở rộng learned component, nên bắt đầu bằng ranking dataset có nhãn và đánh giá riêng retrieval/ranking. Giữ invariant kiểm được bằng máy ở coordinator. Nếu thêm LLM, coi output là proposal và giữ validator; lưu model/prompt version; đo số token, độ trễ, groundedness và tỷ lệ lỗi theo nhóm yêu cầu.

Đối với dữ liệu intent, bước cải tiến ưu tiên là dữ liệu tự nhiên, từ vựng chưa thấy, câu đa ý định, phủ định, thiếu thông tin và ngoài miền. So sánh classifier với keyword baseline trên cùng dữ liệu. Không nên tăng mạng sâu trước khi kiểm tra bộ dữ liệu có thực sự yêu cầu năng lực đó hay chưa.

21 Unit 20 — Tổng kết, demo và câu hỏi bảo vệ

Một bản demo 5–7 phút có thể đi qua sáu bước: giới thiệu PEAS; chạy portable/100; mở trace; thử timeout và stale; trình bày confusion matrix cùng lỗi OOD; kết thúc bằng bảng B0–B3. Đây là trình tự cho thấy cả cơ chế và giới hạn, không chỉ một output thành công được chọn sẵn.

Câu hỏi bảo vệ gợi ý: vì sao dùng MAS ở catalog nhỏ; timeout nào thực sự được áp dụng; vì sao softmax cao vẫn sai; classifier có phải RL không; B3 đo điều gì; vì sao baseline đơn vẫn cần kiểm kho; và dữ liệu nào sẽ cần thêm trước khi dùng thật. Các câu trả lời phải gắn với code, dữ liệu và protocol đã công bố.

Thông điệp kỹ thuật của tutorial là xây một hệ có hợp đồng, kiểm chứng và bằng chứng đo được. Phần học máy đóng góp một thành phần dự đoán đã train và có giới hạn được đo; phần điều phối giữ các điều kiện quan trọng bất kể dự đoán. Cả hai cần được kiểm tra bằng cùng tiêu chí nhiệm vụ.

22 Tutorial thực hành: xây dựng và chạy hệ đa tác tử bằng Python

Phần này trình bày thực hành trực tiếp theo năm bài tập của tutorial: chạy hệ mua hàng cơ bản, mở rộng thành hệ đa tác tử, huấn luyện mô hình phân loại ý định, đưa lỗi vào hệ thống và so sánh các kiến trúc. Toàn bộ mã nguồn thực thi của sáu tệp tutorial nằm trong phụ lục mã ngay sau phần kết quả; mã gốc có dấu tiếng Việt được cung cấp kèm để chạy lại. Các đầu ra dưới đây được lấy từ chương trình đã chạy, không điền thủ công.

22.1 Chuẩn bị và tổ chức tệp

Mở terminal tại thư mục `solution`, cài các thư viện và thực hiện lần lượt:

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
python3 tutorial/slide_baseline.py
python3 tutorial/mas_system.py
python3 tutorial/train_intent.py
python3 tutorial/thuc_hanh_tong_hop.py
python3 tutorial/evaluate_mas.py
python3 -m unittest discover -s tutorial -p 'test_*.py' -v
```

Hệ mua hàng cơ bản dùng thư viện chuẩn Python; PyTorch và Matplotlib cần cho mô hình và biểu đồ. Tệp `train_intent.py` tạo checkpoint và dữ liệu đánh giá; có thể dùng checkpoint đã cung cấp để chạy ngay ví dụ tổng hợp. Dữ liệu sản phẩm, kho và câu yêu cầu là dữ liệu tạo phục vụ bài học.

22.2 Bài 1: định nghĩa hợp đồng thông điệp và môi trường

Một thông điệp có mã yêu cầu, người gửi, người nhận, loại và payload. `CatalogAgent`, `RecommendationAgent`, `PolicyAgent` và `InventoryAgent` có chức năng riêng; coordinator kiểm tra người gửi, loại phản hồi và task ID. Mã sau lấy trực tiếp từ tệp đã thực thi:

Listing 1: Dữ liệu sản phẩm và hợp đồng Message trong hệ thực hành

```
@dataclass(frozen=True)
class Product:
    id: str
    name: str
    category: str
    price: float
    tags: tuple[str,...]

PRODUCTS=(Product('p1','Lightweight laptop','electronics',850,('portable','study')),
 Product('p2','Wireless mouse','electronics',25,('portable','accessory')),
 Product('p3','Running shoes','sports',70,('fitness','outdoor')),
 Product('p4','Travel backpack','travel',55,('portable','travel')),
```

```
 Product('p5','Study tablet','electronics',150,('study','portable')))
STOCK={'p1':2,'p2':5,'p3':0,'p4':3,'p5':1}

@dataclass(frozen=True)
class Message:
    task_id: str
    sender: str
    receiver: str
    kind: str
    payload: dict[str,Any]
    def validate(self,receiver,kind):
        if not self.task_id or self.receiver!=receiver or self.kind!=kind or not isinstance(self.payload,
     dict):
            raise ValueError('Invalid message envelope')

class CatalogAgent:
```

Để kiểm tra, tìm `portable` với budget 100: sản phẩm p2 và p4 hợp lệ; p1 và p5 vượt budget. Nếu query rỗng, catalog không trả tất cả sản phẩm. Nếu kho không xác minh được, coordinator không khẳng định còn hàng.

22.3 Bài 2: coordinator, luật và xác minh tồn kho

Luồng hoạt động là SEARCH, RANK, VALIDATE rồi CHECK. Việc kiểm tra policy xác nhận ID thuộc catalog và tập kết quả truy hồi, đồng thời không vượt ngân sách. Inventory trả số lượng và tuổi dữ liệu; quá 60 giây được xem là chưa xác minh. Dưới đây là phần cốt lõi, bản đầy đủ có trong phụ lục:

Listing 2: PolicyAgent và luồng thực thi có kiểm hợp đồng, timeout và độ mới

```
    name='policy'
    async def handle(self,m):
        m.validate(self.name,'VALIDATE');lookup={p.id:p for p in PRODUCTS}
        ids=m.payload['ids'];allowed=set(m.payload['candidates']);budget=m.payload['budget']
        if any(i not in lookup or i not in allowed or lookup[i].price>budget for i in ids):
            raise ValueError('Candidate provenance/budget violation')
        return Message(m.task_id,self.name,m.sender,'VALIDATED',{'ids':ids})

class Coordinator:
    def __init__(self,validate=True,deadline=.01):
        self.catalog=CatalogAgent();self.recommender=RecommendationAgent();self.inventory=InventoryAgent()
        self.policy=PolicyAgent();self.validate=validate;self.deadline=deadline;self.trace=[]
    async def call(self,agent,m,expected):
        start=time.perf_counter(); status='error'
        try:
            result=await asyncio.wait_for(agent.handle(m),timeout=self.deadline)
            if not isinstance(result,Message):raise TypeError('Malformed response')
            if result.task_id!=m.task_id or result.receiver!=m.sender or result.sender!=agent.name or result
     .kind!=expected:
                raise ValueError('Response correlation/type violation')
            status='ok';return result
        except (TimeoutError,asyncio.TimeoutError,ValueError,TypeError,KeyError):
            status='error';raise
        finally:
            self.trace.append({'task_id':m.task_id,'agent':agent.name,'kind':m.kind,
                'status':status,'milliseconds':(time.perf_counter()-start)*1000})
    async def shop(self,query,budget,scenario='normal',task_id='demo'):
        self.trace=[]
```

```
        if not isinstance(query,str) or not isinstance(budget,(int,float)) or not math.isfinite(budget) or budget<0:
            return {'status':'invalid_input','items':[]}
        def msg(receiver,kind,payload):return Message(task_id,'coordinator',receiver,kind,payload)
        try:
            c=await self.call(self.catalog,msg('catalog','SEARCH',{'query':query,'scenario':scenario}),'CANDIDATES')
            candidates=c.payload['ids']
            if not candidates:return {'status':'no_results','items':[]}
            r=await self.call(self.recommender,msg('recommender','RANK',{'ids':candidates,'budget':budget}),'RANKED')
            ids=r.payload['ids']
            # Deliberate adversarial component output, to test coordinator validation.
            if scenario=='forged':ids=['p99','p1']+ids
            if self.validate:
                p=await self.call(self.policy,msg('policy','VALIDATE',{'ids':ids,'candidates':candidates,'budget':budget}),'VALIDATED')
                ids=p.payload['ids']
            inv=await self.call(self.inventory,msg('inventory','CHECK',{'ids':ids,'scenario':scenario}),'AVAILABILITY')
            if self.validate and (not isinstance(inv.payload.get('age_seconds'),(int,float)) or not 0<=inv.payload['age_seconds']<=60):
                return {'status':'availability_unverified','items':[]}
            stocks=inv.payload['stock']
            if self.validate and (set(stocks)!=set(ids) or any(type(n) is not int or n<0 for n in stocks.values())):
                raise ValueError('Invalid stock schema')
            lookup={p.id:p for p in PRODUCTS}
            items=[{'id':i,'price':lookup[i].price if i in lookup else 0,'availability':'verified'}
                   for i in ids if stocks.get(i,0)>0 or not self.validate]
            return {'status':'ok' if items else 'no_results','items':items}
        except (TimeoutError,asyncio.TimeoutError):return {'status':'timeout','items':[]}
        except (ValueError,TypeError,KeyError):return {'status':'contract_error','items':[]}
```

22.4 Bài 3: huấn luyện PyTorch và đánh giá

Tập dữ liệu gồm 432 câu tổng hợp thuộc ba lớp. Các nhóm động từ được chia thành 270 câu train, 54 validation và 108 test; vocabulary chỉ xây từ train. Mạng gồm lớp tuyến tính, ReLU và lớp logits; CrossEntropyLoss nhận logits và nhãn. Validation được dùng để chọn checkpoint và ngưỡng, sau đó test để báo precision và recall theo lớp.

Đoạn mã sau là phần định nghĩa mạng và chuyển câu thành vector. Vòng huấn luyện, việc chọn checkpoint, threshold và test được in đầy đủ trong phụ lục tệp `train_intent.py`.

Listing 3: Mạng PyTorch và biểu diễn bag-of-words

```
class IntentNet(nn.Module):
    def __init__(self,n):super().__init__();self.net=nn.Sequential(nn.Linear(n,32),nn.ReLU(),nn.Linear(32,3))
    def forward(self,x):return self.net(x)

def features(texts,vocab):
    x=torch.zeros((len(texts),len(vocab)))
    for i,text in enumerate(texts):
        for token in set(normalize(text)):
            if token in vocab:x[i,vocab[token]]=1
    return x
```

Độ chính xác trên test tổng hợp đạt 108/108. Đây là bộ test với đối tượng chia sẻ giữa các split. Các câu ngoài miền được thử riêng cho thấy mô hình vẫn có thể tự tin khi định tuyến sai; vì vậy kết quả không chứng minh mô hình đã hiểu tốt mọi yêu cầu tự nhiên.

22.5 Bài 4: chạy các ca bình thường và đưa lỗi vào hệ thống

Tệp `thuc_hanh_tong_hop.py` gọi trực tiếp chương trình, mô hình đã huấn luyện và phép so sánh, rồi in kết quả. Tệp mã hoàn chỉnh cũng nằm trong phụ lục. Kết quả thực tế của lệnh `python3 tutorial/thuc_hanh_tong_hop.py`:

Listing 4: Output thật: shopping MAS, classifier và so sánh baseline

```
EXERCISE 2 - SHOPPING MAS
query='portable' budget=100 scenario=normal -> {"status": "ok", "items": [{"id": "p2", "price": 25, "
    availability": "verified"}, {"id": "p4", "price": 55, "availability": "verified"}]}
query='sports' budget=100 scenario=normal -> {"status": "no_results", "items": []}
query='' budget=100 scenario=normal -> {"status": "no_results", "items": []}
query='portable' budget=100 scenario=timeout -> {"status": "timeout", "items": []}
query='portable' budget=100 scenario=malformed -> {"status": "contract_error", "items": []}
query='portable' budget=100 scenario=stale -> {"status": "availability_unverified", "items": []}
query='portable' budget=100 scenario=forged -> {"status": "contract_error", "items": []}

EXERCISE 3 - TRAINED INTENT CLASSIFIER
test_count=108 accuracy=1.000 macro_f1=1.000 threshold=0.50
ecommerce        precision=1.000 recall=1.000
tourism          precision=1.000 recall=1.000
healthcare_admin precision=1.000 recall=1.000
input='xin ch\xe0o' -> intent=clarify confidence=0.494
input='th\u1eddi ti\u1ebft h\xf4m nay th\u1ebf n\xe0o' -> intent=ecommerce confidence=0.988
input='gi\u1ea3i ph\u01b0\u01a1ng tr\xecnh' -> intent=tourism confidence=0.931

EXERCISE 5 - BASELINE VS MAS
single_agent: {"status": "ok", "items": [{"id": "p2", "price": 25, "availability": "verified"}, {"id": "p4",
     "price": 55, "availability": "verified"}]}
validated_mas: {"status": "ok", "items": [{"id": "p2", "price": 25, "availability": "verified"}, {"id": "p4
    ", "price": 55, "availability": "verified"}]}
same_result: True
without_validation_stale: {"status": "ok", "items": [{"id": "p2", "price": 25, "availability": "verified"},
    {"id": "p4", "price": 55, "availability": "verified"}]}
```

Quan sát: timeout, malformed và forged trả trạng thái lỗi; trường hợp kho cũ được báo là chưa xác minh. Khi tắt kiểm chứng, kịch bản stale vẫn đưa ra kết luận verified; đây là chủ đích của ablation và cho thấy vì sao cần kiểm độ mới. Câu ngoài miền bị phân loại nhầm được giữ nguyên trong kết quả để minh họa giới hạn của ngưỡng confidence.

22.6 Bài 5: so sánh kiến trúc và chạy kiểm thử

Đầu ra của phép so sánh trong chương trình gồm một pipeline đơn, MAS có kiểm chứng và phiên bản bỏ validation. Benchmark đầy đủ dùng 11 tình huống lặp 20 lần cho mỗi kiến trúc, dữ liệu trong `results/tutorial/mas_benchmark_raw.csv`. Pipeline đơn và MAS có kiểm chứng đều xử lý đúng 11 ca trong tập toy. Việc chia vai tạo thêm trace và overhead; bảng benchmark không chứng minh chất lượng tăng nhờ nhiều agent.

Lệnh kiểm thử thực tế đã chạy thành công trên 17 ca, gồm đầu vào rỗng, quá ngắn

sách, timeout, message sai, kho cũ, ID giả, correlation ID và dữ liệu ngoài miền:

Listing 5: Kết quả thật của bộ kiểm thử tutorial

```
test_budget (test_tutorial.TutorialTests) ... ok
test_correlation (test_tutorial.TutorialTests) ... ok
test_empty (test_tutorial.TutorialTests) ... ok
test_forged (test_tutorial.TutorialTests) ... ok
test_malformed (test_tutorial.TutorialTests) ... ok
test_message_receiver (test_tutorial.TutorialTests) ... ok
test_nan_budget (test_tutorial.TutorialTests) ... ok
test_negative_budget (test_tutorial.TutorialTests) ... ok
test_normal_equivalence (test_tutorial.TutorialTests) ... ok
test_ood_abstention (test_tutorial.TutorialTests) ... ok
test_out_of_stock (test_tutorial.TutorialTests) ... ok
test_split_groups (test_tutorial.TutorialTests) ... ok
test_stale (test_tutorial.TutorialTests) ... ok
test_tag_retrieval (test_tutorial.TutorialTests) ... ok
test_timeout (test_tutorial.TutorialTests) ... ok
test_trace (test_tutorial.TutorialTests) ... ok
test_unknown (test_tutorial.TutorialTests) ... ok

----------------------------------------------------------------------
Ran 17 tests in 0.091s

OK
```

23 Phụ lục mã nguồn đầy đủ của tutorial

Các tệp bên dưới được trích toàn bộ từ chương trình đã chạy, kèm số dòng. Để pdfLaTeX hiển thị ổn định, các ký tự không ASCII trong `train_intent.py` và tệp demo được biểu diễn bằng escape Python `\uXXXX`; kiểm tra cú pháp cho thấy biểu diễn này tương đương mã gốc. Mã gốc và notebook có output được cung cấp cùng báo cáo. Khi chạy, dùng các tệp `tutorial/*.py` từ thư mục mã; những đoạn trong PDF được in để người đọc có thể xem toàn bộ quá trình triển khai.

23.1 Mã gốc: hệ cơ bản theo slide

Chương trình gốc của Unit 7–9 có hai agent và coordinator. Nó chạy năm kiểm tra đầu vào, budget và provenance.

Listing 6: slide_baseline.py – baseline theo slide

```
"""Executable transcription of Units 7-9 of the supplied teaching slide.
This baseline intentionally has no inventory/freshness/timeout controls.
The extended implementation is in mas_system.py.
"""
from dataclasses import dataclass
from typing import Any
from pathlib import Path
import json

@dataclass
class Message:
    sender: str
    receiver: str
```

```
    kind: str
    payload: dict[str, Any]

class Agent:
    def __init__(self,name):self.name=name
    def handle(self,message):raise NotImplementedError

PRODUCTS=[{'id':'p1','name':'Lightweight laptop','category':'electronics','price':850,'tags':['portable','study']},
          {'id':'p2','name':'Wireless mouse','category':'electronics','price':25,'tags':['portable','accessory']},
          {'id':'p3','name':'Running shoes','category':'sports','price':70,'tags':['fitness','outdoor']},
          {'id':'p4','name':'Travel backpack','category':'travel','price':55,'tags':['portable','travel']}]

class CatalogAgent(Agent):
    def __init__(self,products):super().__init__('catalog');self.products=products
    def handle(self,message):
        query=message.payload.get('query','').strip().lower()
        def matches(item):
            fields=[item['name'],item['category']]+item.get('tags',[])
            return any(query in str(value).lower() for value in fields)
        items=[item for item in self.products if query and matches(item)]
        return Message(self.name,message.sender,'SEARCH_RESULTS',{'items':items})

class RecommendationAgent(Agent):
    def __init__(self):super().__init__('recommender')
    def handle(self,message):
        budget=float(message.payload.get('budget',0));items=message.payload.get('items',[])
        candidates=[item for item in items if item['price']<=budget]
        ranked=sorted(candidates,key=lambda item:('portable' not in item['tags'],item['price']))
        return Message(self.name,message.sender,'RECOMMENDATIONS',{'items':ranked[:3]})

class Coordinator(Agent):
    def __init__(self,catalog,recommender):super().__init__('coordinator');self.catalog=catalog;self.recommender=recommender
    def shop(self,query,budget):
        search=Message(self.name,'catalog','SEARCH',{'query':query})
        found=self.catalog.handle(search)
        rank=Message(self.name,'recommender','RANK',{'items':found.payload['items'],'budget':budget})
        return self.recommender.handle(rank).payload['items']

def run_baseline():
    system=Coordinator(CatalogAgent(PRODUCTS),RecommendationAgent())
    items=system.shop('portable',100)
    assert [p['id'] for p in items]==['p2','p4']
    assert all(p['price']<=100 for p in items)
    assert not system.shop('unknown-query',100)
    assert not system.shop('',100)
    candidates={p['id'] for p in CatalogAgent(PRODUCTS).handle(Message('x','catalog','SEARCH',{'query':'portable'})).payload['items']}
    assert all(p['id'] in candidates for p in items)
    result={'query':'portable','budget':100,'items':items,'checks_passed':5,'source':'Teaching slide Units 7-9; original four-product baseline'}
    out=Path(__file__).resolve().parents[1]/'results/tutorial/slide_baseline.json'
    out.write_text(json.dumps(result,ensure_ascii=False,indent=2));print(json.dumps(result,indent=2));return result
if __name__=='__main__':run_baseline()
```

23.2 Mã mở rộng: shopping multi-agent system

Tệp này chứa dữ liệu, hợp đồng message, bốn agent, coordinator, baseline đơn và demo lỗi.

Listing 7: mas_system.py – hệ đa tác tử đầy đủ

```
"""Local teaching MAS. All catalog/inventory data are synthetic, prices in toy units."""
from __future__ import annotations
import asyncio, math, time
from dataclasses import dataclass, asdict
from typing import Any

@dataclass(frozen=True)
class Product:
    id: str
    name: str
    category: str
    price: float
    tags: tuple[str,...]

PRODUCTS=(Product('p1','Lightweight laptop','electronics',850,('portable','study')),
 Product('p2','Wireless mouse','electronics',25,('portable','accessory')),
 Product('p3','Running shoes','sports',70,('fitness','outdoor')),
 Product('p4','Travel backpack','travel',55,('portable','travel')),
 Product('p5','Study tablet','electronics',150,('study','portable')))
STOCK={'p1':2,'p2':5,'p3':0,'p4':3,'p5':1}

@dataclass(frozen=True)
class Message:
    task_id: str
    sender: str
    receiver: str
    kind: str
    payload: dict[str,Any]
    def validate(self,receiver,kind):
        if not self.task_id or self.receiver!=receiver or self.kind!=kind or not isinstance(self.payload,
     dict):
            raise ValueError('Invalid message envelope')

class CatalogAgent:
    name='catalog'
    async def handle(self,m):
        m.validate(self.name,'SEARCH');q=m.payload['query'].strip().lower()
        ids=[p.id for p in PRODUCTS if q and any(q in s.lower() for s in (p.name,p.category,*p.tags))]
        if m.payload.get('scenario')=='forged':ids+=['p99','p1']
        return Message(m.task_id,self.name,m.sender,'CANDIDATES',{'ids':ids})

class RecommendationAgent:
    name='recommender'
    async def handle(self,m):
        m.validate(self.name,'RANK'); products={p.id:p for p in PRODUCTS}
        ids=m.payload['ids'];budget=m.payload['budget']
        ranked=sorted((i for i in ids if i in products and products[i].price<=budget),key=lambda i:products[
     i].price)
        return Message(m.task_id,self.name,m.sender,'RANKED',{'ids':ranked[:3]})

class InventoryAgent:
    name='inventory'
    async def handle(self,m):
        m.validate(self.name,'CHECK');scenario=m.payload.get('scenario','normal')
        if scenario=='timeout':await asyncio.sleep(.05)
        if scenario=='malformed':return {'ids':m.payload['ids']}
```

```
        age=3600. if scenario=='stale' else 0.
        return Message(m.task_id,self.name,m.sender,'AVAILABILITY',
            {'stock':{i:STOCK.get(i,0) for i in m.payload['ids']},'age_seconds':age})

class PolicyAgent:
    name='policy'
    async def handle(self,m):
        m.validate(self.name,'VALIDATE');lookup={p.id:p for p in PRODUCTS}
        ids=m.payload['ids'];allowed=set(m.payload['candidates']);budget=m.payload['budget']
        if any(i not in lookup or i not in allowed or lookup[i].price>budget for i in ids):
            raise ValueError('Candidate provenance/budget violation')
        return Message(m.task_id,self.name,m.sender,'VALIDATED',{'ids':ids})

class Coordinator:
    def __init__(self,validate=True,deadline=.01):
        self.catalog=CatalogAgent();self.recommender=RecommendationAgent();self.inventory=InventoryAgent()
        self.policy=PolicyAgent();self.validate=validate;self.deadline=deadline;self.trace=[]
    async def call(self,agent,m,expected):
        start=time.perf_counter(); status='error'
        try:
            result=await asyncio.wait_for(agent.handle(m),timeout=self.deadline)
            if not isinstance(result,Message):raise TypeError('Malformed response')
            if result.task_id!=m.task_id or result.receiver!=m.sender or result.sender!=agent.name or result
    .kind!=expected:
                raise ValueError('Response correlation/type violation')
            status='ok';return result
        except (TimeoutError,asyncio.TimeoutError,ValueError,TypeError,KeyError):
            status='error';raise
        finally:
            self.trace.append({'task_id':m.task_id,'agent':agent.name,'kind':m.kind,
                'status':status,'milliseconds':(time.perf_counter()-start)*1000})
    async def shop(self,query,budget,scenario='normal',task_id='demo'):
        self.trace=[]
        if not isinstance(query,str) or not isinstance(budget,(int,float)) or not math.isfinite(budget) or
    budget<0:
            return {'status':'invalid_input','items':[]}
        def msg(receiver,kind,payload):return Message(task_id,'coordinator',receiver,kind,payload)
        try:
            c=await self.call(self.catalog,msg('catalog','SEARCH',{'query':query,'scenario':scenario}),'
    CANDIDATES')
            candidates=c.payload['ids']
            if not candidates:return {'status':'no_results','items':[]}
            r=await self.call(self.recommender,msg('recommender','RANK',{'ids':candidates,'budget':budget}),
    'RANKED')
            ids=r.payload['ids']
            # Deliberate adversarial component output, to test coordinator validation.
            if scenario=='forged':ids=['p99','p1']+ids
            if self.validate:
                p=await self.call(self.policy,msg('policy','VALIDATE',{'ids':ids,'candidates':candidates,'
    budget':budget}),'VALIDATED')
                ids=p.payload['ids']
            inv=await self.call(self.inventory,msg('inventory','CHECK',{'ids':ids,'scenario':scenario}),'
    AVAILABILITY')
            if self.validate and (not isinstance(inv.payload.get('age_seconds'),(int,float)) or not 0<=inv.
    payload['age_seconds']<=60):
                return {'status':'availability_unverified','items':[]}
            stocks=inv.payload['stock']
            if self.validate and (set(stocks)!=set(ids) or any(type(n) is not int or n<0 for n in stocks.
    values())):
                raise ValueError('Invalid stock schema')
            lookup={p.id:p for p in PRODUCTS}
            items=[{'id':i,'price':lookup[i].price if i in lookup else 0,'availability':'verified'}
```

```
                    for i in ids if stocks.get(i,0)>0 or not self.validate]
            return {'status':'ok' if items else 'no_results','items':items}
        except (TimeoutError,asyncio.TimeoutError):return {'status':'timeout','items':[]}
        except (ValueError,TypeError,KeyError):return {'status':'contract_error','items':[]}

def single_pipeline(query,budget,scenario='normal'):
    """BO: same factual checks in one component, no messaging."""
    if not isinstance(query,str) or not isinstance(budget,(int,float)) or not math.isfinite(budget) or
     budget<0:
        return {'status':'invalid_input','items':[]}
    q=query.strip().lower()
    candidates=[p for p in PRODUCTS if q and any(q in s.lower() for s in (p.name,p.category,*p.tags))]
    if not candidates:return {'status':'no_results','items':[]}
    eligible=sorted([p for p in candidates if p.price<=budget],key=lambda p:p.price)[:3]
    if scenario=='forged':return {'status':'contract_error','items':[]}
    if scenario=='timeout':return {'status':'timeout','items':[]}
    if scenario=='malformed':return {'status':'contract_error','items':[]}
    if scenario=='stale':return {'status':'availability_unverified','items':[]}
    items=[{'id':p.id,'price':p.price,'availability':'verified'} for p in eligible if STOCK[p.id]>0]
    return {'status':'ok' if items else 'no_results','items':items}

async def demo():
    s=Coordinator()
    for scenario in ['normal','timeout','malformed','stale','forged']:
        result=await s.shop('portable',100,scenario,task_id=scenario)
        print(scenario,result)
    return s
if __name__=='__main__':asyncio.run(demo())
```

23.3 Mã huấn luyện và đánh giá mô hình PyTorch

Tệp tạo dữ liệu tổng hợp, chia nhóm, xây vocabulary từ train, train, chọn checkpoint và threshold bằng validation rồi đánh giá test. Phần router có cơ chế từ chối đơn giản và probe ngoài miền.

Listing 8: train_intent.py – train và đánh giá classifier

```
"""Train a real classifier on a transparent synthetic, group-split Vietnamese dataset.
No raw clinical data; healthcare class means administrative scheduling only.
"""
from pathlib import Path
import csv,json,re,unicodedata
import numpy as np
import torch
from torch import nn
ROOT=Path(__file__).resolve().parents[1]
LABELS=['ecommerce','tourism','healthcare_admin']
VERBS=[['t\u00ecm mua','ch\u1ecdn mua','c\u1ea7n mua','t\u01b0 v\u1ea5n mua','mu\u1ed1n \u0111\u1eb7t mua','
     \u0111\u1ec1 xu\u1ea5t mua','gi\u00fap ch\u1ecdn','g\u1ee3i \u00fd mua'],
 ['l\u00ean l\u1ecbch tham quan','l\u1eadp chuy\u1ebfn \u0111i','t\u00ecm \u0111i\u1ec3m du l\u1ecbch','s\
     u1eafp x\u1ebfp h\u00e0nh tr\u00ecnh','mu\u1ed1n kh\u00e1m ph\u00e1','t\u01b0 v\u1ea5n du l\u1ecbch','g
     \u1ee3i \u00fd tham quan','l\u1eadp l\u1ecbch tr\u00ecnh'],
 ['\u0111\u1eb7t l\u1ecbch kh\u00e1m','h\u1eb9n g\u1eb7p b\u00e1c s\u0129','t\u00ecm l\u1ecbch ph\u00f2ng kh
     \u00e1m','chu\u1ea9n b\u1ecb kh\u00e1m','h\u1ecfi gi\u1edd b\u1ec7nh vi\u1ec7n','xem l\u1ecbch kh\
     u00e1m','h\u01b0\u1edbng d\u1eabn \u0111\u1eb7t h\u1eb9n','s\u1eafp x\u1ebfp kh\u00e1m']]
OBJECTS=[['laptop','chu\u1ed9t m\u00e1y t\u00ednh','ba l\u00f4','gi\u00e0y th\u1ec3 thao','m\u00e1y t\
     u00ednh b\u1ea3ng','tai nghe'],
 ['b\u1ea3o t\u00e0ng','ph\u1ed1 c\u1ed5','b\u00e3i bi\u1ec3n','c\u00f4ng vi\u00ean','khu di t\u00edch','v\
     u01b0\u1eddn th\u1ef1c v\u1eadt'],
 ['qu\u1ea7y ti\u1ebfp nh\u1eadn','th\u1ee7 t\u1ee5c \u0111\u0103ng k\u00fd','gi\u1ea5y t\u1edd b\u1ea3o hi\
```

```
     u1ec3m','l\u1ecbch chuy\u00ean khoa','h\u1ed3 s\u01a1 h\u1eb9n','gi\u1edd ti\u1ebfp nh\u1eadn']]
SUFFIX=['cho t\u00f4i','v\u00e0o ng\u00e0y mai','theo nhu c\u1ea7u c\u1ee7a t\u00f4i']

def normalize(text):
    text=unicodedata.normalize('NFD',text.lower()).replace('\u0111','d')
    return re.findall('[a-z]+',''.join(c for c in text if not unicodedata.combining(c)))

def dataset():
    rows=[]
    for y in range(3):
        for g,verb in enumerate(VERBS[y]):
            split='train' if g<5 else ('validation' if g==5 else 'test')
            for obj in OBJECTS[y]:
                for suffix in SUFFIX:
                    rows.append({'text':f'{verb} {obj} {suffix}','label':y,'class':LABELS[y],
                                 'group':f'{y}:{g}','split':split})
    assert len({r['text'] for r in rows})==len(rows)
    return rows

class IntentNet(nn.Module):
    def __init__(self,n):super().__init__();self.net=nn.Sequential(nn.Linear(n,32),nn.ReLU(),nn.Linear(32,3)
    )
    def forward(self,x):return self.net(x)

def features(texts,vocab):
    x=torch.zeros((len(texts),len(vocab)))
    for i,text in enumerate(texts):
        for token in set(normalize(text)):
            if token in vocab:x[i,vocab[token]]=1
    return x

def metrics(y,p):
    cm=np.zeros((3,3),dtype=int)
    for a,b in zip(y,p):cm[a,b]+=1
    pr=np.diag(cm)/np.maximum(cm.sum(0),1);rc=np.diag(cm)/np.maximum(cm.sum(1),1)
    f=2*pr*rc/np.maximum(pr+rc,1e-12)
    return {'accuracy':float(np.trace(cm)/cm.sum()),'macro_f1':float(f.mean()),'precision':pr.tolist(),
            'recall':rc.tolist(),'f1':f.tolist(),'confusion_matrix':cm.tolist()}

class Router:
    def __init__(self,path=ROOT/'results/tutorial'):
        ck=torch.load(Path(path)/'intent_model.pt',map_location='cpu',weights_only=True)
        self.vocab=ck['vocab'];self.threshold=ck['threshold'];self.model=IntentNet(len(self.vocab))
        self.model.load_state_dict(ck['state_dict']);self.model.eval()
        # This lexicon is defined before inspecting test/OOD outputs.
        domain=[*sum(OBJECTS,[]),*sum(VERBS,[])]
        self.domain_tokens=set(sum([normalize(s) for s in domain],[]))-set(normalize('cho toi vao ngay mai
    theo nhu cau cua toi tim can muon giup goi y'))
    def route(self,text):
        with torch.inference_mode():probs=self.model(features([text],self.vocab)).softmax(1)[0]
        confidence=float(probs.max());label=int(probs.argmax())
        known=bool(set(normalize(text)) & self.domain_tokens)
        return {'intent':LABELS[label] if known and confidence>=self.threshold else 'clarify','confidence':
    confidence}

def run(out=ROOT/'results/tutorial'):
    out=Path(out);out.mkdir(parents=True,exist_ok=True);torch.set_num_threads(1);torch.manual_seed(42)
    rows=dataset();splits={s:[r for r in rows if r['split']==s] for s in ['train','validation','test']}
    vocab={w:i for i,w in enumerate(sorted(set(sum([normalize(r['text']) for r in splits['train']],[]))))}
    data={s:(features([r['text'] for r in rr],vocab),torch.tensor([r['label'] for r in rr])) for s,rr in
    splits.items()}
    model=IntentNet(len(vocab));optim=torch.optim.Adam(model.parameters(),lr=.02);lossfn=nn.CrossEntropyLoss
```

```
    ()
    history=[];best_loss=float('inf');best=None
    for epoch in range(120):
        model.train();optim.zero_grad();loss=lossfn(model(data['train'][0]),data['train'][1]);loss.backward
    ();optim.step()
        model.eval()
        with torch.inference_mode():val=float(lossfn(model(data['validation'][0]),data['validation'][1]))
        history.append({'epoch':epoch+1,'train_loss':float(loss.detach()),'validation_loss':val})
        if val<best_loss:best_loss=val;best={k:v.detach().clone() for k,v in model.state_dict().items()};
    best_epoch=epoch+1
    model.load_state_dict(best);model.eval()
    with torch.inference_mode():
        pv=model(data['validation'][0]).softmax(1).numpy();pt=model(data['test'][0]).softmax(1).numpy()
    # Maximize coverage subject to zero observed validation routing error (small toy set).
    choices=[]
    for threshold in [.5,.6,.7,.8,.9,.95,.99]:
        accept=pv.max(1)>=threshold
        if accept.any() and np.all(pv.argmax(1)[accept]==data['validation'][1].numpy()[accept]):choices.
    append((int(accept.sum()),threshold))
    threshold=max(choices,key=lambda x:(x[0],-x[1]))[1] if choices else 1.
    ck={'state_dict':best,'vocab':vocab,'threshold':threshold};torch.save(ck,out/'intent_model.pt')
    y=data['test'][1].numpy();pred=pt.argmax(1);result=metrics(y,pred);accepted=pt.max(1)>=threshold
    result.update({'splits':{s:len(rr) for s,rr in splits.items()},'threshold':threshold,'selected_epoch':
    best_epoch,
                   'test_coverage':float(accepted.mean()),'selective_accuracy':float(np.mean(y[accepted]==
    pred[accepted])) if accepted.any() else None,
                   'vocabulary_size':len(vocab),'seed':42,'torch':torch.__version__,
                   'limitation':'Synthetic template dataset; group split by verb family, objects shared. Not
     evidence of real-world NLU.'})
    router=Router(out);ood=['xin ch\u00e0o','th\u1eddi ti\u1ebft h\u00f4m nay th\u1ebf n\u00e0o','vi\u1ebft
    th\u01a1 cho t\u00f4i','gi\u1ea3i ph\u01b0\u01a1ng tr\u00ecnh','c\u1ea3m \u01a1n b\u1ea1n','abc xyz']
    result['ood']=[{'text':s,**router.route(s)} for s in ood]
    for name,rr in [('intent_dataset.csv',rows),('intent_training.csv',history),('intent_test_predictions.
    csv',[
        {'text':r['text'],'label':r['label'],'predicted':int(p),'confidence':float(probs.max()),'accepted':
    bool(probs.max()>=threshold)}
        for r,p,probs in zip(splits['test'],pred,pt)])]:
        with open(out/name,'w',newline='',encoding='utf-8') as f:w=csv.DictWriter(f,fieldnames=list(rr[0]));
    w.writeheader();w.writerows(rr)
    (out/'intent_metrics.json').write_text(json.dumps(result,ensure_ascii=False,indent=2))
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    fig,ax=plt.subplots(1,2,figsize=(9,3.5));ax[0].plot([r['train_loss'] for r in history],label='train');ax
    [0].plot([r['validation_loss'] for r in history],label='validation');ax[0].set(xlabel='Epoch',ylabel='
    Cross entropy');ax[0].legend()
    cm=np.array(result['confusion_matrix']);ax[1].imshow(cm,cmap='Blues');ax[1].set(xticks=range(3),yticks=
    range(3),xticklabels=['shop','tourism','admin'],yticklabels=['shop','tourism','admin'],xlabel='
    Prediction',ylabel='True label')
    for i in range(3):
        for j in range(3):ax[1].text(j,i,str(cm[i,j]),ha='center',va='center',color='white' if cm[i,j]>cm.
    max()/2 else 'black')
    fig.tight_layout();fig.savefig(out/'intent_training.png',dpi=160);plt.close(fig)
    print(json.dumps(result,ensure_ascii=False,indent=2));return result
if __name__=='__main__':run()
```

23.4 Mã chạy ví dụ thực hành tổng hợp

Chương trình này tạo output vừa được trình bày và có thể chạy độc lập sau khi đã tạo checkpoint.

Listing 9: thuc_hanh_tong_hop.py – minh họa thực thi đầu cuối

```
"""Exercise the tutorial systems, producing readable real console evidence.
Run from solution/: python3 tutorial/thuc_hanh_tong_hop.py
Uses the trained checkpoint created by train_intent.py and synthetic data.
"""
import asyncio
import json
from pathlib import Path
import torch
from mas_system import Coordinator, single_pipeline
from train_intent import Router, LABELS, dataset, features, metrics

ROOT = Path(__file__).resolve().parents[1]

async def main():
    coordinator = Coordinator(deadline=0.01)
    print('EXERCISE 2 - SHOPPING MAS')
    for query, budget, scenario in [
        ('portable', 100, 'normal'), ('sports', 100, 'normal'),
        ('', 100, 'normal'), ('portable', 100, 'timeout'),
        ('portable', 100, 'malformed'), ('portable', 100, 'stale'),
        ('portable', 100, 'forged'),
    ]:
        response = await coordinator.shop(query, budget, scenario, task_id='demo')
        print('query=%r budget=%s scenario=%s -> %s' %
              (query, budget, scenario, json.dumps(response, ensure_ascii=True)))

    print('\nEXERCISE 3 - TRAINED INTENT CLASSIFIER')
    router = Router(ROOT / 'results/tutorial')
    test = [row for row in dataset() if row['split'] == 'test']
    x = features([row['text'] for row in test], router.vocab)
    with torch.inference_mode():
        probabilities = router.model(x).softmax(1)
    labels = [row['label'] for row in test]
    predicted = probabilities.argmax(1).tolist()
    report = metrics(labels, predicted)
    print('test_count=%d accuracy=%.3f macro_f1=%.3f threshold=%.2f' %
          (len(test), report['accuracy'], report['macro_f1'], router.threshold))
    for i, label in enumerate(LABELS):
        print('%-16s precision=%.3f recall=%.3f' %
              (label, report['precision'][i], report['recall'][i]))
    for text in ['xin ch\u00e0o', 'th\u1eddi ti\u1ebft h\u00f4m nay th\u1ebf n\u00e0o', 'gi\u1ea3i ph\u01b0\
     u01a1ng tr\u00ecnh']:
        result = router.route(text)
        print('input=%s -> intent=%s confidence=%.3f' %
              (ascii(text), result['intent'], result['confidence']))

    print('\nEXERCISE 5 - BASELINE VS MAS')
    expected = single_pipeline('portable', 100)
    actual = await Coordinator().shop('portable', 100)
    print('single_agent:', json.dumps(expected))
    print('validated_mas:', json.dumps(actual))
    print('same_result:', expected == actual)
    unsafe = await Coordinator(validate=False).shop('portable', 100, 'stale')
    print('without_validation_stale:', json.dumps(unsafe))

if __name__ == '__main__':
    torch.set_num_threads(1)
    asyncio.run(main())
```

23.5 Mã benchmark B0–B3

Benchmark dùng cùng danh sách ca, ghi dữ liệu thô và tính success, violation, số message và latency. Các hạn chế về phép đo đã được nêu ở phần kết quả tutorial.

Listing 10: evaluate_mas.py – so sánh kiến trúc

```
from pathlib import Path
import asyncio,csv,json,platform,time
import numpy as np
from mas_system import Coordinator,single_pipeline
from train_intent import Router
ROOT=Path(__file__).resolve().parents[1]
CASES=[('portable',100,'normal','ok',['p2','p4']),('unknown',100,'normal','no_results',[]),
 ('',100,'normal','no_results',[]),('portable',0,'normal','no_results',[]),
 ('sports',100,'normal','no_results',[]),('study',200,'normal','ok',['p5']),
 ('portable',100,'timeout','timeout',[]),('portable',100,'malformed','contract_error',[]),
 ('portable',100,'stale','availability_unverified',[]),('portable',100,'forged','contract_error',[]),
 ('portable',-1,'normal','invalid_input',[])]

async def run(out=ROOT/'results/tutorial',repetitions=20):
    out=Path(out);out.mkdir(parents=True,exist_ok=True);router=Router(out);rows=[];trace=[]
    for system in ['B0_single','B1_rule_MAS','B2_learned_MAS','B3_no_validation']:
        coordinator=Coordinator(validate=system!='B3_no_validation')
        for rep in range(repetitions):
            for idx,(q,b,scenario,status,ids) in enumerate(CASES):
                start=time.perf_counter();coordinator.trace=[]
                if system=='B0_single':result=single_pipeline(q,b,scenario)
                else:
                    if system=='B2_learned_MAS':
                        intent=router.route('t\u00ecm mua laptop cho t\u00f4i')
                        if intent['intent']!='ecommerce':result={'status':'clarify','items':[]}
                        else:result=await coordinator.shop(q,b,scenario,f'{system}:{rep}:{idx}')
                    else:result=await coordinator.shop(q,b,scenario,f'{system}:{rep}:{idx}')
                latency=(time.perf_counter()-start)*1000
                success=result['status']==status and [i['id'] for i in result['items']]==ids
                violation=any(i['price']>b or i['id'] not in ['p1','p2','p3','p4','p5'] or scenario=='stale'
     for i in result['items'])
                rows.append({'system':system,'repeat':rep,'case':idx,'scenario':scenario,'status':result['
    status'],
                    'success':int(success),'violation':int(violation),'latency_ms':latency,'messages':2*len(
    coordinator.trace)})
                if rep==0:trace.append({'system':system,'case':idx,'result':result,'trace':list(coordinator.
    trace)})
    summary=[]
    for s in sorted(set(r['system'] for r in rows)):
        rr=[r for r in rows if r['system']==s];lat=[r['latency_ms'] for r in rr]
        summary.append({'system':s,'requests':len(rr),'success_rate':float(np.mean([r['success'] for r in rr
    ])),
          'p50_ms':float(np.median(lat)),'p95_ms':float(np.percentile(lat,95)),
          'messages_mean':float(np.mean([r['messages'] for r in rr])),
          'violation_rate':float(np.mean([r['violation'] for r in rr]))})
    for filename,rr in [('mas_benchmark_raw.csv',rows),('mas_benchmark.csv',summary)]:
        with open(out/filename,'w',newline='',encoding='utf-8') as f:w=csv.DictWriter(f,fieldnames=list(rr
    [0]));w.writeheader();w.writerows(rr)
    (out/'mas_traces.json').write_text(json.dumps(trace,ensure_ascii=False,indent=2))
    (out/'mas_metrics.json').write_text(json.dumps({'summary':summary,'case_count':len(CASES),'repetitions':
    repetitions,
        'python':platform.python_version(),'note':'Local synthetic services; single baseline injects
    declared service error outcomes synchronously; its timeout latency is not comparable to real wait_for
    deadline.'},indent=2))
    print(json.dumps(summary,indent=2));return summary
```

```
if __name__==’__main__’:asyncio.run(run())
```

23.6 Mã kiểm thử tutorial

Bộ test chạy tự động và được dùng để tạo kết quả thực tế ở trên.

Listing 11: test_tutorial.py – 17 ca kiểm thử

```
import asyncio,unittest
from mas_system import *
from train_intent import dataset, Router

class TutorialTests(unittest.IsolatedAsyncioTestCase):
    async def test_tag_retrieval(self):
        r=await Coordinator().shop(’portable’,100);self.assertEqual([p[’id’] for p in r[’items’]],[’p2’,’p4’])
    async def test_budget(self):
        r=await Coordinator().shop(’portable’,30);self.assertTrue(all(p[’price’]<=30 for p in r[’items’]))
    async def test_unknown(self):self.assertEqual((await Coordinator().shop(’zzz’,100))[’status’],’no_results’)
    async def test_empty(self):self.assertEqual((await Coordinator().shop(’’,100))[’items’],[])
    async def test_out_of_stock(self):self.assertEqual((await Coordinator().shop(’sports’,100))[’items’],[])
    async def test_negative_budget(self):self.assertEqual((await Coordinator().shop(’portable’,-1))[’status’],’invalid_input’)
    async def test_nan_budget(self):self.assertEqual((await Coordinator().shop(’portable’,float(’nan’)))[’status’],’invalid_input’)
    async def test_timeout(self):self.assertEqual((await Coordinator().shop(’portable’,100,’timeout’))[’status’],’timeout’)
    async def test_malformed(self):self.assertEqual((await Coordinator().shop(’portable’,100,’malformed’))[’status’],’contract_error’)
    async def test_stale(self):self.assertEqual((await Coordinator().shop(’portable’,100,’stale’))[’status’],’availability_unverified’)
    async def test_forged(self):self.assertEqual((await Coordinator().shop(’portable’,100,’forged’))[’status’],’contract_error’)
    async def test_message_receiver(self):
        with self.assertRaises(ValueError):await CatalogAgent().handle(Message(’a’,’x’,’wrong’,’SEARCH’,{’query’:’x’}))
    async def test_correlation(self):
        class Bad:
            name=’bad’
            async def handle(self,m):return Message(’other’,’bad’,’coordinator’,’OK’,{})
        with self.assertRaises(ValueError):await Coordinator().call(Bad(),Message(’a’,’coordinator’,’bad’,’X’,{}),’OK’)
    async def test_trace(self):
        s=Coordinator();await s.shop(’portable’,100,task_id=’trace-1’);self.assertEqual(len(s.trace),4)
        self.assertTrue(all(t[’task_id’]==’trace-1’ for t in s.trace))
    async def test_split_groups(self):
        r=dataset();groups={s:{x[’group’] for x in r if x[’split’]==s} for s in [’train’,’validation’,’test’]}
        self.assertFalse(groups[’train’] & groups[’test’]);self.assertFalse(groups[’validation’] & groups[’test’])
    async def test_ood_abstention(self):self.assertEqual(Router().route(’abc xyz’)[’intent’],’clarify’)
    async def test_normal_equivalence(self):
        for q in [’portable’,’study’,’unknown’,’sports’,’’]:
            for b in [0,30,100,1000]:self.assertEqual(await Coordinator().shop(q,b),single_pipeline(q,b))
if __name__==’__main__’:unittest.main(verbosity=2)
```

24 Kết quả thực hành và bằng chứng chạy

24.1 Kiến trúc B0–B3

Các số dưới được lấy từ mas_benchmark.csv. Mỗi hệ chạy 11 ca, lặp 20 lần. Success có tính cả trạng thái từ chối đúng; tỷ lệ không đồng nghĩa tỷ lệ yêu cầu có gợi ý sản phẩm.

Hệ	Success (%)	P50 ms	P95 ms	Message/request	Violation (%)
B0_single	100.00	0.0048	0.0090	0.00	0.00
B1_rule_MAS	100.00	0.2406	11.0973	6.00	0.00
B2_learned_MAS	100.00	0.3104	11.4029	6.00	0.00
B3_no_validation	72.73	0.2172	11.3042	4.73	18.18

Không so tỷ lệ tốc độ B0/B1 trên các ca lỗi: baseline đơn trả trạng thái timeout được tiêm đồng bộ, còn MAS chờ deadline thực. Phần learned routing trong benchmark luôn nhận một câu ecommerce cố định, nên chỉ phản ánh overhead tích hợp. Đánh giá đa miền nằm ở bảng classifier riêng.

24.2 Chỉ số classifier trên held-out test

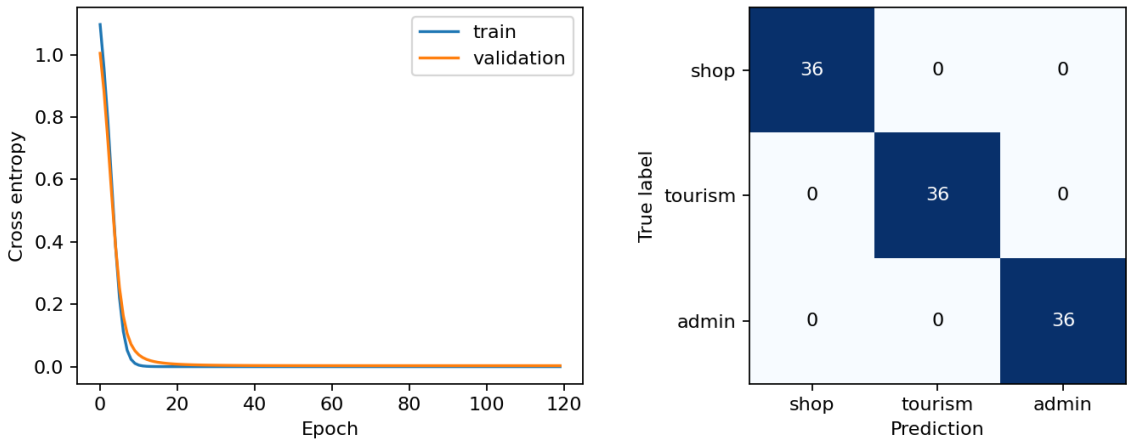
Lớp	Precision	Recall	F1	Số câu
ecommerce	1.0000	1.0000	1.0000	36
tourism	1.0000	1.0000	1.0000	36
healthcare admin	1.0000	1.0000	1.0000	36

Accuracy: 1.0000; macro-F1: 1.0000; threshold 0.5; coverage 1.0000. Tập train, validation và test: 270, 54 và 108 câu. Vocabulary 79; seed 42.

24.3 Ngoài miền: những lỗi cần giữ lại để phân tích

Câu thử	Kết quả	Confidence
xin chào	clarify	0.4943
thời tiết hôm nay thế nào	ecommerce	0.9877
viết thơ cho tôi	clarify	0.6866
giải phương trình	tourism	0.9311
cảm ơn bạn	clarify	0.4943
abc xyz	clarify	0.4943

Có 4/6 câu được từ chối và 2/6 bị nhận nhầm vào miền dù confidence cao. Sáu câu là probe nhỏ, không phải một benchmark OOD đại diện. Không dùng tỷ lệ này để ước lượng lỗi trên mọi yêu cầu người dùng.



Hình 2: Loss train/validation và confusion matrix trên 108 câu test tổng hợp.

24.4 Kết quả failure injection và test

Ca lỗi	Status của B1	Số item
0	ok	2
1	no_results	0
2	no_results	0
3	no_results	0
4	no_results	0
5	ok	1
6	timeout	0
7	contract_error	0
8	availability_unverified	0
9	contract_error	0
10	invalid_input	0

Có 17 test tutorial chạy thành công; log đầy đủ nằm trong build/tutorial_tests.log. Hệ đã kiểm 4 lỗi bắt buộc của slide và thêm input NaN, ID giả, sai correlation. Trường hợp OOD trong test chỉ xác nhận câu không có token miền trả clarify, không phủ nhận các lỗi OOD đã báo ở bảng trên.

```
python3 -m unittest discover -s tutorial -p 'test_*.py'
.................
Ran 17 tests
OK
```

24.5 Mã thực thi cần đọc khi bảo vệ

Listing 12: Deadline, xác thực envelope và trace của coordinator

```
class Coordinator:
    def __init__(self,validate=True,deadline=.01):
```

```
        self.catalog=CatalogAgent();self.recommender=RecommendationAgent();self.inventory=InventoryAgent()
        self.policy=PolicyAgent();self.validate=validate;self.deadline=deadline;self.trace=[]
    async def call(self,agent,m,expected):
        start=time.perf_counter(); status='error'
        try:
            result=await asyncio.wait_for(agent.handle(m),timeout=self.deadline)
            if not isinstance(result,Message):raise TypeError('Malformed response')
            if result.task_id!=m.task_id or result.receiver!=m.sender or result.sender!=agent.name or result
    .kind!=expected:
                raise ValueError('Response correlation/type violation')
            status='ok';return result
        except (TimeoutError,asyncio.TimeoutError,ValueError,TypeError,KeyError):
            status='error';raise
        finally:
            self.trace.append({'task_id':m.task_id,'agent':agent.name,'kind':m.kind,
                'status':status,'milliseconds':(time.perf_counter()-start)*1000})
```

Listing 13: Luồng điều phối và trạng thái kết thúc

```
    async def shop(self,query,budget,scenario='normal',task_id='demo'):
        self.trace=[]
        if not isinstance(query,str) or not isinstance(budget,(int,float)) or not math.isfinite(budget) or
    budget<0:
            return {'status':'invalid_input','items':[]}
        def msg(receiver,kind,payload):return Message(task_id,'coordinator',receiver,kind,payload)
        try:
            c=await self.call(self.catalog,msg('catalog','SEARCH',{'query':query,'scenario':scenario}),'
    CANDIDATES')
            candidates=c.payload['ids']
            if not candidates:return {'status':'no_results','items':[]}
            r=await self.call(self.recommender,msg('recommender','RANK',{'ids':candidates,'budget':budget})
    ,'RANKED')
            ids=r.payload['ids']
            # Deliberate adversarial component output, to test coordinator validation.
            if scenario=='forged':ids=['p99','p1']+ids
            if self.validate:
                p=await self.call(self.policy,msg('policy','VALIDATE',{'ids':ids,'candidates':candidates,'
    budget':budget}),'VALIDATED')
                ids=p.payload['ids']
            inv=await self.call(self.inventory,msg('inventory','CHECK',{'ids':ids,'scenario':scenario}),'
    AVAILABILITY')
            if self.validate and (not isinstance(inv.payload.get('age_seconds'),(int,float)) or not 0<=inv.
    payload['age_seconds']<=60):
                return {'status':'availability_unverified','items':[]}
            stocks=inv.payload['stock']
            if self.validate and (set(stocks)!=set(ids) or any(type(n) is not int or n<0 for n in stocks.
    values())):
                raise ValueError('Invalid stock schema')
            lookup={p.id:p for p in PRODUCTS}
            items=[{'id':i,'price':lookup[i].price if i in lookup else 0,'availability':'verified'}
                   for i in ids if stocks.get(i,0)>0 or not self.validate]
            return {'status':'ok' if items else 'no_results','items':items}
        except (TimeoutError,asyncio.TimeoutError):return {'status':'timeout','items':[]}
        except (ValueError,TypeError,KeyError):return {'status':'contract_error','items':[]}
```

Đoạn mã đầy đủ trong tutorial/mas_system.py là nguồn sự thật khi cần kiểm các dòng sau trích đoạn. Notebook thể hiện lời gọi trực tiếp, đầu ra thực tế, dữ liệu học và các bảng kết quả; bản HTML cho phép xem offline mà không cần chạy kernel.

Phần II

ĐỌC VÀ PHÂN TÍCH HAI PAPER

25 Mở đầu: Hai paper trong một khung nghiên cứu

Báo cáo đọc và phân tích đầy đủ hai tài liệu giảng viên cung cấp: survey về cơ chế cộng tác nhiều LLM agent và tổng quan học tăng cường trong các lĩnh vực. Phần thứ nhất trả lời cách tổ chức các thành viên, thông tin và quyết định; phần thứ hai trả lời cách policy học từ tương tác và reward. Cả hai được liên hệ với tutorial và Laboratory Experiment 02 đã thực thi trong cùng bộ bài.

Mục tiêu của báo cáo là giải thích đủ sâu để người đọc có thể chuyển từ khái niệm sang thiết kế, đồng thời nêu điều kiện và giới hạn của từng kết luận. Nội dung không dịch nguyên văn hai paper; phần diễn giải, phép tính bổ sung và thiết kế thực nghiệm do người viết phát triển được trình bày tách khỏi mô tả đóng góp của tác giả.

25.1 Cấu trúc và quy ước bằng chứng

Paper 1 được đọc theo background, mô hình tác tử, collaboration channels, loại quan hệ, chiến lược, topology, orchestration, ứng dụng và vấn đề mở. Paper 2 được đọc theo MDP, Bellman, các họ thuật toán, ứng dụng, thách thức và giả định. Phần tổng hợp cuối đối chiếu hai hướng, xây thiết kế mẫu và đề xuất protocol kiểm chứng.

Trong toàn bộ báo cáo, "kết quả paper" chỉ dùng cho những gì tài liệu gốc thực sự trình bày; "kết quả mô phỏng" chỉ dùng cho dữ liệu do mã đi kèm sinh; "đề xuất" chỉ phương án chưa được kiểm nghiệm đầy đủ. Không dùng demo cục bộ như bằng chứng tái lập benchmark LLM, không dùng dữ liệu tổng hợp như bằng chứng sẵn sàng triển khai thực tế.

26 Paper 1: Cơ chế cộng tác đa tác tử dựa trên LLM

26.1 Hồ sơ tài liệu, phạm vi và cách tiếp cận

Tài liệu được phân tích: Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O'Sullivan và Hoang D. Nguyen, *Multi-Agent Collaboration Mechanisms: A Survey of LLMs*, arXiv:2501.06322v1, công bố ngày 10/01/2025, 35 trang. Báo cáo bám bản PDF giảng viên cung cấp, không thay bản này bằng một phiên bản khác xuất hiện sau đó. Các nhận định về nội dung paper được đối chiếu trực tiếp với Sections 1–7 và các bảng, hình trong bản PDF. [3]

Đây là một bài **survey về cơ chế phối hợp**, không phải paper đề xuất một thuật toán tối ưu duy nhất và đánh giá nó bằng một thí nghiệm thống nhất. Vì vậy, câu hỏi đọc hiểu thích hợp là: tác giả tổ chức lĩnh vực như thế nào, khung phân loại có giúp phân biệt thiết kế không, và các luận điểm tổng quát được hỗ trợ đến đâu? Việc chỉ kể tên AutoGen, MetaGPT hoặc các agent nổi tiếng sẽ bỏ qua đóng góp trọng tâm của bài.

Báo cáo triển khai ba lớp: diễn giải nội dung gốc; phân tích kỹ thuật và phản biện của người viết; liên hệ với mã tutorial và assignment đã chạy. Những phương trình chi phí, thiết kế kiểm chứng và ví dụ thương mại điện tử do báo cáo phát triển sẽ được gọi rõ là phân tích hoặc đề xuất. Không có thí nghiệm gọi LLM trong bộ mã đi kèm; do đó kết quả mô phỏng chỉ minh họa cơ chế phối hợp, không tái lập benchmark của các hệ LLM được paper tổng thuật.

26.2 Vấn đề nghiên cứu và động lực

Một LLM có thể tạo văn bản, viết mã và giải một số bài toán suy luận, nhưng vẫn có giới hạn về thông tin có sẵn, độ dài ngữ cảnh, độ tin cậy của hành động và khả năng tự kiểm tra. Tác giả đặt vấn đề chuyển từ cải thiện một mô hình đơn lẻ sang tổ chức nhiều tác tử có thể chia sẻ công việc. Ý tưởng hấp dẫn ở chỗ mỗi thành viên phụ trách một phần nhỏ hơn, có công cụ và góc nhìn riêng, rồi một cơ chế phối hợp tạo ra lời giải chung.

Tuy nhiên, phân chia công việc phát sinh một bài toán mới: làm sao kết quả của các thành viên tương thích? Một agent truy hồi có thể lấy đúng đoạn dữ liệu nhưng agent tổng hợp diễn giải sai; agent lập kế hoạch có thể đề xuất bước ngoài quyền của agent thực thi; hai agent có thể cùng sửa một tài nguyên. Khi ấy năng lực của hệ thống bị giới hạn bởi chất lượng quan hệ giữa các thành viên. Đó là lý do paper đặt **collaboration channels** ở trung tâm phân tích.

Một ví dụ xuyên suốt của báo cáo là trợ lý mua hàng. Agent danh mục truy hồi sản phẩm, agent đề xuất xếp hạng, agent kho kiểm tra số lượng và agent chính sách kiểm tra ngân sách. Nếu chỉ cộng bốn văn bản trả lời lại, hệ thống có thể khuyến nghị một sản phẩm hết hàng hoặc vượt ngân sách. Giá trị của phối hợp nằm ở điều kiện chuyển giao: ID sản phẩm phải tồn tại, giá phải lấy từ danh mục có thẩm quyền, thông tin kho phải còn mới và kết quả cuối phải được kiểm tra.

Phân tích của người viết: việc tăng số agent làm tăng số cơ hội trao đổi thông tin nhưng

cũng tăng số điểm có thể hỏng. Một mô hình đơn với công cụ tốt có thể hoàn thành bài đơn giản nhanh hơn một nhóm nhiều agent. Bởi vậy, lý do xây MAS cần gắn với yêu cầu phân quyền, chuyên môn, dữ liệu phân tán hoặc quyết định tương tác, rồi kiểm tra lợi ích bằng baseline tương xứng. Đây là một luận điểm kỹ thuật cần mang theo khi đọc các lời hứa về trí tuệ tập thể.

26.3 Đóng góp và vị trí của survey

Trong Section 1.2, tác giả nêu bốn hướng đóng góp: khảo sát cơ chế phối hợp giữa các LLM agent; đưa ra một framework tổng hợp; xem xét ứng dụng; và rút ra bài học cùng vấn đề mở. Bảng so sánh các survey ở phần đầu bài nhằm cho thấy nghiên cứu này tập trung vào phương diện *cách cộng tác diễn ra*, thay vì chỉ liệt kê ứng dụng hoặc mô tả vòng đời agent.

Đóng góp đầu tiên là một **ngôn ngữ phân tích chung**. Cùng dùng LLM nhưng một hệ có thể phối hợp qua bỏ phiếu, một hệ qua chia vai và một hệ qua mô hình dự đoán đối tác. Nếu gọi chung cả ba là "multi-agent reasoning", người đọc sẽ khó biết yếu tố nào tạo ra hiệu quả. Framework tách loại quan hệ, chiến lược, cấu trúc và điều phối, nhờ đó cho phép mô tả một thiết kế theo nhiều chiều độc lập.

Đóng góp thứ hai là nối các hướng nghiên cứu thường nằm riêng: MAS cổ điển, hệ nhiều LLM, cộng tác người–máy, chia sẻ mô hình và mô phỏng xã hội. Phạm vi rộng giúp nhận ra những điểm chung về thông tin, mục tiêu và điều phối. Đổi lại, phạm vi này khiến một vài ví dụ khác nhau đáng kể về cơ chế: một khách hàng federated learning và một tác tử hội thoại không có cùng chu trình hành động hay độ tự chủ. Khi áp dụng framework, cần ghi rõ cấp độ trừu tượng đang xét.

Về phương pháp, bài tự mô tả việc phân tích các nghiên cứu và tổng hợp đặc trưng. Báo cáo không tìm thấy một quy trình tìm kiếm, sàng lọc và đánh giá chất lượng theo chuẩn systematic review được mô tả đầy đủ như một sơ đồ PRISMA. Do đó nên đọc các bảng của bài như bản đồ có tổ chức của tài liệu được chọn, không xem đó là thống kê bao phủ hoàn toàn mọi công bố trong lĩnh vực. Đây là giới hạn phương pháp, không làm mất giá trị của taxonomy.

26.4 Nền tảng: MAS, LLM và collaborative AI

Section 2.1 mô tả MAS bằng bốn nhóm thành phần: tác tử, môi trường, tương tác và tổ chức. Tác tử có vai trò, năng lực và chính sách hành vi; môi trường cung cấp điều kiện để quan sát và hành động; tương tác gồm trao đổi thông tin, phối hợp hoặc thương lượng; tổ chức xác định quan hệ kiểm soát. Một hệ nhiều mô hình chỉ trở thành MAS theo nghĩa hữu ích khi có các thành viên và quan hệ tương tác đủ rõ để phân tích.

Section 2.2 đặt LLM vào vai trò bộ xử lý trung tâm của tác tử. Mô hình có thể tiếp nhận chỉ dẫn, suy luận trên văn bản và tạo lời gọi công cụ. Nhưng "LLM là bộ não" chỉ là phép diễn giải kiến trúc. Bộ nhớ, quyền truy cập, giao thức thông điệp, vòng lặp công cụ và tiêu chí dừng vẫn phải được thiết kế. Khả năng sinh một câu "đã kiểm tra kho" không đồng nghĩa với việc một truy vấn kho hợp lệ đã được thực hiện.

Section 2.3 xem collaborative AI như hướng phối hợp giữa các hệ AI hoặc giữa AI và con người. Hợp tác là một trường hợp; tranh luận, cạnh tranh và vừa hợp tác vừa cạnh tranh cũng thuộc phổ tương tác mà paper xem xét. Từ đó cần phân biệt mục tiêu ở hai cấp: hai thành viên có thể bất đồng ở cấp lập luận nhưng vẫn cùng phục vụ chất lượng đáp án cuối của hệ thống.

Ví dụ phản biện: một người viết và một người kiểm chứng có thể dùng hai system prompt khác nhau nhưng cùng một mô hình, cùng dữ liệu và cùng thiên kiến. Chia vai làm thay đổi quá trình xử lý, chưa chứng minh hai nguồn sai số độc lập. Muốn đánh giá tính bổ sung, cần đo tỷ lệ lỗi chung, độ đa dạng của bằng chứng và khả năng phát hiện sai của người kiểm chứng, thay vì đếm số persona.

26.5 Mô hình hóa tác tử và hệ cộng tác

Ở Section 3.1, tác giả biểu diễn một agent bằng bộ $a = \{m, o, e, x, y\}$: mô hình, mục tiêu, môi trường/ngữ cảnh, đầu vào và đầu ra. Thành phần mô hình được mô tả thêm bằng kiến trúc, bộ nhớ và adapter. Với LLM, đầu vào thường là chuỗi token, ngữ cảnh chịu giới hạn cửa sổ và đầu ra có thể là văn bản hoặc hành động được mã hóa. Ký hiệu này tổ chức khái niệm, không phải một bộ phương trình bảo đảm tính tối ưu.

Hệ cộng tác gồm tập tác tử $\mathcal{A}$, mục tiêu chung O, môi trường chia sẻ $\mathcal{E}$, tập kênh $\mathcal{C}$, đầu vào và đầu ra hệ thống. Có thể diễn giải ngắn gọn:

$$y_{\text{system}} = S(O, \mathcal{E}, x \mid \mathcal{A}, \mathcal{C}).$$

Điểm có ích là đầu ra phụ thuộc cả agent lẫn **kênh cộng tác**. Giữ nguyên các mô hình nhưng đổi ai nói trước, ai được thấy đầu ra nào và ai có quyền kết thúc có thể làm hành vi thay đổi. Điều này giải thích vì sao chỉ báo tên backbone chưa đủ để tái lập một thí nghiệm MAS.

Mỗi kênh cần mô tả: ai tham gia; mục tiêu hoặc nội dung trao đổi; loại quan hệ; cấu trúc kết nối; chiến lược đưa ra quyết định; điều kiện được tạo và kết thúc. Trong tutorial, lời gọi từ coordinator đến inventory là một kênh request–response hợp tác, tập trung, theo vai trò và có deadline. Đầu vào là danh sách ID; đầu ra là số lượng và tuổi dữ liệu. Mô tả này kiểm tra được bằng trace, khác với nhãn mơ hồ "agent kho hỗ trợ agent bán hàng".

Framework gốc giả định các agent có nền tảng giao tiếp chung. Phân tích của báo cáo: cùng dùng tiếng Anh hoặc JSON mới giải quyết một phần cú pháp. Hai bên còn phải thống nhất đơn vị tiền, nghĩa của "available", múi giờ, trạng thái lỗi, độ mới của dữ liệu và quyền hành động. Những chi tiết hợp đồng này quyết định liệu sự hiểu nhau có đúng ở mức vận hành hay chỉ đúng ở mức ngôn ngữ.

26.6 Cộng tác sớm, giữa và muộn

Paper mô tả cộng tác ở các giai đoạn khác nhau. Cộng tác sớm có thể chia sẻ dữ liệu, ngữ cảnh hoặc biểu diễn đầu vào; cộng tác giữa có thể trao đổi tham số, trọng số hoặc adapter; cộng tác muộn kết hợp đầu ra hoặc hành động. Ba mức cho thấy cộng tác không đồng nghĩa

với hội thoại bằng ngôn ngữ tự nhiên.

Chia sẻ ngữ cảnh giúp hai agent cùng thấy bằng chứng, nhưng có thể làm lan một nguồn sai chung. Chia sẻ mô hình giúp học từ nhiều nơi, nhưng kéo theo tương thích kiến trúc, quyền dữ liệu và bảo mật cập nhật. Kết hợp đầu ra đơn giản hơn về triển khai, nhưng không sửa được những lỗi đã bị mất khỏi các bản tóm tắt trung gian. Mỗi điểm kết nối làm thay đổi thứ thông tin còn có thể được kiểm chứng ở cuối quy trình.

Một thiết kế mua hàng có thể dùng cả ba mức: các agent đọc một catalog snapshot chung; mô hình intent được cập nhật theo dữ liệu tổng hợp; coordinator kết hợp các danh sách đề xuất đã qua kiểm tra. Tuy nhiên, bộ mã của báo cáo chỉ huấn luyện một classifier cục bộ và truyền dữ liệu có cấu trúc, không thực hiện federated learning hoặc trao đổi trọng số giữa nhiều tác tử. Việc liên hệ kiến trúc không phải tuyên bố đã triển khai mọi cấp độ trong survey.

26.7 Ba loại quan hệ cộng tác

26.7.1 Cooperation: mục tiêu được căn chỉnh

Trong hợp tác, các tác tử chia công việc để tăng chất lượng mục tiêu chung. Chuyên môn hóa giúp giảm phạm vi mỗi thành viên phải xử lý. Người tìm tài liệu có thể tập trung vào độ bao phủ; người kiểm chứng tập trung vào dẫn chứng; người biên tập tập trung vào tính mạch lạc. Các thành phần khác nhau không cần tối ưu một chỉ số cục bộ giống hệt nhau, nhưng kết quả phải phục vụ cùng tiêu chí nghiệm thu.

Rủi ro là mục tiêu cục bộ dễ lệch. Nếu agent đề xuất chỉ tối ưu điểm hấp dẫn, nó có thể chọn sản phẩm vượt ngân sách. Nếu agent tổng hợp chỉ tối ưu độ đầy đủ, nó có thể bổ sung thông tin không có trong nguồn. Vì vậy cần một hàm mục tiêu hoặc tập ràng buộc ở cấp hệ thống, cùng cơ chế kiểm tra đầu ra từng bước. “Cùng giúp người dùng” chưa phải hợp đồng phối hợp đủ chặt.

26.7.2 Competition: bất đồng có cấu trúc

Cạnh tranh xuất hiện khi thành viên tối ưu lợi ích riêng hoặc được giao bảo vệ các lập luận trái chiều. Tranh luận có thể làm lộ giả định và lỗi suy luận, đặc biệt khi mỗi bên phải viện dẫn bằng chứng. Tuy nhiên, một agent phản đối mạnh không nhất thiết đưa ra phản biện đúng. Cơ chế phán quyết cần ưu tiên tính kiểm chứng, thay vì văn phong tự tin hoặc số lượng phát biểu.

Ví dụ của người viết: agent A đề xuất chọn sản phẩm p1 vì hiệu năng, agent B phản biện rằng p1 vượt ngân sách. Nếu giá p1 được xác nhận từ catalog, đây là phản biện có giá trị quyết định. Nếu B chỉ nói “có thể đắt”, coordinator chưa có đủ cơ sở. Sự khác biệt giữa phản đối và kiểm chứng phải được phản ánh trong dữ liệu thông điệp: claim, evidence, constraint và verdict nên là các trường riêng.

26.7.3 Coopetition: hợp tác ở một cấp, cạnh tranh ở cấp khác

Trong coopetition, các bên có lợi ích vừa trùng vừa khác. Hai công ty có thể cùng dùng hạ tầng logistics nhưng cạnh tranh khách hàng; nhiều nhóm có thể phối hợp chia tài nguyên tính toán nhưng cạnh tranh thứ hạng. Thiết kế phải nêu rõ thứ gì được chia sẻ và thứ gì cần giữ riêng. Một mục tiêu chung không xóa bỏ xung đột lợi ích ở cấp tác tử.

Assignment bán hàng có hai lợi nhuận riêng và ảnh hưởng qua giảm giá. Nó là tương tác cạnh tranh về lợi nhuận, nhưng không tự động là trò chơi tổng bằng không: lượng mua của hai cửa hàng được lấy mẫu riêng, và không có ràng buộc tổng doanh số bằng một thị trường cố định. Gọi mọi biến động lợi nhuận là "một bên thắng thì bên kia thua" sẽ sai với mô hình đang dùng.

26.8 Ba chiến lược: rule-based, role-based, model-based

Chiến lược	Cơ chế chính	Thứ phải khai báo	Điểm dễ sai
Rule-based	Luật quyết định ai nói, nói khi nào, kết hợp ra sao	Điều kiện, ưu tiên, tiêu chí dừng	Luật thiếu trường hợp, xung đột thứ tự
Role-based	Phân công chuyên môn và trách nhiệm	Quyền, đầu vào, đầu ra, phạm vi	Chồng vai, bỏ sót nhiệm vụ, phụ thuộc dây chuyền
Model-based	Dự đoán môi trường, đối tác hoặc bất định để quyết định	Mô hình, quan sát, giả định, cách cập nhật	Dự đoán đối tác sai, chi phí lớn, khó kiểm chứng

26.8.1 Giao thức theo luật

Rule-based trong survey mô tả **giao thức tương tác**, chẳng hạn thứ tự tranh luận hoặc bỏ phiếu. Nó không đồng nhất với rule-based seller của assignment, nơi luật xác định hành động giảm giá. Một hệ có thể có agent học máy nhưng phối hợp theo luật cố định; ngược lại, một coordinator học cách phân việc có thể điều khiển các công cụ hoàn toàn xác định. Giữ riêng hai cấp tránh nhầm "có luật" với "không có học".

Ưu điểm của luật là truy vết được: mỗi quyết định có điều kiện và nhánh xử lý cụ thể. Nhược điểm là khi không gian tình huống lớn, số điều kiện tăng và tương tác giữa luật khó quản lý. Thiết kế tốt thường giữ các bất biến quan trọng bằng luật, rồi cho phép mô hình đề xuất phương án bên trong miền hợp lệ. Đây là cấu trúc hybrid phù hợp với tutorial.

26.8.2 Giao thức theo vai trò

Role-based đặt câu hỏi ai có chuyên môn hoặc tài nguyên thích hợp. MetaGPT là ví dụ paper dùng để minh họa quy trình có vai trò và SOP; AutoGen cung cấp cách tổ chức các agent có thể hội thoại, dùng công cụ hoặc tương tác với con người. Những ví dụ này cho thấy cấu trúc vai trò ảnh hưởng đến cách phân rã nhiệm vụ; chúng không tạo ra định lý

rằng mọi quy trình nhiều vai đều tốt hơn mọi baseline. [8, 7]

Một vai trò cần có khả năng bị kiểm tra và thay thế. Trong tutorial, CatalogAgent chỉ biết truy hồi ID; InventoryAgent chỉ cung cấp thông tin kho; PolicyAgent xác nhận nguồn gốc và ngân sách. Nếu thay ranker mà các hợp đồng không đổi, hệ còn chạy được. Đây là lợi ích thiết kế cụ thể của chia vai, ngay cả khi chất lượng tác vụ không tăng trên catalog nhỏ.

26.8.3 Giao thức dựa trên mô hình

Model-based trong paper có thể bao gồm suy luận xác suất và Theory of Mind để dự đoán mục tiêu hoặc hành vi của bên khác. Thuật ngữ này có phạm vi khác "model-based RL" trong paper thứ hai, nơi mô hình thường là chuyển trạng thái và phần thưởng. Cả hai cùng dùng mô hình để hỗ trợ quyết định nhưng đối tượng mô hình hóa và bảo đảm lý thuyết khác nhau.

Nếu agent bán hàng dự đoán xác suất đối thủ giảm giá ở bước kế tiếp, nó có thể tính lợi nhuận kỳ vọng của các lựa chọn. Nhưng nếu đối thủ cũng cập nhật chiến lược, dự đoán vừa học sẽ lỗi thời. Cần đo chất lượng mô hình đối tác trên dữ liệu tương lai và đánh giá chính sách dưới nhiều loại đối thủ. Một mô hình mô tả tốt lịch sử chưa chắc dẫn đến quyết định tốt trong tương tác chiến lược.

26.9 Cấu trúc giao tiếp và chi phí

26.9.1 Tập trung

Trong cấu trúc tập trung, coordinator nhận yêu cầu, phân việc, thu kết quả và quyết định đầu ra. Nó giúp đặt ràng buộc và logging ở một điểm. Tutorial chọn cấu trúc này để trình tự kiểm chứng rõ và dễ tái lập. Hạn chế là coordinator có thể thành nút nghẽn và điểm hỏng chung. Chia các worker độc lập không giải quyết hỏng của chính bộ điều phối.

Với n worker, một lượt request–response đến mọi worker cần xấp xỉ $2n$ thông điệp nếu bỏ qua retry. Chi phí tuyến tính này thuận tiện để dự toán. Tuy nhiên, mỗi request có thể chứa toàn bộ ngữ cảnh rất dài; số thông điệp nhỏ chưa chắc số token nhỏ. Vì vậy cần đo đồng thời số message, dung lượng payload và thời gian xử lý.

26.9.2 Phi tập trung và phân tán

Trong cấu trúc phi tập trung, quyết định được chia giữa các thành viên. Cấu trúc phân tán nói về vị trí thực thi; phi tập trung nói về quyền kiểm soát. Một hệ chạy trên nhiều máy nhưng mọi quyết định qua một manager vẫn có thể là tập trung về điều khiển. Đây là phân biệt kỹ thuật của báo cáo để làm rõ những khái niệm được đặt gần nhau trong survey.

Nếu mọi agent trao đổi với mọi agent khác, một vòng có tối đa $n(n-1)$ thông điệp có hướng. Với $n = 10$, đó là 90 thông điệp, chưa tính phản hồi và số vòng. Topology thưa, publish–subscribe hoặc giao tiếp theo sự kiện giúp giảm lưu lượng, nhưng cũng giới hạn thông tin mỗi thành viên nhìn thấy. Không có một topology tốt nhất độc lập với nhiệm vụ.

26.9.3 Phân cấp

Cấu trúc phân cấp đưa các nhóm agent dưới manager cấp trung rồi tổng hợp lên trên. Nó giúp chia việc và giảm bề rộng ở nút gốc, nhưng làm tăng số chặng truyền. Thông tin bị tóm tắt qua nhiều cấp có thể mất chi tiết cần kiểm chứng. Một cơ chế tốt cho phép truy ngược từ kết luận về bằng chứng gốc, thay vì chỉ chuyển bản tóm tắt của bản tóm tắt.

Một cây phân cấp còn đòi hỏi chính sách khi manager con hỏng: giao lại nhóm, trả kết quả một phần hay dừng toàn nhiệm vụ? Lựa chọn phụ thuộc tác vụ. Một bản gợi ý mua hàng có thể loại sản phẩm chưa kiểm chứng; một quy trình giao dịch tài chính không thể tự coi phần thiếu là thành công. Cấu trúc giao tiếp phải gắn với nghĩa của thất bại.

26.10 Điều phối tĩnh và động

Điều phối tĩnh thiết lập kênh và thứ tự trước khi nhận yêu cầu. Một pipeline catalog → ranker → policy → inventory là ví dụ. Ưu điểm là dễ kiểm tra số bước tối đa, quyền gọi công cụ và tình huống lỗi. Nó phù hợp khi cấu trúc tác vụ ổn định. Thêm một LLM vào từng bước không biến một quy trình đã cố định thành điều phối động.

Điều phối động cho phép tạo vai trò, chọn thành viên hoặc thay đồ thị thực hiện theo yêu cầu. Cách này có thể tiết kiệm khi chỉ gọi chuyên gia cần thiết, nhưng manager phải giải thêm bài toán lập kế hoạch. Một sơ đồ động sai có thể bỏ mất bước kiểm chứng bắt buộc. Do đó phần động nên chịu một lớp ràng buộc tĩnh: các bước tùy chọn có thể đổi, còn bất biến về nguồn gốc, quyền và xác nhận phải giữ.

Phân tích đồ thị của báo cáo: biểu diễn các bước bằng DAG $G = (V, E)$, cạnh thể hiện phụ thuộc dữ liệu. Tổng thời gian xấp xỉ đường găng cộng overhead điều phối; tổng chi phí bằng tổng công việc của mọi node được thực hiện. Song song hóa có thể giảm thời gian mà tăng tổng chi phí. Một đánh giá chỉ báo latency sẽ bỏ qua trade-off này.

26.11 Bỏ phiếu, tranh luận và kiểm chứng

Bỏ phiếu đa số đơn giản, nhưng chỉ có lợi khi lỗi giữa các thành viên có mức độc lập đủ lớn. Giả sử ba bộ trả lời độc lập, mỗi bộ đúng với xác suất p, xác suất đa số đúng là $3p^2(1-p)+p^3$. Với $p = 0.7$, kết quả là 0.784. Đây là ví dụ xác suất do báo cáo tính, không phải kết quả thực nghiệm của survey.

Nếu cả ba sao chép cùng một lời giải sai, tính độc lập không còn. Khi sai số tương quan hoàn toàn, bỏ phiếu không cải thiện xác suất đúng so với một thành viên. Vì vậy nên ưu tiên đa dạng bằng chứng, công cụ hoặc cách kiểm tra thay vì chỉ đổi tên vai. Đếm phiếu có thể làm tăng độ tự tin trong khi không tăng độ đúng.

Tranh luận nhiều vòng cho phép sửa câu trả lời theo phản biện. Nghiên cứu của Du và cộng sự là một ví dụ gốc về cách nhiều mô hình đề xuất rồi trao đổi qua nhiều vòng. Nhưng kết quả cải thiện trên một số tác vụ không phải bảo đảm cho mọi cấu hình agent. Khi tái lập cần công bố số agent, số vòng, prompt, cách lấy đáp án và tổng ngân sách suy luận. [9]

Kiểm chứng bằng công cụ có vị trí khác bỏ phiếu. Nếu yêu cầu là tính tổng, trình tính

có thể đưa ra phép kiểm xác định. Nếu yêu cầu là xác nhận còn hàng, chỉ dịch vụ kho có thẩm quyền mới trả lời được. Hai agent đồng ý rằng "còn hàng" không thay thế truy vấn kho. Bài học này được triển khai trực tiếp bằng trường `age_seconds` và kiểm tra freshness trong tutorial.

26.12 Ứng dụng được survey phân tích

26.12.1 Mạng 5G/B6G và Industry 5.0

Section 5.1 bàn về các hệ mạng, phân bổ tài nguyên, môi trường công nghiệp và phối hợp thông minh. Giá trị của MAS nằm ở quyết định phân tán, dữ liệu ở nhiều nơi và yêu cầu điều chỉnh theo trạng thái. Một tác tử cục bộ có thể phản ứng nhanh, trong khi cơ chế phối hợp tránh việc tối ưu cục bộ làm xấu chất lượng toàn mạng.

Thách thức triển khai là deadline, giới hạn tài nguyên và độ tin cậy. Một LLM tạo một phương án có vẻ hợp lý nhưng trả lời sau thời điểm cần hành động vẫn có thể không dùng được. Vì thế báo cáo đề xuất đánh giá tỷ lệ đáp ứng hạn, chi phí suy luận và vi phạm ràng buộc, bên cạnh chất lượng phương án. Phần này là cách chuyển chủ đề ứng dụng của survey thành tiêu chí kỹ thuật.

26.12.2 Hỏi đáp và sinh ngôn ngữ

Section 5.2 xem xét phân rã câu hỏi, sử dụng chuyên gia, phản biện và tổng hợp. Một câu hỏi nhiều bước có thể cần truy hồi từ nhiều nguồn rồi hợp nhất. Chia vai giúp quan sát các bước trung gian, nhưng cũng tạo thêm khả năng mất ngữ cảnh hoặc bóp méo bằng chứng. Cần lưu provenance từ mỗi claim đến đoạn nguồn.

Đánh giá chỉ bằng độ trôi chảy sẽ bỏ qua sự thật. Với hỏi đáp, nên tách độ đúng của đáp án, độ đúng của dẫn chứng, độ bao phủ và chi phí. Với viết mã, cần test hoặc đặc tả; với tổng hợp tài liệu, cần kiểm tra mâu thuẫn nguồn. LLM làm giám khảo có thể hỗ trợ nhưng bản thân giám khảo cũng cần đánh giá sai số.

26.12.3 Miền xã hội và văn hóa

Section 5.3 khảo sát mô phỏng cộng đồng, tương tác xã hội, nhân vật và hoạt động mang tính văn hóa. Các môi trường này thích hợp để nghiên cứu hành vi emergent vì nhiều thành viên có ký ức và quan hệ khác nhau. Tuy nhiên, một hành vi trông giống con người trong mô phỏng không đủ chứng minh mô hình phản ánh dân số thực.

Phản biện của người viết: kết quả mô phỏng chịu ảnh hưởng của prompt, dữ liệu tiền huấn luyện, giả định tác giả và cách thiết kế phần thưởng. Nếu dùng mô phỏng để hỗ trợ quyết định xã hội, cần đối chiếu với dữ liệu và chuyên gia bên ngoài. Gán quá nhiều thuộc tính tâm lý cho agent có thể khiến người đọc hiểu sai phạm vi của bằng chứng.

26.13 Vấn đề mở và đánh giá phê bình

26.13.1 Quản trị và quyết định tập thể

Section 6.1 đặt ra thống nhất quản trị, phân vai, phục hồi lỗi và ra quyết định chung. Một hệ đáng tin phải biết ai có quyền sửa mục tiêu, ai được gọi công cụ ghi và ai xác nhận hoàn tất. Khi manager và worker bất đồng, không thể chỉ thêm một vòng chat vô hạn; phải có cơ chế giải quyết hoặc trả quyền quyết định về người dùng.

Đề xuất của báo cáo là dùng ngân sách rõ cho vòng sửa: số lần thử, thời gian và chi phí tối đa; lưu lý do thay đổi; phân biệt "không tìm thấy lời giải" với "lỗi dịch vụ". Bộ điều phối cần có các trạng thái kết thúc thành công, kết quả một phần, thiếu bằng chứng, hết hạn và vi phạm hợp đồng. Cách mô hình hóa này biến một cuộc hội thoại thành quy trình có thể vận hành.

26.13.2 Benchmark và nguyên nhân cải thiện

Section 6.2 nhấn mạnh thiếu chuẩn đánh giá thống nhất. Một hệ nhiều agent có thể dùng nhiều token hơn baseline một agent, nên chênh lệch điểm chưa tách được tác dụng của phối hợp khỏi tác dụng của thêm tính toán. Baseline nên có phiên bản ngân sách tương đương, ví dụ một agent được dùng cùng số lượt công cụ hoặc cùng tổng token.

Ablation phải bỏ đúng thành phần đang muốn đo. Bỏ coordinator validation trong tutorial kiểm tra giá trị của validation; nó không chứng minh việc chia nhiều agent tạo ra cải thiện. So sánh B0 và B1 trên cùng bộ ca mới cho biết overhead của chia vai. So sánh B1 và B2 mới giúp xem classifier bổ sung có thay đổi khả năng định tuyến hay không.

26.13.3 An toàn, tấn công và độ tin cậy

Section 6.3 lưu ý lỗi và thông tin sai có thể lan qua các kênh. Một message từ agent nội bộ vẫn có thể chứa dữ liệu không đáng tin vì agent đó đọc nguồn ngoài. Cần tách dữ liệu khỏi chỉ dẫn, xác thực schema và giới hạn quyền công cụ. Trong tutorial, một ID lạ hoặc sản phẩm vượt ngân sách bị PolicyAgent bác bỏ trước khi có kết luận sẵn hàng.

Không nên suy diễn rằng thêm nhiều agent luôn tăng an toàn. Mỗi agent thêm một bề mặt lỗi và một chuỗi phụ thuộc. Một thiết kế phòng vệ có ích khi bước kiểm chứng có thẩm quyền và thực sự chặn hành động sai. Bản ablation B3 minh họa hệ có nhiều thành viên nhưng vẫn đưa ra thông tin sai khi bỏ kiểm tra nguồn gốc và tuổi dữ liệu.

26.14 Thiết kế nghiên cứu tiếp nối do báo cáo đề xuất

Câu hỏi nghiên cứu: khi catalog nhỏ và các ràng buộc xác định, chia vai có cải thiện tỷ lệ hoàn thành so với pipeline đơn không? Câu hỏi phụ: verification đóng góp bao nhiêu, classifier intent đóng góp bao nhiêu và độ trễ tăng bao nhiêu? Đây là các câu hỏi đủ hẹp để có đối chứng bằng mã cục bộ, thay vì khẳng định chung về mọi MAS.

Thiết kế gồm bốn kiến trúc B0–B3, cùng catalog và tập yêu cầu. Dữ liệu lỗi được tạo có chủ đích: timeout, sai envelope, dữ liệu kho cũ và output giả từ thành phần. Ground truth là trạng thái và danh sách ID hợp lệ; thành công được tính cả các ca từ chối đúng. Điều này

khác với chỉ đếm số lần hệ trả được danh sách sản phẩm.

Để mở rộng thành nghiên cứu LLM, giữ nguyên hợp đồng công cụ và bộ kiểm chứng rồi thay bộ đề xuất bằng LLM. Cần công bố model version, decoding, prompt, ngân sách, số vòng, dữ liệu và trace. Chạy nhiều seed hoặc nhiều lần lấy mẫu; phân tích theo loại lỗi; báo cáo cả phân phối chi phí. Bộ mã hiện tại là nền tảng kiểm thử kiến trúc, chưa phải thí nghiệm này.

26.15 Kết luận riêng cho Paper 1

Giá trị lớn nhất của paper là đổi trọng tâm từ số lượng agent sang cấu trúc quan hệ giữa chúng. Năm chiều actors, types, strategies, structures và coordination cho phép mô tả khá nhiều hệ trong một khung chung. Khi chuyển sang phần mềm, khung đó cần được cụ thể hóa bằng schema, quyền, nguồn dữ liệu, deadline và phép kiểm chứng.

Giới hạn cần giữ rõ là phạm vi survey rộng, các kết quả được tổng thuật đến từ những bài có cấu hình và benchmark khác nhau, và nhiều kỳ vọng về trí tuệ tập thể còn là hướng nghiên cứu. Báo cáo vì thế sử dụng paper để tổ chức câu hỏi thiết kế, rồi dùng các thực nghiệm cục bộ để kiểm tra những nhận định hẹp mà bộ mã thực sự đo được.

27 Paper 2: Học tăng cường và ứng dụng trong các lĩnh vực

27.1 Hồ sơ bài báo và mục đích đọc

Bài *A Gentle Introduction to Reinforcement Learning and Its Application in Different Fields* do Muddasar Naeem, Syed Tahir Hussain Rizvi và Antonio Coronato thực hiện, đăng trên IEEE Access, tập 8, năm 2020, trang 209320–209344, DOI 10.1109/ACCESS.2020.3038605. Bản PDF được cung cấp có 25 trang, gồm nền tảng MDP, nguyên lý RL, các nhóm thuật toán, ứng dụng và thảo luận. [4]

Mục đích của bài là cung cấp một lộ trình nhập môn tương đối rộng, giúp người đọc hiểu những khái niệm trước khi bước vào thuật toán chuyên sâu. Đóng góp phù hợp nhất để đánh giá là cách hệ thống hóa và nối kiến thức với ứng dụng. Bài không cung cấp một benchmark thống nhất chạy mọi thuật toán trên mọi lĩnh vực, nên không thể suy ra một bảng xếp hạng phổ quát từ số lượng ví dụ được nhắc đến.

Báo cáo đọc paper theo chuỗi: mô hình quyết định → đại lượng cần học → cách ước lượng → điều kiện áp dụng → kiểm tra bằng assignment. Các công thức được viết lại với ký hiệu nhất quán và kỳ vọng rõ ràng. Đây là phần diễn giải kỹ thuật của báo cáo, giúp tránh những giản lược ký hiệu trong tài liệu nhập môn. Ví dụ số và kết quả mô phỏng được phân biệt rõ với nội dung nguyên bản.

27.2 RL khác học có giám sát và học không giám sát

Trong học có giám sát, một quan sát đi cùng nhãn hoặc giá trị mục tiêu. Mô hình học quan hệ giữa đặc trưng và nhãn trên tập dữ liệu cho trước. Trong học không giám sát, mục tiêu thường là tìm cấu trúc, biểu diễn hoặc nhóm trong dữ liệu không có nhãn tác vụ. Trong RL, tác tử chọn hành động rồi nhận phản hồi; hành động ảnh hưởng dữ liệu quan sát tiếp theo, vì vậy quá trình thu thập dữ liệu gắn với chính sách hiện tại.

Phần thưởng khác nhãn đáp án. Khi cửa hàng giảm giá và nhận lợi nhuận 30, môi trường không nói hành động còn lại sẽ cho lợi nhuận bao nhiêu trong cùng tình huống. Tác tử phải học thông qua nhiều lần tương tác, đối mặt với nhiễu và đánh đổi giữa thử mới với khai thác điều đã biết. Nếu hậu quả xuất hiện chậm, còn phải xác định hành động nào trong chuỗi đóng góp cho kết quả cuối.

Tutorial intent sử dụng **supervised learning**: mỗi câu có nhãn ecommerce, tourism hoặc healthcare administration và mạng tối ưu cross entropy. Assignment sử dụng **reinforcement learning**: agent chọn hành động giảm giá và học từ lợi nhuận ngẫu nhiên. Hai phần đều có quá trình training nhưng không nên gọi classifier là RL. Tương tự, một hệ có nhiều agent không tự động là MARL nếu các thành viên không cập nhật chính sách từ tương tác.

27.3 Mô hình MDP và giả định Markov

Một MDP thường được biểu diễn bằng $(\mathcal{S}, \mathcal{A}, P, R, \gamma, \rho_0)$: không gian trạng thái, hành động, xác suất chuyển, phần thưởng kỳ vọng, hệ số chiết khấu và phân phối khởi đầu. Với bài hữu

hạn chân trời, cần nêu thêm H hoặc dùng trạng thái chứa thời gian. MDP là mô hình bài toán; RL là một họ phương pháp học quyết định, thường khi không biết đầy đủ P và R.

Tính Markov có thể viết:

$$P(S_{t+1}, R_{t+1} \mid S_0, A_0, \ldots, S_t, A_t) = P(S_{t+1}, R_{t+1} \mid S_t, A_t).$$

Điều kiện không nói quá khứ không quan trọng trong thế giới thực, mà nói trạng thái đã tóm tắt đủ thông tin quá khứ liên quan đến tương lai. Nếu trạng thái thiếu tồn kho, đơn hàng đang xử lý hoặc hành động trước mà đối thủ dùng để phản ứng, điều kiện có thể không còn. Một chuỗi quan sát có dạng vector không mặc nhiên là trạng thái Markov.

Cần tách tính Markov và tính dừng. Markov liên quan đến thông tin cần điều kiện hóa; tính dừng liên quan đến việc cùng một trạng thái và hành động có cùng quy luật qua thời gian. Một quá trình có thể Markov nhưng quy luật phụ thuộc thời điểm. Có thể bổ sung thời gian hoặc chế độ vào trạng thái để mô hình hóa sự phụ thuộc đó, nhưng khi không quan sát chế độ thì bài toán vẫn có bất định.

27.4 Những thành phần cơ bản của môi trường

Trạng thái phải chứa thông tin cần cho quyết định, không nhất thiết là mọi dữ liệu có thể lưu. Thêm quá nhiều biến làm tăng độ phức tạp; bỏ biến quan trọng làm giá trị ước lượng bị trộn. Trong assignment, quan sát tham chiếu có hai bit nên chỉ có bốn trạng thái cục bộ. Khi kiểm tra giả định Markov, báo cáo chỉ ra cần thêm hành động trước của chính mình để mô tả phản ứng của rule opponent.

Hành động là lựa chọn thực sự được môi trường cho phép. Ở đây $a \in \{0, 1\}$, không giảm và giảm 20%. Nếu mở rộng sang nhiều mức giảm, cần kiểm tra biên lợi nhuận và tập hợp hành động hợp lệ. Một mạng xuất ra giá trị liên tục không tự giải quyết tính hợp lệ của giao dịch hoặc hạn mức kinh doanh.

Chuyển trạng thái mô tả điều xảy ra sau hành động. Nhu cầu trong mô phỏng được lấy độc lập với hành động, nhưng hành động hiện tại trở thành lịch sử quan sát ở bước sau. Vì rule opponent phản ứng theo lịch sử này, agent vẫn có ảnh hưởng đến tương lai dù không điều khiển nhu cầu. Chỉ nhìn nhu cầu độc lập rồi kết luận bài toán là bandit sẽ bỏ qua cơ chế phản hồi đối thủ.

Phần thưởng phản ánh mục tiêu đo được. Trong mô phỏng là lợi nhuận số học, không phải doanh thu hoặc số đơn. Việc chọn reward là lựa chọn mô hình hóa có hệ quả: tối ưu số lượng bán sẽ có động lực khác tối ưu lợi nhuận; thêm phạt tồn kho hay phạt biến động giá lại tạo chính sách khác. Cần xác định mục tiêu trước khi chọn thuật toán.

27.5 Return, hệ số chiết khấu và tiêu chí kinh doanh

Return hữu hạn bắt đầu từ bước t được viết:

$$G_t = \sum_{k=0}^{H-t-1} \gamma^k R_{t+k+1}.$$

Khi $\gamma = 0$, chỉ phần thưởng kế tiếp được xem xét. Khi $0 < \gamma < 1$, các phần thưởng xa hơn có trọng số nhỏ hơn và chuỗi vô hạn bị chặn nếu reward bị chặn. Khi $\gamma = 1$, nhiệm vụ hữu hạn vẫn hợp lệ; trong nhiệm vụ vô hạn phải có điều kiện bổ sung để tổng có nghĩa. Vì vậy không thể nói mọi lựa chọn $\gamma = 1$ đều sai hoặc đều an toàn.

Có một khác biệt quan trọng trong assignment: cập nhật Q dùng $\gamma = 0.9$, trong khi báo cáo hiệu năng yêu cầu lợi nhuận trung bình không chiết khấu qua 20 bước. Hai tiêu chí có liên quan nhưng không đồng nhất trong mọi môi trường. Nếu một hành động hy sinh lợi nhuận đầu kỳ để có lợi ích cuối kỳ, việc giảm trọng số tương lai có thể thay đổi lựa chọn tối ưu theo chỉ số kinh doanh.

Trong mô hình hiện tại, quy hoạch động xác nhận chính sách không giảm giá là best response với rule opponent ở các tham số được dùng. Do đó khác biệt tiêu chí không làm thay đổi mốc hành động này. Tuy nhiên, báo cáo vẫn nêu sự không đồng nhất vì nó quan trọng khi thêm tồn kho, ngân sách khuyến mãi hoặc khách hàng quay lại.

Ví dụ của báo cáo: chuỗi reward $(5, 5, 20)$ có tổng 30, nhưng return với $\gamma = 0.9$ là $5 + 0.9 \times 5 + 0.9^2 \times 20 = 25.7$. Một chính sách khác nhận $(12, 10, 6)$ có tổng 28 và return 25.86. Như vậy chính sách có tổng ít hơn vẫn có thể được ưu tiên dưới chiết khấu. Ví dụ này cho thấy phải công bố rõ tối ưu đại lượng nào.

27.6 Chính sách, kế hoạch và hàm giá trị

Chính sách xác định cách hành động theo trạng thái hoặc lịch sử. Chính sách xác định viết $a = \pi(s)$; chính sách ngẫu nhiên viết $\pi(a \mid s)$. Một kế hoạch là chuỗi hoặc đồ thị bước để thực hiện, còn chính sách là quy tắc phản ứng cho những tình huống có thể gặp. Trong môi trường ngẫu nhiên, một danh sách hành động cố định thường không đủ thay thế chính sách.

Hàm giá trị trạng thái và hành động được định nghĩa bằng kỳ vọng:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s], \qquad Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

Một reward lớn trong một lượt không bằng Q lớn. Q phản ánh lợi ích dài hạn kỳ vọng khi thực hiện hành động rồi tiếp tục theo chính sách. Mẫu sales có nhiễu, nên hai lần cùng trạng thái và hành động vẫn có thể cho reward khác nhau. Bảng Q giảm bớt nhiễu nhờ cập nhật qua nhiều quan sát, nhưng cách giảm phụ thuộc learning rate và cấu trúc bài toán.

Hàm giá trị tối ưu V^*, Q^* dùng chính sách tốt nhất trong lớp đang xét và giả định mô hình. Nếu trạng thái thiếu thông tin, một chính sách tối ưu trên biểu diễn đó có thể kém chính sách dùng bộ nhớ đầy đủ. Từ "tối ưu" luôn cần đi kèm điều kiện: tối ưu đối với trạng thái nào, môi trường nào, đối thủ nào và mục tiêu nào.

27.7 Phương trình Bellman

Với reward kỳ vọng $R(s, a)$ và chuyển trạng thái $P(s' \mid s, a)$, phương trình kỳ vọng Bellman có dạng:

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^{\pi}(s') \right].$$

Phương trình tối ưu thay trung bình theo chính sách bằng phép cực đại:

$$V^{*}(s) = \max_{a} \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^{*}(s') \right].$$

Đối với Q, ta có:

$$Q^{*}(s,a) = R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \max_{a'} Q^{*}(s',a').$$

Các biểu thức này làm rõ hai thành phần: lợi ích tức thời và giá trị của trạng thái tiếp theo. Trong một số phần nhập môn của paper, ký hiệu được rút gọn để dễ trình bày. Báo cáo thêm tổng xác suất và điều kiện chính sách để tránh hiểu rằng chỉ cộng mọi giá trị tương lai với trọng số bằng nhau.

Với chân trời hữu hạn, nên viết V_t thay vì một V không phụ thuộc thời gian, và điều kiện biên $V_H = 0$. Khi cùng một quan sát xuất hiện ở đầu và cuối episode, giá trị phần thưởng còn có thể nhận khác nhau. Một bảng Q không chứa thời gian nhưng nhận cả target terminal lẫn nonterminal sẽ trộn các ngữ cảnh này; đây là hạn chế của baseline được giữ lại để phân tích.

27.8 Dynamic programming: biết mô hình để lập kế hoạch

Section IV-A trình bày dynamic programming, gồm đánh giá chính sách, cải thiện chính sách, policy iteration và value iteration. Khi biết mô hình, ta có thể tính kỳ vọng thay vì chờ mẫu thực nghiệm. Policy evaluation giữ nguyên π và giải giá trị; policy improvement chọn hành động tốt hơn theo giá trị hiện có; policy iteration luân phiên hai bước.

Value iteration áp dụng trực tiếp toán tử tối ưu Bellman. Trong MDP hữu hạn có chiết khấu thích hợp, phép lặp có nền tảng hội tụ. Tuy nhiên, khi mô hình lớn hoặc không biết, liệt kê toàn bộ trạng thái và chuyển tiếp có thể không khả thi. Đây là lý do học từ mẫu trở nên cần thiết, chứ không phải vì quy hoạch động vốn kém thông minh hơn RL.

Trong assignment, mô hình chỉ có hai mức nhu cầu, hai hành động và phản ứng rule rõ ràng. Vì thế quy hoạch động hữu hạn chân trời là mốc kiểm tra rất mạnh. Nó giúp phân biệt việc "Q-learning đã học được một chiến lược" với việc "chiến lược đó đạt best response". Một thuật toán học phức tạp mà không so với mốc đơn giản có thể tạo ra kết luận sai về hiệu quả.

Mã DP của báo cáo dùng trạng thái $(d, a^{A}_{t-1}, a^{B}_{t-1})$ và thời điểm t. Với rule opponent, hành động hiện tại $b = d\, a^{A}_{t-1}$. Với mỗi lựa chọn a, thuật toán tính reward kỳ vọng và trung bình giá trị ở hai mức nhu cầu kế tiếp. Việc thực hiện từ $t = H - 1$ về 0 cho ra chính sách phụ thuộc thời gian một cách chính xác trong mô hình.

27.9 Monte Carlo: học từ return hoàn chỉnh

Section IV-B giới thiệu Monte Carlo như cách ước lượng giá trị qua các episode hoàn tất. Với mỗi lần thăm trạng thái, tính return thực tế rồi lấy trung bình. Nếu dữ liệu được lấy đúng từ chính sách mục tiêu và episode phù hợp, trung bình mẫu là cách tự nhiên để ước lượng kỳ vọng. Không cần biết bảng xác suất chuyển trạng thái.

Ưu điểm là không bootstrap từ một giá trị ước lượng khác. Nhược điểm là phải chờ tương lai của episode và return có thể biến động mạnh theo nhiều bước ngẫu nhiên. Nếu một episode dài gồm nhiều thành phần nhiễu, mẫu return ở đầu kỳ có phương sai lớn. First-visit và every-visit phân biệt cách dùng những lần gặp lại trạng thái trong một episode.

Off-policy Monte Carlo cần xử lý khác biệt giữa chính sách thu thập và chính sách đánh giá, thường qua importance sampling với điều kiện hỗ trợ thích hợp. Trọng số có thể rất lớn khi chuỗi dài hoặc chính sách khác nhau mạnh. Do đó việc có log lịch sử không tự động cho phép đánh giá mọi chính sách mới một cách ổn định.

Liên hệ bài lab: mỗi episode có 20 bước nên có thể bổ sung MC để đối chiếu với Q-learning. Tuy vậy, bản thực nghiệm chính không chạy MC; báo cáo không gán cho nó số liệu chưa đo. Đề xuất có thể thực hiện là giữ cùng seed và budget tương tác, thêm MC control với cùng không gian quan sát rồi so mức sai lệch so với oracle.

27.10 Temporal difference và ý nghĩa bootstrapping

Section IV-C trình bày temporal difference, kết hợp việc học từ mẫu với ước lượng giá trị tương lai. TD(0) cho trạng thái có dạng:

$$V(S_t) \leftarrow V(S_t) + \alpha \left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \right].$$

Đại lượng trong ngoặc là TD error. Khi target cao hơn dự đoán, giá trị tăng; khi thấp hơn, giá trị giảm. Không cần chờ episode hoàn tất, nhưng target phụ thuộc ước lượng hiện tại nên có thể mang bias. Nhận định "TD phương sai thấp hơn MC" là trực giác thường hữu ích, không phải bất đẳng thức phổ quát cho mọi cấu hình dữ liệu và estimator.

Bootstrapping là dùng một giá trị ước lượng để cập nhật giá trị khác. Trong Q-learning, target có phép max trên trạng thái kế tiếp. Nó truyền thông tin từ tương lai về các hành động trước. Nhưng nếu ước lượng kế tiếp không phù hợp do trạng thái thiếu hoặc môi trường thay đổi, sai lệch cũng có thể truyền ngược.

Ở terminal thật, giá trị tương lai bằng 0. Trong nhiệm vụ continuing bị cắt chỉ để quản lý thời lượng, "hết 20 bước" có thể là truncation và việc bootstrap cần khác terminal. Đề tham chiếu dùng episode hữu hạn với terminal ở bước cuối, nên báo cáo giữ quy ước này; đồng thời chỉ ra cần thêm thời gian vào trạng thái nếu muốn biểu diễn chính xác nhiệm vụ hữu hạn.

27.11 SARSA và Q-learning

SARSA cập nhật theo hành động thực sự được chọn ở trạng thái tiếp theo:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)].$$

Nó gắn với chính sách đang hành động, gồm cả khám phá, nên thường được gọi là on-policy. Q-learning thay $Q(S_{t+1}, A_{t+1})$ bằng $\max_a Q(S_{t+1}, a)$, hướng đến chính sách greedy dù dữ liệu có thể đến từ epsilon-greedy, nên là off-policy trong cách phân loại này.

Sự khác nhau không nói SARSA luôn an toàn còn Q-learning luôn rủi ro. Kết quả phụ thuộc môi trường, reward, biểu diễn và quy trình đánh giá. Ví dụ kinh điển về hành động khám phá nguy hiểm chỉ minh họa một cấu hình cụ thể. Khi thảo luận một ứng dụng kinh doanh, cần đo chi phí khám phá và ràng buộc thực tế.

Ví dụ cập nhật theo đề: $Q = 4$, $\alpha = 0.1$, reward 6, $\gamma = 0.9$, giá trị tốt nhất kế tiếp 5. Target là $6 + 0.9 \times 5 = 10.5$; TD error là 6.5; giá trị mới $4 + 0.1 \times 6.5 = 4.65$. Nếu cùng transition kết thúc thật, target chỉ là 6 và giá trị mới 4.2. Hai ca được kiểm thử riêng trong mã.

27.12 Epsilon-greedy và khám phá

Với hai hành động, epsilon-greedy chọn một hành động ngẫu nhiên trong hai lựa chọn với xác suất ϵ, còn lại chọn hành động tốt nhất theo bảng. Vì nhánh ngẫu nhiên vẫn có thể chọn hành động greedy, xác suất chọn hành động không greedy là $\epsilon/2$ khi chỉ có một greedy action. Nói epsilon bằng đúng tỷ lệ hành động sai sẽ không chính xác.

Lịch tham chiếu giảm theo episode: $\epsilon_e = \max(0.02, 0.2 \times 0.995^e)$ với e bắt đầu từ 0. Giải $0.2 \times 0.995^e \leq 0.02$ cho thấy ngưỡng sàn đạt xấp xỉ episode chỉ số 460. Hơn 4.500 episode còn lại vẫn giữ khám phá nhỏ. Khi đánh giá, báo cáo đặt epsilon bằng 0 và không cập nhật Q.

Khám phá quá ít có thể làm bỏ sót hành động, nhất là trạng thái hiếm. Khám phá quá nhiều gây mất reward trong training và có thể ảnh hưởng hành vi đối thủ. Trong bài bán hàng, một hành động giảm giá hiện tại còn làm rule opponent có khả năng giảm ở bước sau khi nhu cầu cao. Vì vậy chi phí khám phá không chỉ là margin của bước hiện tại.

Báo cáo chạy ba cấu hình khám phá: lịch giảm tham chiếu, epsilon cố định 0.05 và 0.30. Việc giữ cố định learning rate, số episode và các seed giúp so sánh có ý nghĩa. Dù vậy, một chênh lệch sau năm seed chưa đủ chứng minh cơ chế khám phá là nguyên nhân duy nhất; kết quả còn chịu nhiễu Q, aliasing và trộn thời điểm terminal.

27.13 Learning rate và điều kiện hội tụ

Learning rate điều khiển mức thay đổi trước một mẫu mới. Với target cố định y, cập nhật $Q \leftarrow (1 - \alpha)Q + \alpha y$ là một trung bình có trọng số. Alpha lớn phản ứng mạnh, alpha nhỏ tích lũy chậm. Trong thực tế target cũng thay đổi theo Q và dữ liệu nên trực giác trung bình chỉ giải thích một phần hành vi.

Các bảo đảm hội tụ cổ điển cho tabular Q-learning đòi hỏi những điều kiện về môi trường Markov và dừng, độ bao phủ state–action và lịch learning rate thỏa các tổng thích hợp, thường $\sum \alpha_t = \infty$, $\sum \alpha_t^2 < \infty$. Alpha cố định 0.1 và số episode hữu hạn của lab không

đủ để viện dẫn trực tiếp bảo đảm hội tụ về Q^*. [6]

Trong lab, alpha 0.01 có thể cần nhiều mẫu hơn để phản ánh reward; alpha 0.50 có thể nhạy với nhiễu sales và các target terminal thấp. Nhìn một đường reward được làm trơn chưa xác định được Q đã ổn định. Nên kết hợp đánh giá đóng băng, phân phối chính sách qua seed và độ lệch với oracle.

Cũng cần phân biệt ổn định và tối ưu. Một policy luôn giảm giá có thể rất ổn định nhưng lợi nhuận thấp. Một policy tốt có thể cho reward thô dao động lớn vì nhu cầu ngẫu nhiên. Chỉ số thay đổi tham số, tỷ lệ thay đổi hành động và lợi nhuận đánh giá giải đáp những câu hỏi khác nhau.

27.14 Actor–critic và policy gradient

Section IV-D giới thiệu actor–critic, nơi actor biểu diễn chính sách còn critic ước lượng chất lượng để hướng dẫn actor. Một dạng mục tiêu là $J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]$. Gradient có thể được ước lượng bằng log xác suất hành động nhân một tín hiệu lợi thế. Critic giúp giảm phương sai hoặc cung cấp tín hiệu thường xuyên hơn.

Actor–critic hữu ích khi hành động liên tục hoặc không gian quá lớn cho bảng Q. Tuy nhiên, nó thêm mô hình, learning rate và quan hệ phụ thuộc giữa hai quá trình học. Critic sai có thể hướng actor sai; actor thay đổi làm dữ liệu critic học thay đổi. Đây không phải phép thay thế mặc nhiên tốt hơn cho bài có tám giá trị Q.

Một số câu trong phần thảo luận của paper nói về hội tụ ở mức tổng quát. Báo cáo đọc thận trọng: policy gradient không được bảo đảm tìm global optimum cho mọi bài toán mạng nơ-ron và mọi ước lượng gradient; model-based cũng không được bảo đảm đúng mô hình khi lớp mô hình sai. Cần đặt các phát biểu về hội tụ trong giả định toán học cụ thể.

Liên hệ cải tiến: nếu mở rộng hành động thành mức chiết khấu liên tục và thêm tồn kho, có thể cân nhắc actor–critic hoặc phương pháp điều khiển tối ưu. Trước đó cần một môi trường được hiệu chỉnh và baseline đủ mạnh. Dùng mạng sâu sớm trong môi trường toy làm tăng khó khăn giải thích mà chưa có lý do về biểu diễn.

27.15 Bayesian methods và bất định

Section IV-E đề cập phương pháp Bayesian, dùng phân phối niềm tin để biểu diễn điều chưa biết. Thay vì giữ một con số reward trung bình duy nhất, có thể giữ posterior cho tham số xác suất chuyển đổi rồi chọn hành động theo bất định. Thompson sampling là một ví dụ về lấy mẫu giả thuyết để cân bằng khám phá và khai thác trong các bài phù hợp.

Cần tách **aleatoric uncertainty** do tính ngẫu nhiên của môi trường và **epistemic uncertainty** do thiếu hiểu biết. Dù biết xác suất mua chính xác, sales binomial vẫn ngẫu nhiên. Thu thập thêm dữ liệu giúp giảm bất định tham số nhưng không xóa tính ngẫu nhiên của từng lượt mua. Nhầm hai loại dễ dẫn đến yêu cầu một policy phải thắng trong mọi episode.

Trong bài bán hàng, nếu mô hình conversion chưa biết, có thể dùng posterior cho từng ngữ cảnh và hành động. Nhưng đối thủ phản ứng qua lịch sử khiến mô hình bandit độc lập

có thể thiếu động lực dài hạn. Một cải tiến cần xác định liệu đang học reward tức thời hay toàn bộ quá trình quyết định có trạng thái.

27.16 Deep Q-Network và các thành phần ổn định

Section IV-F trình bày DQN như cách dùng mạng nơ-ron xấp xỉ Q cho không gian trạng thái lớn. Hai thành phần nổi bật là experience replay và target network. Replay tái sử dụng transition và giảm tương quan theo thời gian trong minibatch; target network làm mục tiêu thay đổi chậm hơn so với việc dùng cùng một mạng cập nhật ngay lập tức.

Một target điển hình có dạng $y = r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a')$ cho transition không terminal, rồi tối ưu bình phương sai số giữa y và $Q_\theta(s, a)$. DQN vẫn chịu vấn đề xấp xỉ hàm, tối ưu không lồi và dữ liệu phụ thuộc chính sách. Thêm replay không tự khắc phục non-stationarity khi nhiều agent cùng học.

Với tám giá trị Q trong lab, mạng sâu sẽ có nhiều tham số hơn đại lượng cần học và tốn thêm tuning. Bảng Q là lựa chọn hợp lý để người học nhìn rõ từng giá trị, từng trạng thái và từng lỗi. Bài học từ paper là chọn công cụ theo kích thước và đặc tính bài toán, không chọn thuật toán mới chỉ vì có vẻ mạnh.

27.17 So sánh các họ phương pháp

Phương pháp	Cần mô hình chuyển tiếp?	Target chính	Điểm mạnh	Hạn chế chính
Dynamic programming	Có	Kỳ vọng Bellman	Mốc chính xác khi mô hình nhỏ, đúng	Chi phí không gian trạng thái, sai mô hình
Monte Carlo	Không	Return sau episode	Không bootstrap, diễn giải rõ	Chờ cuối kỳ, phương sai return
TD/SARSA	Không	Reward và giá trị hành động kế tiếp	Cập nhật online, theo policy hành động	Bootstrap và phụ thuộc biểu diễn
Q-learning	Không	Reward và max Q kế tiếp	Tabular nhỏ dễ triển khai	Không bảo đảm ở mọi môi trường, nhạy trạng thái thiếu
Actor–critic	Không nhất thiết	Gradient policy và critic	Phù hợp policy tham số, hành động lớn	Nhiều nguồn bất ổn và tham số
Bayesian	Tùy thiết kế	Posterior và quyết định dưới bất định	Biểu diễn điều chưa biết	Chi phí suy luận, giả định prior/model
DQN	Không cần mô hình tường minh	Bellman target từ mạng mục tiêu	Xấp xỉ Q trên đầu vào lớn	Data hungry, khó giải thích, cần ổn định

Các trục phân loại không hoàn toàn loại trừ nhau. Bayesian có thể dùng cho model-based hoặc một thành phần reward; actor–critic có thể dùng mô hình hoặc không; deep RL có thể value-based hoặc policy-based. Do vậy không nên vẽ một cây phân loại duy nhất rồi coi mỗi thuật toán chỉ được đặt vào đúng một lá.

27.18 Ứng dụng trong paper: đọc bằng cấu trúc bài toán

27.18.1 Healthcare

Paper xem xét lựa chọn điều trị tuần tự, cá nhân hóa và một số quy trình hỗ trợ. Điểm phù hợp với RL là quyết định hiện tại ảnh hưởng sức khỏe và lựa chọn ở tương lai. Nhưng việc quan sát dữ liệu điều trị lịch sử không tương đương có quyền khám phá trực tiếp trên bệnh nhân. Nhiễu quan sát, confounding và thiếu độ bao phủ hành động khiến đánh giá chính sách khó.

Báo cáo chỉ dùng miền healthcare ở **phân loại yêu cầu hành chính**, không tạo chính sách điều trị, chẩn đoán hay khuyến nghị thuốc. Liên hệ này giúp nhận ra khác biệt mức rủi ro giữa một ví dụ nhập môn và ứng dụng clinical mà paper khảo sát. Các chi tiết y khoa thực hành nằm ngoài phạm vi bộ bài.

27.18.2 Robotics và điều khiển tự động

Robot có chuỗi quan sát–hành động liên tục và hậu quả động học, nên RL là khung tự nhiên. Phần thưởng có thể gắn với hoàn thành nhiệm vụ, tiêu hao năng lượng hoặc tránh va chạm. Khó khăn là tương tác thật tốn kém và hành động sai có thể gây hỏng vật lý. Mô phỏng giúp thu thập dữ liệu nhưng có khoảng cách sim-to-real.

Một thí nghiệm robot tốt không chỉ báo reward trung bình; cần tỷ lệ thành công, vi phạm giới hạn, độ bền qua điều kiện tải và khả năng khôi phục. Các chỉ số này có thể được chuyển thành tư duy cho tutorial: một hệ trả lời nhanh nhưng không giữ bất biến ngân sách vẫn thất bại nhiệm vụ.

27.18.3 Communication và networking

Ứng dụng gồm điều khiển tài nguyên, lập lịch, đường truyền hoặc cấu hình. Trạng thái mạng biến động và quyết định cục bộ có thể ảnh hưởng người dùng khác. RL phù hợp khi có tương tác lặp và feedback, nhưng không thể bỏ qua độ trễ quyết định và sự thay đổi phân phối lưu lượng.

Nếu nhiều node học độc lập, throughput của một node phụ thuộc chính sách của các node còn lại. Khi mọi node cập nhật, mỗi node quan sát một môi trường hiệu dụng đang đổi. Đây là cầu nối trực tiếp đến vấn đề non-stationarity trong câu 8 của assignment.

27.18.4 NLP

Paper thảo luận ứng dụng RL trong các tác vụ ngôn ngữ, đối thoại và tạo chuỗi, nơi đầu ra dài và tiêu chí cuối khó chia thành nhãn cho từng bước. Phần thưởng có thể là độ hoàn thành mục tiêu hoặc đánh giá chất lượng. Tuy nhiên, một reward dễ tối ưu có thể khác mục tiêu con người thực sự muốn.

Cần giữ bối cảnh thời gian: bài công bố năm 2020 và không phải survey về toàn bộ các kỹ thuật căn chỉnh LLM xuất hiện sau đó. Báo cáo dùng nó cho nền tảng RL và không gán các mô hình mới vào danh sách đóng góp nguyên bản. So sánh với paper MAS năm 2025 cần nêu khoảng cách này.

27.18.5 Autonomous IoT và computer vision

Trong IoT, bài toán thường liên quan năng lượng, kết nối, thu thập dữ liệu và phân bổ tài nguyên. Hành động có thể làm thay đổi thời lượng pin hoặc mức thông tin nhận được. Với computer vision, RL có thể tham gia lựa chọn vùng quan sát hoặc quyết định tuần tự, trong khi nhiều bài nhận dạng thuần túy vẫn thích hợp hơn với supervised learning.

Bài học lựa chọn phương pháp: có ảnh hoặc có sensor chưa đủ để kết luận cần RL. Nếu nhiệm vụ chỉ dự đoán nhãn từ một ảnh và hành động không ảnh hưởng dữ liệu tương lai, classifier có thể phù hợp hơn. RL được biện minh bởi cấu trúc quyết định và phản hồi, không phải loại dữ liệu đầu vào.

27.18.6 Games, self-organized systems và scheduling

Trò chơi cung cấp môi trường có luật rõ, vòng tương tác lặp và công cụ đo reward, vì vậy thuận tiện cho nghiên cứu. Tuy nhiên, thành công trong trò chơi có thông tin sạch không đồng nghĩa thành công trong hệ kinh doanh có nhu cầu thay đổi và hạn chế can thiệp. Cần kiểm tra cơ chế chuyển từ benchmark sang miền ứng dụng.

Trong scheduling, quyết định hiện tại có thể chiếm tài nguyên và ảnh hưởng các công việc về sau. Ràng buộc cứng như deadline hoặc năng lực máy cần được thể hiện trực tiếp. Nếu chỉ thêm một penalty nhỏ vào reward, agent có thể chọn vi phạm để đổi lợi ích khác. Điều này tương tự việc phải lọc sản phẩm vượt ngân sách trước khi xếp hạng trong tutorial.

27.19 Các thách thức lớn và đọc phản biện

27.19.1 Sample efficiency và độ bao phủ

Paper nhấn mạnh chi phí dữ liệu và không gian trạng thái lớn. Đối với lab, 5.000 episode tương đương 100.000 bước mỗi lượt training, nhưng không có nghĩa mọi state–action được quan sát đủ cân bằng. Một trạng thái chứa đối thủ giảm giá có thể hiếm khi policy tiến về không giảm giá. Chỉ báo tổng số bước che khuất sự thiếu bao phủ cục bộ.

Đánh giá sample efficiency nên xem hiệu năng theo số tương tác và dùng cùng ngân sách giữa thuật toán. So sánh một agent được train gấp mười lần với baseline ít dữ liệu sẽ

không tách được chất lượng phương pháp. Một đề xuất mở rộng là lưu visit count cho từng state–action và phân tích sai lệch Q ở các ô hiếm.

27.19.2 Reward design và reward hacking

Mục tiêu được số hóa có thể bỏ sót điều người dùng quan tâm. Nếu reward là số đơn, agent dễ ưu tiên giảm giá mạnh dù lợi nhuận thấp. Nếu reward là doanh thu, nó có thể bán hàng lỗ. Nếu reward chỉ là lợi nhuận ngắn hạn, nó có thể không tính trải nghiệm khách hàng hoặc biến động giá. Việc định nghĩa reward là một phần nội dung khoa học, không chỉ tham số code.

Cải tiến phù hợp là tách objective và constraints: tối đa lợi nhuận trong miền hành động tuân thủ các điều kiện về margin và ngân sách. Có thể đưa một số ưu tiên mềm vào reward, nhưng phải công bố trọng số và phân tích độ nhạy. Một con số tổng hợp cao không chứng minh mọi mặt đều được cải thiện.

27.19.3 Partial observability và non-stationarity

Khi chỉ quan sát một phần, agent có thể cần lịch sử hoặc belief state. Trong lab, rule opponent dùng hành động trước của agent A, nhưng bảng Q của A chỉ nhận nhu cầu và hành động trước của B. Hai lịch sử khác nhau có cùng quan sát A nhưng dẫn đến hành động hiện tại khác nhau của B. Đây là ví dụ cụ thể về state aliasing, ngay cả khi đối thủ không học.

Khi cả hai agent cùng học, chính sách đối thủ thay đổi qua episode nên reward và chuyển tiếp hiệu dụng đối với một learner thay đổi. Frozen evaluation sau training giúp đo chất lượng cặp policy tại một thời điểm, nhưng không chứng minh quá trình học đã tiến đến cân bằng. Cần thêm cross-play, nhiều seed và phân tích dao động hoặc best response.

27.19.4 Bias–variance và những phát biểu cần điều kiện

Phần Challenges của paper so sánh MC, TD và biến động ước lượng. Báo cáo giữ trực giác rằng target dài có thể nhiều nhiễu hơn, còn bootstrap có thể mang sai lệch, nhưng không biến trực giác đó thành kết luận tuyệt đối. Một estimator cụ thể có thể có hành vi khác tùy dữ liệu và cấu trúc môi trường.

Paper cũng có những lời khẳng định rộng về dữ liệu tổng hợp và giảm bias. Dữ liệu tổng hợp không bảo đảm loại thiên lệch; nó có thể tái tạo hoặc làm nặng thiên lệch mô hình sinh. Trong tutorial, classifier đạt 100% trên test tổng hợp nhưng vẫn nhận sai hai câu ngoài miền. Đây là minh chứng thực tế vì sao cần tách chất lượng trên phân phối tạo dữ liệu khỏi độ bền ngoài phân phối.

27.20 Kiểm tra giả định trên chính môi trường assignment

Gọi trạng thái đầy đủ $x_t = (d_t, u_t, v_t)$ với $u_t = a^A_{t-1}$ và $v_t = a^B_{t-1}$. Với rule opponent, $b_t = d_t u_t$. Quan sát tham chiếu của A là $o_t = (d_t, v_t)$, không có u_t. Xét $d_t = 1, v_t = 0$: nếu $u_t = 0$ thì B

chọn 0; nếu $u_t = 1$ thì B chọn 1. Cùng một quan sát và cùng hành động A dẫn đến reward kỳ vọng khác nhau.

Ví dụ A không giảm: ở nhu cầu cao, B không giảm cho conversion 0.55 và lợi nhuận kỳ vọng $20 \times 0.55 \times 5 = 55$; B giảm cho conversion 0.45 và kỳ vọng 45. Chênh 10 không phải nhiễu ngẫu nhiên, mà do thông tin thiếu. Vì A biết hành động mình vừa làm, việc bổ sung bit này là khả thi và chi phí nhỏ.

Một yếu tố khác là thời điểm. Ở bước cuối, Q chỉ gồm reward ngay; ở bước đầu còn nhiều reward tương lai. Nếu dùng một bảng cho cả hai mà không có thời gian, cùng state–action nhận target theo những chân trời khác nhau. Báo cáo giữ baseline tám giá trị để đúng tinh thần đề, rồi dùng DP với đầy đủ trạng thái và thời gian để kiểm tra.

27.21 Phân tích lợi nhuận kỳ vọng và chính sách mốc

Với $a, b \in \{0, 1\}$, conversion ở các tham số mặc định luôn nằm giữa 0.20 và 0.70 nên clip không hoạt động. Reward kỳ vọng là:

$$\bar{r}(d, a, b) = (10 + 10d)(0.30 + 0.25d + 0.15a - 0.10b)(5 - 2a).$$

Nếu $b = 0$, nhu cầu thấp: không giảm thu 15, giảm thu 13.5; nhu cầu cao: không giảm 55, giảm 42. Nếu $b = 1$, các giá trị lần lượt là 10 và 10.5 ở thấp, 45 và 36 ở cao. Vì rule opponent chỉ giảm khi nhu cầu cao và A vừa giảm ở bước trước, trường hợp thấp mà B giảm không xuất hiện trong baseline.

Không giảm là tốt hơn tức thời ở các trạng thái mà rule opponent tạo ra. Nó còn tránh kích hoạt phản ứng giảm giá của B ở bước kế tiếp khi nhu cầu cao. Quy hoạch động xác nhận hành động 0 ở mọi thời điểm và trạng thái đầy đủ trong mô hình tham chiếu. Đây là best response với đối thủ đã định nghĩa, không phải chứng minh luôn không giảm là tối ưu cho mọi thị trường.

Khi cả hai khởi đầu với lịch sử 0 và dùng rule, không bên nào tạo ra tín hiệu giảm giá nên cả hai luôn chọn 0. Nhu cầu thấp/cao có xác suất 1/2, vì vậy lợi nhuận kỳ vọng mỗi bước là $(15 + 55)/2 = 35$. Kết quả đánh giá thực nghiệm dao động quanh 35 do nhu cầu và sales; kỳ vọng 35 không được chép thành mọi số đo của bảng kết quả.

27.22 Cầu nối với MAS và phương pháp nghiên cứu chung

Paper MAS nhấn mạnh kênh phối hợp, còn paper RL nhấn mạnh điều chỉnh chính sách bằng feedback. Hai quan điểm bổ sung: một hệ có thể thiết kế cơ chế giao tiếp rất rõ nhưng policy quyết định kém; một learner có reward tốt trong mô phỏng vẫn có thể bị tích hợp sai vào một workflow. Cần kiểm tra cả vòng học và hợp đồng vận hành.

Ví dụ nếu chuyển learner bán hàng thành một thành phần của hệ thương mại, nó chỉ nên đề xuất mức giảm trong tập được phép. Agent chính sách kiểm tra margin, inventory xác nhận khả năng giao hàng và hệ giao dịch thực hiện với xác nhận tương ứng. Reward training không tự cấp quyền thay đổi giá hoặc bỏ qua dữ liệu kho.

Một khuôn nghiên cứu chung là: mô hình hóa giả định; xây baseline đơn giản; lưu dữ

liệu thô; đánh giá đóng băng; phân tích theo loại trạng thái; thử thay đổi môi trường; và nêu giới hạn suy rộng. Cách làm này có giá trị hơn kết luận "AI học được nên thông minh hơn luật" từ một lượt chạy.

27.23 Kết luận riêng cho Paper 2

Paper cung cấp đường đi từ MDP đến các họ thuật toán và ứng dụng. Phần có giá trị cho bài học là những câu hỏi nền tảng: trạng thái có đủ không, reward đo gì, khám phá tốn gì, target được tính thế nào và kết quả có được đánh giá độc lập không? Khi áp dụng vào assignment, các câu hỏi này phát hiện những vấn đề cụ thể mà việc chỉ chạy chương trình tham chiếu sẽ bỏ qua.

Báo cáo bổ sung ký hiệu kỳ vọng rõ, phân biệt Markov với tính dừng, hữu hạn chân trời với continuing, hiệu năng training với evaluation và tối ưu với hội tụ. Các thực nghiệm đi kèm không nhằm thay thế survey, mà biến kiến thức tổng quan thành một trường hợp có thể tính tay, chạy lại và phản biện bằng dữ liệu.

28 Chuyên đề toán học và kỹ thuật bổ sung

Phần này phát triển các phép tính và thiết kế cụ thể để làm rõ hai paper. Đây là phân tích của báo cáo trên mô hình được cung cấp, không phải các định lý hoặc kết quả thực nghiệm mới được gán cho tác giả hai survey. Các suy luận đi từ giả định đến hệ quả, giúp kiểm tra tính nhất quán giữa mô hình, code và kết luận.

28.1 1. Từ collaboration channel đến hợp đồng có thể kiểm

Giả sử một ranker trả ba sản phẩm, mỗi sản phẩm có ID, score và lời giải thích. Coordinator muốn trả kết quả cho người dùng cần kiểm ít nhất bốn thuộc tính: ID có trong catalog; ID đến từ tập candidate của truy vấn hiện tại; giá nằm trong ngân sách; kho đã được đọc với tuổi dữ liệu hợp lệ. Score cao không thay thế bất kỳ điều kiện nào trong bốn điều kiện đó.

Có thể biểu diễn một proposal z bằng predicate:

$$\text{Valid}(z) = \text{KnownID}(z) \wedge \text{Retrieved}(z, q) \wedge \text{Affordable}(z, B) \wedge \text{FreshStock}(z, \tau).$$

Các predicate dùng dữ liệu từ những nguồn có thẩm quyền khác nhau. Catalog xác nhận ID và price; retrieval trace xác nhận candidate provenance; inventory xác nhận stock và thời điểm. Coordinator không cần làm lại mọi công việc chuyên môn, nhưng phải biết nguồn nào quyết định thuộc tính nào. Nếu output một agent nói "đã kiểm tra tất cả" mà không có dữ liệu xác nhận, chuỗi kiểm chứng bị đứt.

Ví dụ một message giả trả p99 với giá 0 có thể vượt qua kiểm budget nhưng thất bại KnownID. Một message trả p1 đúng catalog nhưng không thuộc truy vấn hoặc vượt budget lại thất bại điều kiện khác. Việc dùng nhiều predicate ngăn một dấu hiệu bề ngoài hợp lệ che mất sai sót ở thuộc tính khác. Đây là cách cụ thể hóa ý tưởng kênh cộng tác trong paper MAS.

28.2 2. State machine cho workflow thay vì vòng chat không giới hạn

Một quy trình mua hàng có thể có các trạng thái RECEIVED, SEARCHED, RANKED, VALIDATED, STOCK_VERIFIED và COMPLETED. Nhánh lỗi gồm INVALID_INPUT, NO_RESULTS, TIMEOUT, CONTRACT_ERROR và AVAILABILITY_UNVERIFIED. Điều kiện chuyển trạng thái phải được định nghĩa trước: chỉ response đúng envelope mới cho phép sang bước sau; chỉ dữ liệu kho mới và đủ ID mới cho phép COMPLETED có claim verified.

Bất biến ở cấp state machine có thể phát biểu: mọi item trong COMPLETED đã đi qua VALIDATED và STOCK_VERIFIED trong cùng task ID. Một response cũ đến sau timeout không được hồi sinh yêu cầu đã kết thúc thành thành công, trừ khi thiết kế explicitly cho phép một phiên mới. Tutorial không có queue response muộn vì lời gọi là coroutine cục bộ, nhưng task ID đã tạo ranh giới phù hợp để mở rộng.

Lợi ích của state machine là phân biệt lỗi chuyên môn và lỗi giao tiếp. Empty search là một kết quả hợp lệ của CatalogAgent; timeout là không có kết quả đúng hạn; malformed là

vi phạm hợp đồng. Nếu tất cả đều chuyển thành danh sách trống không có status, người sử dụng và nhà phát triển không thể biết nên sửa truy vấn, thử lại hay kiểm tra dữ liệu.

28.3 3. Complexity của giao tiếp và đường găng

Với n tác tử và r vòng trao đổi, một topology đầy đủ có hướng có thể tạo $r\,n(n-1)$ message. Cấu trúc tập trung gọi mỗi worker một lần theo request–response có khoảng $2n$ message. Nhưng số message chỉ là một phần chi phí. Nếu mỗi message lặp một context dài L token, tổng token có thể tăng theo L nhân số lượt, cộng với output và phần tính toán của mô hình.

Một mô hình chi phí do báo cáo đề xuất là:

$$C_{total} = \sum_{v \in V} C_{compute}(v) + \sum_{e \in E} C_{message}(e) + C_{retry} + C_{verification}.$$

Thời gian hoàn tất có thể gần độ dài đường găng trong DAG cộng scheduling overhead. Nếu catalog và dữ liệu chính sách độc lập, hai thao tác có thể chạy cùng lúc. Nếu inventory chỉ nhận danh sách sau xếp hạng, nó nằm sau ranker. Việc chạy mọi agent song song mà chưa có input hợp lệ không phải tối ưu hợp lý.

Một baseline công bằng nên được cấp cùng dữ liệu và quyền công cụ. Nếu MAS chạy song song trên bốn CPU còn baseline chỉ một CPU, chênh lệch latency trộn kiến trúc với tài nguyên. Nếu cả hai đều dùng dịch vụ ngoài, cần cùng timeout, pool và cache. Bảng cục bộ của tutorial ghi rõ giới hạn này, đặc biệt baseline timeout được tiêm đồng bộ.

28.4 4. Mô hình lỗi dây chuyền và giá trị kiểm chứng

Giả sử một pipeline có k bước bắt buộc và mỗi bước thành công độc lập với xác suất p_i. Xác suất mọi bước thành công là $\prod_i p_i$. Nếu bốn bước đều đạt 0.95 thì tích là khoảng 0.8145, không còn 0.95. Đây chỉ là ví dụ giả định độc lập để giải thích vì sao nhiều thành phần tạo thêm rủi ro; hệ thật có thể có tương quan lỗi và fallback.

Một validator có thể phát hiện tỷ lệ d của lỗi nhưng cũng có false rejection. Giá trị của nó phụ thuộc chi phí sai sót và chi phí từ chối. Nếu lỗi tạo claim tồn kho sai có hậu quả đáng kể, chặn một kết quả chưa xác minh có thể là lựa chọn đúng dù làm giảm số danh sách được trả. Vì vậy success không nên định nghĩa chỉ bằng tỷ lệ có output không rỗng.

Điểm kiểm chứng tốt nhất nằm ở nơi có thông tin độc lập. Một LLM reviewer đọc cùng output không có catalog khó phát hiện p99 là bịa. Một phép kiểm membership trong tập ID thực hiện điều đó xác định. Khi nhiệm vụ có công cụ kiểm chứng chính xác, cần tận dụng thay vì chỉ thêm một vai trò hội thoại.

28.5 5. Đa số và tương quan sai số

Với ba bộ trả lời độc lập có cùng xác suất đúng p, đa số đúng khi đúng cả ba hoặc đúng đúng hai. Kết quả $p^3 + 3p^2(1-p)$ tăng so với p nếu $p > 0.5$ trong mô hình nhị phân này. Nếu $p < 0.5$, đa số có thể làm kết quả tệ hơn. Vì vậy ngay mô hình độc lập đơn giản cũng đòi hỏi chất lượng thành viên đủ tốt.

Trong hệ LLM, lỗi không độc lập: các thành viên có thể chung backbone, dữ liệu hoặc prompt; sau vòng đầu, chúng đọc câu trả lời của nhau và càng tương quan. Thêm vòng tranh luận có thể cung cấp phản biện, nhưng cũng có thể gây anchoring. Một thí nghiệm tốt cần đo cấu hình độc lập trước trao đổi và sau trao đổi, không chỉ báo cuối cùng có đa số.

Đa dạng hữu ích có thể đến từ công cụ kiểm khác nhau hoặc nguồn dữ liệu độc lập. Người tính bằng Python và người kiểm đơn vị vật lý có khả năng phát hiện những nhóm lỗi khác nhau. Đa dạng persona không đủ để suy ra đa dạng năng lực. Đây là điểm quan trọng khi chuyển taxonomy role-based thành thiết kế nghiên cứu thực tế.

28.6 6. Confusion matrix, macro-F1 và selective prediction

Confusion matrix M_{ij} đếm số ví dụ có nhãn thật i nhưng dự đoán j. Precision lớp i là $M_{ii}/\sum_j M_{ji}$; recall là $M_{ii}/\sum_j M_{ij}$; F1 là trung bình điều hòa của hai chỉ số. Macro-F1 lấy trung bình đều các lớp, hữu ích khi không muốn lớp đông áp đảo chỉ số tổng.

Trong dataset tutorial, ba lớp có cùng 36 câu test và toàn bộ đường chéo là 36, nên accuracy và macro-F1 đều bằng 1. Kết quả này được tính từ predictions CSV, không suy ra từ training loss. Một mô hình có loss nhỏ không tự động được phép gọi là đạt 100% nếu chưa so dự đoán với nhãn held-out.

Với ngưỡng τ, coverage là tỷ lệ ví dụ được chấp nhận và selective risk là tỷ lệ sai trong tập chấp nhận. Một hệ có thể tăng độ đúng bằng cách từ chối nhiều hơn; cần báo đồng thời coverage. Tuy nhiên, các đường coverage–risk trên in-domain validation không mô tả đầy đủ false acceptance OOD. Sáu probe ngoài miền của tutorial cho thấy một câu xa nhiệm vụ vẫn có thể có confidence trên 0.9.

28.7 7. Vì sao chia theo nhóm dữ liệu quan trọng

Nếu sinh nhiều câu bằng một template rồi chia ngẫu nhiên từng câu, train và test có thể chỉ khác một hậu tố nhỏ. Mô hình dễ khai thác mẫu bề mặt và test trở nên lạc quan. Chia theo họ động từ trong tutorial làm các pattern mở đầu test không có nguyên dạng trong train, giảm một loại rò rỉ.

Dù vậy, các đối tượng như laptop, bảo tàng hoặc giấy tờ bảo hiểm vẫn lặp giữa split; vocabulary chỉ xây trên train nhưng các token nổi bật của từng class vẫn xuất hiện. Đây là lý do kết quả không được gọi là tổng quát hóa ngôn ngữ tự nhiên rộng. Một split khó hơn có thể giữ riêng cả nhóm đối tượng hoặc dùng câu người dùng viết độc lập.

Tập validation được dùng chọn epoch và threshold; test dùng đo cuối. Nếu sau khi đọc các lỗi OOD ta sửa lexicon, các câu đã xem trở thành dữ liệu development. Muốn báo hiệu năng mới đáng tin, phải tạo test khác. Tính độc lập là thuộc tính của quy trình nghiên cứu, không chỉ của tên tệp train/test.

28.8 8. Chứng minh observation baseline chưa đủ Markov

Xét hai lịch sử có cùng $d_t = 1$ và hành động trước của B bằng 0. Lịch sử thứ nhất có $a^A_{t-1} = 0$, thứ hai có $a^A_{t-1} = 1$. Theo rule B, hành động hiện tại lần lượt là 0 và 1. A không

giảm trong cả hai trường hợp. Khi đó reward kỳ vọng A là 55 hoặc 45. Cùng observation và action nhưng khác luật reward có điều kiện theo phần lịch sử bị bỏ.

Kết luận không phải "mọi Q-learning ở đây vô nghĩa". Bảng Q vẫn có thể học một chính sách tốt trong lớp phản ứng theo observation. Nhưng không thể áp nguyên các giả định tabular MDP đã đủ trạng thái. Việc thêm own previous action là một cách sửa có lý do rõ, vì A vốn có thể ghi nhớ hành động của chính mình mà không cần cảm biến mới.

Nếu opponent thay luật dựa trên cả mười bước quá khứ, một bit lịch sử có thể vẫn chưa đủ. Khi đó cần state augmentation lớn hơn, belief hoặc mô hình đối thủ. Chọn state là quá trình cân bằng đủ thông tin và chi phí biểu diễn, không phải chỉ chọn vector ngắn để code thuận tiện.

28.9 9. Hữu hạn chân trời và terminal aliasing

Trong nhiệm vụ 20 bước, cùng một trạng thái kinh tế ở bước 1 và bước 20 có số bước tương lai khác nhau. Một hàm giá trị chính xác nên có chỉ số t. Ở bước 20, return chỉ gồm reward hiện tại; ở bước 1 còn 19 reward sau đó. Nếu bảng Q không chứa thời gian nhưng update cuối dùng target không bootstrap, nó nhận các target không đồng nhất cho cùng ô.

Có hai cách mô hình hóa hợp lệ tùy ý nghĩa bài toán. Nếu 20 bước là horizon thật, thêm thời gian và terminal ở cuối. Nếu thị trường continuing còn 20 chỉ là cách cắt log, có thể bootstrap qua điểm cắt thích hợp thay vì xem đó là terminal kinh tế. Đề tham chiếu dùng cách đầu nhưng biểu diễn không có thời gian; báo cáo giữ để đối chiếu rồi bổ sung oracle đầy đủ.

Sửa terminal mà không hiểu mục tiêu có thể gây lỗi khác. Một số bài thật có thanh lý tồn kho cuối kỳ, khi đó reward ở terminal còn bao gồm giá trị tồn và nghĩa vụ chưa hoàn tất. Chỉ gán future value 0 mà bỏ các phần đó cũng là mô hình hóa sai. Vì vậy thảo luận terminal cần bắt đầu từ bài toán, không từ công thức code.

28.10 10. Quy hoạch động lùi: một ví dụ hai bước

Dùng horizon hai bước với rule opponent và gamma 0.9. Ở bước cuối, khi nhu cầu thấp thì B không giảm; action 0 có reward kỳ vọng 15 so với 13.5. Khi nhu cầu cao, B có thể là 0 hoặc 1; action 0 lần lượt đạt 55 hoặc 45, đều lớn hơn 42 hoặc 36 của action 1. Do đó hành động cuối tối ưu là 0 ở mọi state.

Ở bước đầu, chọn action 0 làm own previous của bước sau bằng 0, nên rule B không giảm bước sau. Giá trị tối ưu kỳ vọng bước sau là 35. Chọn action 1 làm B giảm nếu nhu cầu sau cao; giá trị khi A chọn 0 ở bước sau là $(15 + 45)/2 = 30$. Như vậy action 1 còn mất thêm $0.9(35 - 30) = 4.5$ giá trị tương lai so với action 0, ngoài chênh reward tức thời.

Lập luận này cho thấy cơ chế dài hạn trong mô hình. Nhu cầu độc lập không làm bài toán thành các lựa chọn tức thời độc lập vì opponent phản ứng theo lịch sử action. Với horizon 20, DP lùi kiểm toàn bộ state/time và cho cùng kết luận hành động 0. Đây là ví dụ liên hệ trực tiếp Bellman với tương tác đa tác tử.

28.11 11. Giá trị kỳ vọng của policy bằng forward occupancy

Để kiểm tra số đo mà không cần thêm random seeds, báo cáo triển khai một phép tính chính xác cho policy cố định. Gọi $\mu_t(x)$ là xác suất full state $x = (d, u, v)$ ở bước t. Khởi đầu $\mu_0(0,0,0) = \mu_0(1,0,0) = 1/2$. Với action $a = \pi(d, v)$ và $b = d\,u$, trạng thái sau là (d', a, b), trong đó mỗi d' có xác suất $1/2$.

Cập nhật occupancy:

$$\mu_{t+1}(d', a, b) \mathrel{+}= \frac{1}{2}\mu_t(d, u, v).$$

Reward kỳ vọng mỗi bước là $\sum_x \mu_t(x)\bar{r}(x, \pi(x), b(x))$; trung bình qua 20 bước cho exact policy mean không chiết khấu. Đây là đánh giá một policy bằng forward probability, khác với DP lùi tìm policy tốt nhất. Cả hai dùng cùng full state để giữ đúng phản ứng của đối thủ.

Tệp `assignment/analyze_exact.py` áp dụng phép tính cho mọi Q-table đã lưu. Policy bits theo thứ tự observation (0,0), (0,1), (1,0), (1,1) giúp đọc hành vi. Một policy có bit 1 ở state không thể tới có thể vẫn đạt cùng mean 35 như policy 0000. Bởi vậy so bảng policy từng ô không thay thế phân tích trạng thái thực sự được thăm.

28.12 12. Tách lỗi học khỏi nhiễu evaluation bằng exact values

Baseline seed 11 và 55 có policy bits 0000, exact mean 35. Seed 44 có 0001 nhưng state gây discount không xuất hiện từ điều kiện khởi đầu khi policy không tạo discount trước, nên exact mean cũng 35. Seed 22 và 33 có 1000, exact mean xấp xỉ 32.448. Lợi nhuận thấp hơn ở hai seed sau là khác biệt policy thực sự, không chỉ nhiễu test.

Lượt minh họa seed 42 có exact mean xấp xỉ 25.698, trong khi evaluation sample mean là 25.5591. Điều này xác nhận policy snapshot thật sự kém trong mô hình chứ không đơn thuần gặp một tape rất xấu. Sự khác biệt nhỏ giữa exact và sampled mean là nhiễu hữu hạn; khoảng cách lớn với 35 chủ yếu nằm ở policy đã học.

Bảng exact cho toàn bộ 25 cấu hình/seed được xuất trong `exact_policy_values.csv`. Phép tính này chỉ khả thi vì môi trường nhỏ và biết đầy đủ reward/transition. Trong ứng dụng thật, không thể gọi một mô hình chưa hiệu chỉnh là exact oracle; độ đúng khi ấy bị giới hạn bởi sai số mô hình.

28.13 13. Kỳ vọng và phương sai profit ở một bước

Nếu action và demand cố định, $S \sim \text{Binomial}(N, p)$ và margin là m, thì $R = mS$. Vì vậy $\mathbb{E}[R] = mNp$ và $\text{Var}(R) = m^2Np(1-p)$. Với không giảm và demand thấp, $N = 10, p = 0.3, m = 5$, ta có mean 15 và variance 52.5. Demand cao với $p = 0.55$ cho mean 55 và variance 123.75.

Nếu demand thấp/cao đều xác suất 1/2, dùng luật phương sai toàn phần:

$$\text{Var}(R) = \mathbb{E}[\text{Var}(R \mid d)] + \text{Var}(\mathbb{E}[R \mid d]) = 88.125 + 400 = 488.125.$$

Standard deviation một bước khoảng 22.094, dù policy hoàn toàn xác định. Với never-discount đối đầu rule và reset như đề, nhu cầu và sales qua các bước độc lập nên SD của average 20 bước xấp xỉ $\sqrt{488.125/20} = 4.94$. Các SD episode đo quanh 5 là phù hợp với cấu trúc này. Một reward curve gồ ghề vì thế không tự chứng minh policy đang thay đổi.

Khi policy phụ thuộc lịch sử, reward qua bước có thể tương quan; không còn được chia variance cho T một cách máy móc. Dùng sample SD trên run means và protocol nhiều seed giúp tránh một phần sai lầm, nhưng muốn ước lượng uncertainty chính xác cần thiết kế phù hợp với cấu trúc phụ thuộc.

28.14 14. Confidence interval và đơn vị mẫu

Báo cáo có năm training seeds. Mỗi run tạo một policy rồi đánh giá trên một tape riêng. Lấy năm mean để tính SD phản ánh kết hợp biến động học và evaluation. Nếu coi 50.000 bước evaluation gộp là 50.000 mẫu độc lập của chất lượng thuật toán, độ chắc chắn sẽ bị phóng đại vì nhiều bước dùng cùng một policy được học từ cùng run.

Với năm mean, khoảng t minh họa dùng 4 bậc tự do. Khoảng rộng là tín hiệu dữ liệu còn ít hoặc policy không ổn định. Không có ý nghĩa khi giảm interval bằng cách chỉ lặp đánh giá một policy tốt đã chọn rồi bỏ các training seeds kém. Đó là selection bias, không phải tăng độ tin cậy.

So sánh paired nên dùng chênh lệch hai policy trên cùng tape. Tuy nhiên, nếu một số seed cho policy giống hệt baseline, các chênh lệch bằng 0 là cấu trúc có ý nghĩa. Năm seed quá ít để mạnh tay khẳng định thứ hạng phổ quát; bảng từng seed thường cho nhiều thông tin hơn một dấu sao significance.

28.15 15. Robustness: zero-shot shift khác retraining

Trong stress test discount 40%, frozen policy giữ nguyên Q học ở 20% và chỉ thay reward environment. Đây là đo zero-shot robustness trước một thay đổi margin. Retrained policy được học lại ở 40%, đo khả năng tìm policy khi dữ liệu phù hợp chế độ mới. Hai thiết lập trả lời hai câu hỏi khác nhau và không được gộp thành một hàng "agent thích nghi".

Những policy luôn không giảm không bị ảnh hưởng vì action 0 không dùng discount fraction. Những policy còn giảm chịu margin từ 3 xuống 1, nên profit kém hơn. Retraining có thể sửa policy nhưng không bảo đảm từng seed thành công do baseline vẫn có thiếu trạng thái và target noise. Tính học không tạo bảo đảm thích nghi cho một lần train hữu hạn.

Một nghiên cứu robustness đầy đủ nên có lưới demand transition, conversion uplift, cost, opponent và horizon. Chọn tham số trên train/validation regimes, rồi giữ test regimes độc lập. Đo mean, worst-case và regret so với oracle từng regime, thay vì chỉ báo một điều kiện có lợi cho phương pháp đề xuất.

28.16 16. Đối thủ cố định, self-play và cân bằng

Best response với đối thủ cố định tối ưu lợi ích khi policy bên kia không đổi. Self-play là quá trình cả hai cập nhật; evaluation cuối chỉ quan sát chất lượng cặp policy đã đóng băng.

Nash equilibrium yêu cầu không bên nào có thể tăng lợi ích bằng cách đơn phương đổi policy trong lớp trò chơi đã định nghĩa. Một trận có reward gần nhau không chứng minh điều kiện này.

Trong lab, cấu trúc reward khiến không giảm là rất mạnh trước rule, nhưng không nên tự tuyên bố mọi policy self-play đã hội tụ cân bằng. Cần tính hoặc xấp xỉ best response với từng opponent snapshot và đo exploitability. Với không gian nhỏ, có thể liệt kê 16 policy reactive theo bốn observation; với policy có lịch sử đầy đủ, lớp phương án rộng hơn nhiều.

Cross-play giữa checkpoint và seed giúp phát hiện co-adaptation: hai policy tốt khi chơi cùng nhau nhưng kém với đối thủ khác. Bộ bài có self-play và dữ liệu cuối trên năm seed, chưa triển khai toàn bộ ma trận cross-play. Vì vậy báo cáo giữ kết luận ở mức minh họa non-stationarity và biến động kết quả, không gán chứng nhận equilibrium.

28.17 17. Từ thuật toán sang thiết kế có thể tái lập

Một kết quả tái lập cần nguồn, tham số, seed, phiên bản thư viện, protocol và dữ liệu đầu ra. Chỉ công bố seed 42 thiếu thông tin về số lần gọi RNG, cách phá hòa, cách reset và việc evaluation có cập nhật Q không. Hai chương trình cùng dùng seed 42 nhưng tách stream khác nhau có thể sinh các policy rất khác mà vẫn cùng mô hình phân phối.

Bộ bài lưu bản gốc, bản mở rộng, Q snapshot, từng episode, summary và version. Notebook đọc dữ liệu đầy đủ đã lưu và chạy thêm các ví dụ kiểm chứng nhỏ; script `--episodes 5000` tái tạo thí nghiệm chính. Sự khác biệt giữa "đọc kết quả đã chạy" và "rerun toàn bộ training" được ghi ở notebook và README.

Nếu môi trường phần mềm thay đổi, bit-level reproducibility của floating point hoặc backend có thể khác. Vì bài chủ yếu dùng NumPy và mạng CPU nhỏ, kiểm tra nên ưu tiên tái lập hành vi và protocol, đồng thời lưu version thực tế. Không được âm thầm thay kết quả cũ rồi giữ cùng mô tả phiên bản.

28.18 18. Phạm vi đóng góp của bộ báo cáo

Bộ bài cung cấp một triển khai học tập có nguồn và bằng chứng kiểm. Phân tích MAS chuyển taxonomy thành hợp đồng và failure injection; phân tích RL chuyển MDP thành phép kiểm trạng thái, oracle và exact policy evaluation. Những phần này làm rõ hai paper bằng một hệ có thể thực hiện, thay vì chỉ tóm tắt abstract.

Các giới hạn vẫn rõ: không gọi LLM, không tái lập các benchmark của AutoGen hoặc MetaGPT, không có dữ liệu intent thật và không có thị trường kinh tế đã hiệu chỉnh. Các đề xuất mở rộng được viết như hướng nghiên cứu để tiếp tục, có protocol và tiêu chí đo, không có kết quả được bịa cho những thí nghiệm chưa thực hiện.

29 Tổng hợp, phản biện và thiết kế nối hai hướng nghiên cứu

29.1 Bảng đối chiếu mục tiêu và loại bằng chứng

Khía cạnh	Survey MAS/LLM	Survey RL
Câu hỏi trung tâm	Ai phối hợp, bằng kênh nào, theo cơ chế nào?	Policy học từ trải nghiệm như thế nào?
Đơn vị phân tích	Tác tử, kênh, tổ chức, kiến trúc	State, action, reward, return, value
Vai trò dữ liệu	Ngữ cảnh, bằng chứng, message, mô hình chia sẻ	Transition và feedback từ tương tác
Rủi ro	Lỗi lan truyền, đồng thuận sai, overhead	Reward lệch, mẫu thiếu, state aliasing, bất ổn học
Đánh giá cần có	Quality, cost, latency, trace, ablation	Frozen evaluation, seed, oracle, robustness
Liên hệ bộ mã	Shopping MAS và failure injection	E-commerce Q-learning và DP

29.2 Hai cấp độ thích nghi không nên gộp

Điều phối động có thể thay graph thực hiện mà không cập nhật trọng số một mô hình nào. Một LLM đọc feedback và sửa kế hoạch trong cùng context có thể thay hành vi, nhưng đó chưa chứng minh có thuật toán RL tối ưu tham số. Ngược lại, Q-learning có thể cập nhật policy trong một môi trường có giao thức hoàn toàn cố định. "Agent biết thích nghi" vì thế cần chỉ rõ thứ gì thay đổi và theo cơ chế nào.

Trong tutorial, classifier được học offline bằng nhãn và giữ cố định trong inference. Coordinator thay trạng thái theo kết quả công cụ nhưng không học policy phân việc. Trong assignment, Q được cập nhật trong training rồi đóng băng để đánh giá. Ba loại thay đổi này cùng xuất hiện trong hệ thông minh nhưng có mục tiêu và bằng chứng khác nhau.

29.3 Thiết kế tham chiếu: trợ lý kinh doanh có tầng quyết định và tầng kiểm chứng

Một kiến trúc nối hai hướng có thể gồm tầng nhận yêu cầu, tầng dự đoán, tầng chọn hành động và tầng kiểm chứng. Intent classifier định tuyến nhiệm vụ; agent dữ liệu thu thập trạng thái có version; policy learner đề xuất mức khuyến mãi; policy validator kiểm ràng buộc; coordinator quản lý trạng thái và dịch vụ thực thi trả xác nhận. Proposal không được coi là một giao dịch đã thành công.

State cho learner nên chứa nhu cầu, tồn kho, lịch sử giá, ngân sách khuyến mãi và thông tin đối thủ được phép quan sát. Action là các mức giảm hợp lệ hoặc đề xuất giữ giá. Reward có thể dựa trên profit đã trừ chi phí liên quan, nhưng các điều kiện pháp lý/nghiệp vụ bắt buộc phải nằm ngoài score học. Nếu phản hồi đến muộn, cơ chế gán reward cần được thiết kế trước khi train.

Đây là đề xuất kiến trúc, chưa được triển khai thành dịch vụ kinh doanh. Bộ bài thực thi hai phần nhỏ có thể kiểm chứng: coordinator shopping và learner bán hàng toy. Chúng chưa tích hợp thành hệ tự đổi giá thật; vì thế báo cáo không đưa con số doanh thu hoặc lợi ích vận hành cho kiến trúc mở rộng.

29.4 Thiết kế thí nghiệm để tách lợi ích chia vai khỏi tăng tính toán

So sánh ít nhất bốn điều kiện: một agent với ngân sách chuẩn; một agent với cùng tổng ngân sách như MAS; MAS đầy đủ; MAS bỏ một cơ chế phối hợp quan trọng. Mọi điều kiện dùng cùng dữ liệu, quyền công cụ, tập tác vụ và tiêu chí thành công. Nếu MAS được thêm nguồn dữ liệu, cần có baseline cùng nguồn để tách ảnh hưởng dữ liệu khỏi tổ chức.

Đo success, groundedness, constraint violation, cost và latency; báo cáo cả phân phối và các trường hợp thất bại. Với LLM, lưu prompt/model version, nhiệt độ, số lần lấy mẫu, token và tool traces. Với RL, lưu training steps, evaluation tapes, seed và snapshot policy. Chỉ tăng số lần chạy mà không giữ protocol không tạo bằng chứng tái lập tốt.

Một ablation có ý nghĩa là bỏ bước inventory verification trong khi giữ số agent khác. Nếu chất lượng giảm, kết luận hẹp là verification có ích trong tập ca, chưa phải mọi cách chia vai đều có ích. Đây chính là cách đọc thí nghiệm B3 của tutorial. Nhận diện đúng biến độc lập giúp tránh gán công lao cho một nhãn kiến trúc rộng.

29.5 Đề xuất kiểm tra độ bền nhiều tầng

Tầng dữ liệu: thêm thiếu trường, schema sai, dữ liệu cũ và nguồn mâu thuẫn. Tầng giao tiếp: thêm timeout, duplicate response, sai task ID và response đến muộn. Tầng quyết định: đổi giá vốn, discount, đối thủ và nhu cầu. Tầng triển khai: đo tải, version và cơ chế khôi phục. Mỗi thay đổi cần giữ một tập đối chứng để xác định tác động.

Bộ mã hiện kiểm schema, freshness, timeout, forged output và shift discount; đồng thời có self-play để minh họa đồng học. Nó chưa kiểm tải mạng, Byzantine agent, đối thủ ngoài tập hoặc deployment thật. Một bản báo cáo chi tiết phải ghi cả những gì đã đo và những gì chưa đo để người đọc đặt đúng độ tin cậy.

29.6 Những luận điểm cần tránh sau khi đọc hai paper

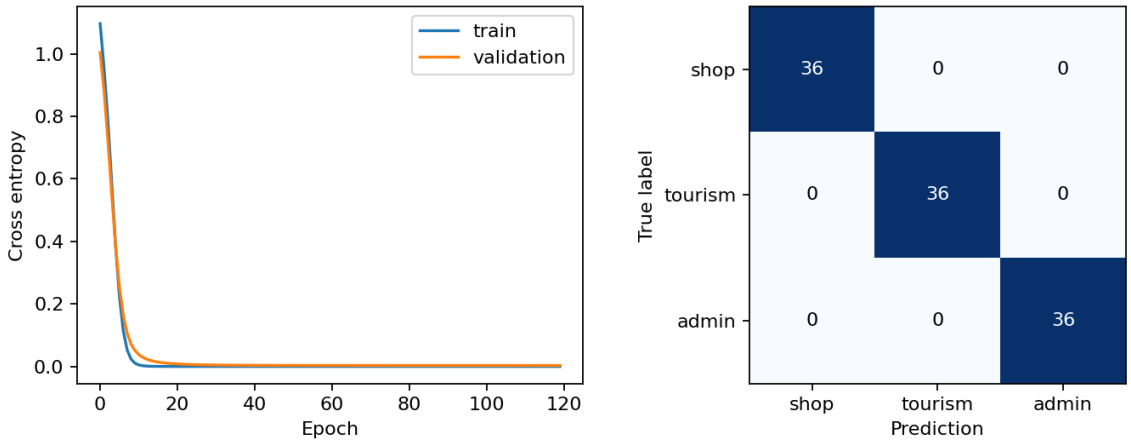
Không suy ra "nhiều agent hơn luôn đúng hơn"; sai số tương quan và chi phí giao tiếp có thể làm kết quả kém đi. Không suy ra "RL luôn học policy tối ưu"; giả định Markov, tính dừng, độ bao phủ và protocol đóng vai trò thiết yếu. Không suy ra "accuracy cao là hệ đáng tin"; một classifier đúng trên synthetic test vẫn nhận sai OOD. Không suy ra "rule-based không thích ứng đầu vào"; nó có thể phản ứng theo state dù không cập nhật từ reward.

Các kết luận tích cực vẫn rất rõ: chia vai có thể tạo hợp đồng kiểm được; RL cung cấp cơ chế cải thiện từ feedback; model-based oracle giúp đo khoảng cách tới chuẩn trong môi trường biết mô hình; và logging/ablation giúp truy nguyên lỗi. Giá trị của hai paper nằm ở cách đặt vấn đề, cộng với kiểm chứng cụ thể khi áp dụng.

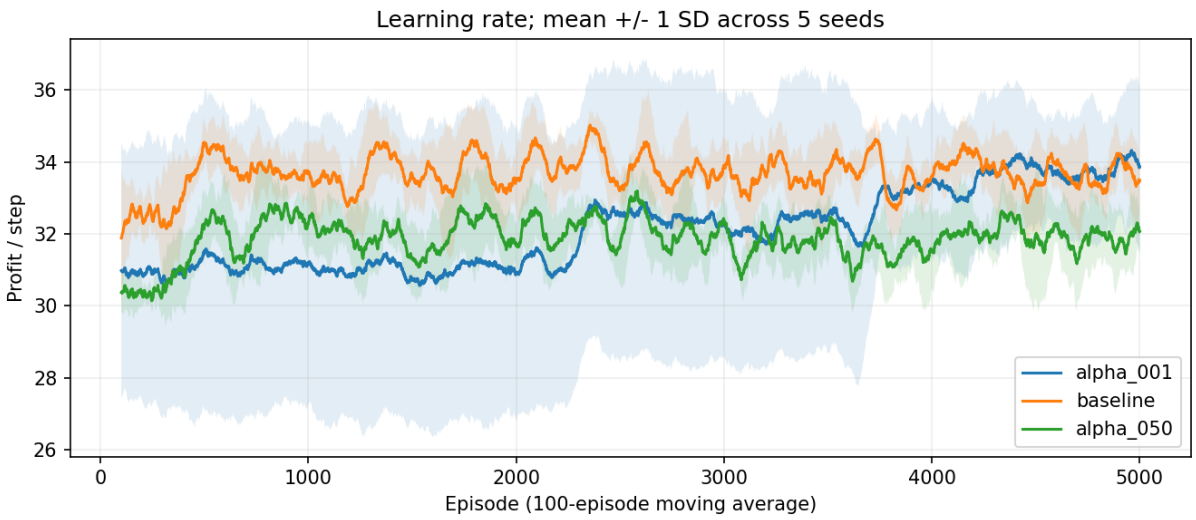
30 Bằng chứng minh họa liên hệ hai paper

Các kết quả dưới thuộc bộ mã của báo cáo, không phải bảng benchmark chép từ hai bài gốc. Tutorial kiểm kiến trúc local; assignment kiểm policy trong mô phỏng. Phần số liệu đầy đủ nằm trong hai báo cáo chuyên biệt và các CSV đi kèm.

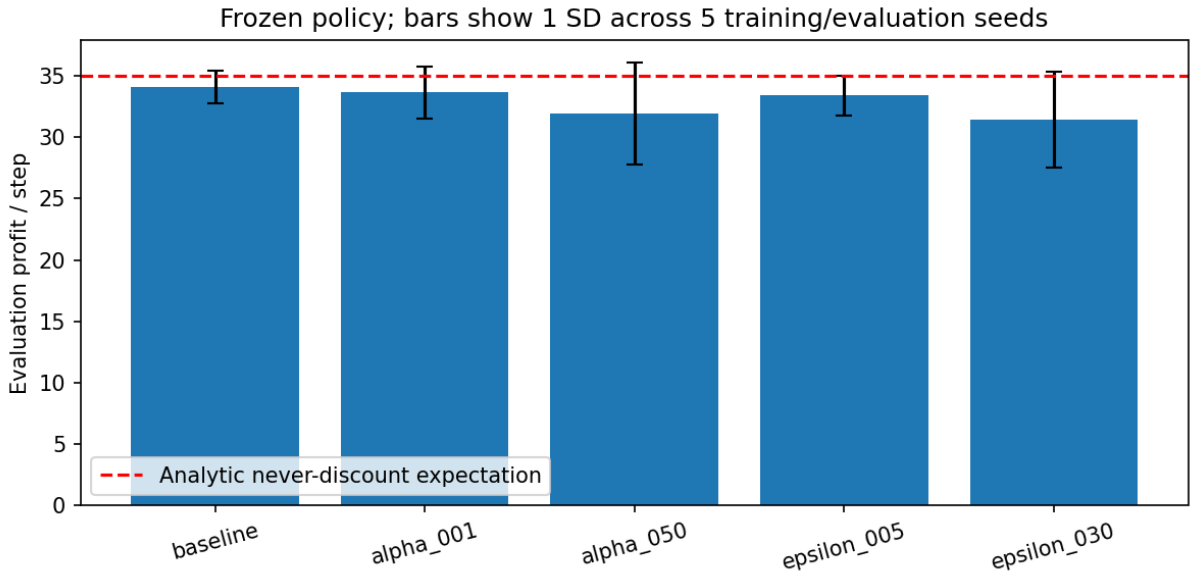
Thực nghiệm của báo cáo	Quan sát	Phạm vi kết luận
MAS B0/B1	Cùng 100% success trên 11 ca	Chia vai chưa tăng chất lượng trong catalog nhỏ
MAS B3	8/11 ca đúng; lỗi khi bỏ kiểm	Validation cần thiết trong tập failure injection
Intent test	108/108 đúng	Chỉ trên test tổng hợp chia nhóm
Intent OOD	4/6 clarify; 2 false accept	Softmax/confidence chưa đủ phát hiện ngoài miền
RL 5 seeds	34.0997 ± 1.3678	Baseline còn biến động và dưới policy không giảm
Rule/DP	Cùng policy và khoảng 35.1295	Best response trong mô hình đối thủ đã chỉ định



Hình 3: Minh họa phân biệt chất lượng trong phân phối và đánh giá ngoài miền; đây là dữ liệu tự thực thi.



Hình 4: Độ nhạy learning rate trong mô hình bán hàng toy; vùng tô là một SD qua năm seed.



Hình 5: Đánh giá policy đóng băng và mốc analytic không giảm giá.

31 Phụ lục đọc paper: bản đồ nội dung và câu hỏi kiểm tra

31.1 Đối chiếu Paper 1 với tài liệu gốc

Phần tài liệu gốc	Nội dung đã phân tích	Câu hỏi khi đọc lại
Section 1	Động lực, đóng góp, vị trí survey	Tại sao cơ chế phối hợp là trọng tâm?
Section 2	MAS, LLM, collaborative AI	Mô hình khác agent ở trách nhiệm nào?
Section 3	Agent tuple, system, collaboration channel	Kênh cần mô tả đủ những thuộc tính gì?

Phần tài liệu gốc	Nội dung đã phân tích	Câu hỏi khi đọc lại
Section 4.2	Cooperation, competition, coopetition	Mục tiêu cấp agent và cấp hệ có khác không?
Section 4.3	Rule-, role-, model-based	Đây là protocol hay decision policy?
Section 4.4	Centralized, decentralized/distributed, hierarchical	Quyền kiểm soát khác vị trí thực thi thế nào?
Section 4.5	Static/dynamic orchestration	Điểm nào thật sự thay đổi lúc chạy?
Section 5	Mạng, QA/NLG, xã hội/văn hóa	Ground truth và authority ở đâu?
Section 6	Collective intelligence, benchmark, safety	Bằng chứng nào còn thiếu?
Section 7	Kết luận taxonomy	Khung có giúp thiết kế phép thử không?

31.2 Đối chiếu Paper 2 với tài liệu gốc

Phần tài liệu gốc	Nội dung đã phân tích	Câu hỏi khi đọc lại
I–II	Mục tiêu, MDP/POMDP, policy, Bellman	State có đủ và quá trình có dừng không?
III	Bandit, exploration, value	Feedback khác nhãn giám sát thế nào?
IV-A	Dynamic programming	Biết mô hình có thể xây oracle nào?
IV-B	Monte Carlo	Return cần chờ đến đâu và nhiễu từ đâu?
IV-C	TD, SARSA, Q-learning	Target theo policy nào và có bootstrap không?
IV-D/E	Actor–critic, Bayesian	Mỗi mô hình ước lượng điều gì?
IV-F	DQN, replay và target network	Vì sao cần ổn định khi xấp xỉ Q?
V-A đến V-I	Các miền ứng dụng	Quyết định có ảnh hưởng tương lai thật không?
V-J/K	Challenges, lessons learned	Reward, coverage, non-stationarity có được kiểm?
VI–VII	Discussion và kết luận	Phát biểu hội tụ cần điều kiện gì?

31.3 Câu hỏi phản biện và lời giải định hướng

Vì sao hai agent đồng ý vẫn có thể sai? Vì cùng mô hình hoặc cùng nguồn có thể gây lỗi tương quan; đa số không tạo nguồn thông tin mới. Cần đối chiếu bằng chứng hoặc phép kiểm độc lập.

Vì sao cùng state–action có reward khác nhau chưa chắc là non-stationarity? Môi trường stochastic vốn tạo reward khác qua các mẫu. Non-stationarity nói quy luật phân phối thay đổi; cần nhiều dữ liệu hoặc kiểm soát policy đối thủ để phân biệt.

Vì sao tăng số seed chưa chứng minh mô hình kinh tế đúng? Seed chỉ thay chuỗi ngẫu nhiên trong mô hình đã giả định. Nếu conversion sai hoặc thiếu thị phần, mọi seed đều chịu cùng sai lệch cấu trúc.

Vì sao một agent biết mô hình có thể là baseline tốt? Nó giúp xác định khả năng đạt được trong mô phỏng nhỏ; chênh lệch với learner cho thấy khoảng cách cần cải thiện. Nhưng lợi thế biết mô hình phải được nêu để không làm so sánh thiếu công bằng về thông tin.

Vì sao không dùng DQN cho bảng tám ô? Không gian nhỏ không đòi xấp xỉ bằng mạng; mạng thêm nhiễu và tuning. Chọn tabular giúp minh bạch cập nhật và dễ đối chiếu oracle.

Vì sao ngưỡng confidence cần validation? Chọn trên test làm rò rỉ thông tin về benchmark cuối. Dù chọn đúng validation, confidence vẫn có thể cao với OOD nên cần đánh giá false acceptance riêng.

Khi nào nên dùng MAS? Khi có trách nhiệm, dữ liệu hoặc quyền tách biệt và phối hợp tạo lợi ích đo được. Nếu baseline đơn đạt cùng chất lượng với ít chi phí, lựa chọn đơn giản vẫn có cơ sở.

Khi nào nên dùng RL? Khi có chuỗi quyết định, feedback và điều kiện tương tác phù hợp để học. Nếu biết mô hình nhỏ và có lời giải chính xác, planning hoặc luật đã được kiểm chứng có thể phù hợp hơn.

32 Kết luận chung của báo cáo paper

Hai paper cung cấp hai mặt bổ sung của hệ tác tử: tổ chức tương tác và học quyết định. Một thiết kế có trách nhiệm cần cả mô hình khái niệm rõ lẫn bằng chứng thực nghiệm phù hợp. Bản báo cáo đã trình bày nền tảng, phân loại, công thức, thuật toán, ứng dụng, phản biện và liên hệ trực tiếp với mã đã chạy.

Giới hạn xuyên suốt là mô phỏng toy và dữ liệu intent tổng hợp. Các con số thực thi chứng minh quy trình và kiểm tra một số giả thuyết hẹp, không thay thế benchmark LLM hay đánh giá vận hành thực tế. Đó là ranh giới giúp sử dụng kiến thức từ survey một cách chính xác và có thể kiểm tra.

Phần III

ASSIGNMENT: RL VÀ RULE-BASED AGENTS

33 Giới thiệu và mục tiêu thí nghiệm

Laboratory Experiment 02 yêu cầu triển khai và so sánh reinforcement-learning-based agent với rule-based agent trong thị trường thương mại điện tử đơn giản. Báo cáo hoàn thành thí nghiệm A–E, giải đủ mười câu hỏi, triển khai biến đổi môi trường, thử đồng học và xây dựng một agent mốc tốt hơn bằng quy hoạch động. Các kết quả trong bảng và hình được tạo từ mã chạy thật; dữ liệu thị trường là mô phỏng, không phải số liệu kinh doanh thực tế. [2]

Theo đề, hạn nộp là 23:30 thứ Năm ngày 29/10/2026. Tên tệp theo quy ước và thông tin kế thừa từ bài trước là `agent02_CT_04_hainn.pdf`. Bìa sử dụng PTIT, lớp D23CTPM01, nhóm học phần CT01, Nguyễn Nam Hải, mã B23DCCN277 và giảng viên PGS.TS. Trần Đình Quế. Các thông tin này lấy từ báo cáo trước trong workspace.

Có ba mục tiêu đánh giá riêng: hành vi học thay đổi thế nào qua tương tác; policy đóng băng đạt lợi nhuận bao nhiêu; và kết quả có ổn định khi thay seed hoặc tham số môi trường không. Một đường reward tăng không tự trả lời hai câu sau. Báo cáo vì thế tách training, evaluation, fair policy comparison và robustness.

34 Mô tả môi trường và hai tác tử

34.1 Mô hình thị trường

Hai cửa hàng A và B chọn đồng thời hành động $a_i \in \{0, 1\}$, lần lượt là không giảm và giảm giá. Không agent nào được nhìn hành động hiện tại của đối thủ trước khi chọn. Mỗi bên quan sát nhu cầu $d \in \{0, 1\}$ và hành động trước của đối thủ. Không gian quan sát cục bộ có bốn phần tử và hai hành động nên bảng Q tham chiếu có tám giá trị.

Nhu cầu thấp có 10 khách hàng tiềm năng, nhu cầu cao có 20. Xác suất chuyển đổi cho agent i là:

$$p_i = \text{clip}(0.30 + 0.25d + 0.15a_i - 0.10a_{-i}, 0.05, 0.95).$$

Doanh số $S_i \sim \text{Binomial}(N_d, p_i)$; giá gốc $P = 10$, unit cost $C = 5$, discount fraction $\delta = 0.20$; reward:

$$r_i = S_i \left[P(1 - \delta a_i) - C\right].$$

Nhu cầu tiếp theo lấy đều độc lập; hành động hiện tại trở thành lịch sử của bước sau. Sales hai cửa hàng được lấy mẫu riêng có điều kiện theo nhu cầu và hành động, không có

ràng buộc tổng sales không vượt một nhóm khách hàng duy nhất. Vì vậy đây là mô hình tương tác cạnh tranh có tổng lợi ích thay đổi, không phải mô phỏng thị phần đầy đủ hoặc trò chơi tổng bằng không.

34.2 Agent A: Q-learning

Agent A dùng bảng $Q[d, c, a]$. Trong training, nó khám phá theo epsilon-greedy và cập nhật:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)].$$

Bước cuối dùng target r vì được coi là terminal trong protocol tham chiếu. Evaluation dùng argmax xác định, epsilon 0 và không gọi learn. Phân xử hòa trong evaluation chọn action 0; training phá hòa ngẫu nhiên. Quy ước này được công bố vì các policy snapshot cần cho cùng kết quả khi chạy lại.

34.3 Agent B: luật cố định

Luật là $\pi_B(d, c) = 1$ khi $d = 1$ và $c = 1$, còn lại bằng 0. Khi nhu cầu thấp, B không giảm. Khi nhu cầu cao, B chỉ giảm nếu A đã giảm ở bước trước. B không cập nhật từ reward và không có bảng Q. Tuy nhiên, hành động của B vẫn phụ thuộc quan sát và có thể thay đổi theo thời gian; không học không đồng nghĩa luôn có cùng một hành động.

34.4 Hai vấn đề trạng thái cần lưu ý

Quan sát cục bộ của A thiếu hành động trước của chính A, trong khi luật B sử dụng đúng bit đó. Với cùng $(d = 1, c_B = 0)$, B có thể chọn 0 hoặc 1 tùy hành động trước của A. Vì vậy quan sát tham chiếu không đảm bảo Markov đối với learner. Mã giữ nó để so sánh đúng baseline, nhưng báo cáo không áp một định lý MDP đầy đủ lên biểu diễn này.

Episode có 20 bước và target terminal ở cuối, nhưng bảng Q không chứa thời gian. Một quan sát ở đầu kỳ và cuối kỳ nhận các target có độ dài tương lai khác nhau. Đây là vấn đề hữu hạn chân trời bị gộp trong một bảng. Agent cải tiến dùng trạng thái đầy đủ và chỉ số thời gian, được trình bày ở Câu 10.

35 Mã nguồn và thay đổi so với tài liệu tham chiếu

Tệp `assignment/reference_agent02.py` là bản sao nguyên văn chương trình được cung cấp trong `agent02_code.tex`, lưu riêng để đối chiếu. Tệp `assignment/agent02_experiment.py` triển khai môi trường, Q-learning, luật, train, evaluate và chương trình xuất dữ liệu. Mã gốc được chạy thêm riêng cho seed 42; kết quả lưu với tiền tố `reference_original_42`.

Các thay đổi được công bố rõ: tách stream random nhu cầu, sales và agent bằng SeedSequence; đánh giá vector hóa theo episode với common random numbers; dùng cùng opponent cho so sánh policy; chạy toàn bộ cấu hình trên năm seed; xuất learning curves cho cả alpha và epsilon; bổ sung discount 40%, self-play và oracle DP. Quy luật phân phối nhu cầu, conversion, sales và margin được giữ nguyên cho baseline.

Tách RNG không tạo ra cùng chuỗi số ngẫu nhiên như chương trình gốc dù cùng seed số học. Vì vậy kết quả baseline 42 của bản mở rộng **không được gọi là tái lập bit-for-bit** của code gốc. Đó là một thực nghiệm cùng mô hình phân phối với protocol random rõ hơn. Sự khác biệt này còn được kiểm tra bằng việc chạy bản gốc riêng và đưa cả hai vào phụ lục số liệu.

Common random numbers trong evaluation được tạo bằng cùng demand tape và cùng tensor uniform Bernoulli cho mọi focal policy trên một seed. Nếu xác suất chuyển đổi tăng, cùng uniform cho phép so sánh có ghép cặp; số lượng sales vẫn có phân phối binomial đúng. Khi policy làm lịch sử khác đi, reward có thể khác, nhưng nhiễu đầu vào được kiểm soát tốt hơn so với hai lần chạy hoàn toàn độc lập.

36 Thiết lập thực nghiệm và protocol

Tham số	Giá trị
Training episodes	5.000 mỗi lần train
Steps/episode	20
Evaluation episodes	500
Evaluation steps	10.000 mỗi cặp policy/seed
Gamma	0.90
Alpha baseline	0.10
Epsilon baseline	max(0.02, 0.20 × 0.995^episode_index)
Alpha sensitivity	0.01, 0.10, 0.50
Epsilon sensitivity	Schedule baseline; fixed 0.05; fixed 0.30
Training seeds	11, 22, 33, 44, 55; run minh họa riêng seed 42
Evaluation seeds	800000 + training seed
Evaluation riêng seed 42	500042
Self-play evaluation	900000 + training seed
Discount robustness	40%, giữ các tham số còn lại

Năm seed được dùng nhất quán cho mỗi cấu hình alpha/epsilon. Mỗi cấu hình có 500.000 bước training cộng 50.000 bước evaluation. Toàn bộ baseline và mở rộng có 36 lượt training của chương trình mở rộng: 25 lượt sensitivity, một lượt minh họa seed 42, năm lượt retrain discount 40% và năm lượt self-play. Tính theo môi trường là 3.600.000 bước training; các lượt self-play cập nhật hai Q-table mỗi bước.

Đơn vị reward trong báo cáo là đơn vị lợi nhuận của mô hình, không được gán thành VND hoặc USD vì đề không quy định tiền tệ. Reward trung bình tính bằng tổng profit chia tổng bước. Standard deviation trong bảng cấu hình là sample SD trên năm **run means**, không phải SD của toàn bộ từng bước. SD từng episode được lưu riêng trong summary và không thay thế biến động giữa seed.

Khoảng tin cậy minh họa cho mean qua năm seed dùng $t_{0.975,4} = 2.776$, tức $\bar{x} \pm$

$2.776s/\sqrt{5}$. Với chỉ năm seed và policy phân phối có thể không chuẩn, khoảng này có tính xấp xỉ và dễ rộng. Báo cáo không dùng nó như bằng chứng chắc chắn về mọi môi trường hoặc mọi lần train.

37 Thí nghiệm A — Baseline và policy đã học

Lượt minh họa mở rộng seed 42 cho focal RL tổng reward 255591, trung bình 25.5591 trên 10.000 bước; opponent rule tổng 246856, trung bình 24.6856. RL không giảm 1849 lần và giảm 8151 lần, tức 81.51%; opponent giảm 38.53%. Đây là một policy seed cụ thể có chất lượng thấp so với mốc không giảm giá, không phải con số đại diện duy nhất cho Q-learning.

Bảng Q đầy đủ và policy ở bốn trạng thái được sinh từ `baseline_42_q.npy`, hiển thị ở phần số liệu tự động cùng heatmap. Vì reward discounted dài hạn có thể lớn hơn reward tức thời, độ lớn Q không được so trực tiếp với profit mỗi bước. Việc Q lớn chưa chứng minh greedy action đúng; phải đánh giá policy và đối chiếu oracle.

Ở năm seed dùng cho so sánh chính, baseline RL đạt mean 34.0997, SD 1.3678. Hai seed 22 và 33 học policy còn giảm giá đáng kể, các seed 11, 44 và 55 đạt hành vi không giảm ở các trạng thái đã gặp trong evaluation. Chênh lệch lớn giữa seed 42 và nhóm năm seed là bằng chứng rất rõ về rủi ro kết luận từ một lần chạy.

Cũng cần lưu ý reward đối thủ trong cùng trận không phải baseline tách biệt cùng đối thủ. Khi A giảm, B có thể phản ứng; hai bên không có cùng chuỗi quyết định. Vì vậy việc RL 25.56 cao hơn opponent 24.69 ở seed 42 không chứng minh RL tốt hơn mọi rule policy. Thí nghiệm E mới so sánh các focal policy dưới cùng opponent cố định.

38 Thí nghiệm B — Quá trình học

Learning curve dùng reward/step mỗi episode, làm trơn bằng trung bình trượt 100 episode. Trục x gắn với cuối cửa sổ; hình tổng hợp thể hiện mean và vùng ± 1 SD trên năm seed. Đó là cửa sổ trượt có chồng lấn, vì vậy các điểm liên tiếp không độc lập thống kê. Số liệu episode thô vẫn được giữ trong CSV.

Baseline có trung bình 500 episode đầu là 32.8595 và 500 episode cuối là 33.6718. Quan sát này phù hợp với việc policy cải thiện một phần trong training, nhưng mức tăng khoảng 0.8124 không chứng minh hội tụ hoặc tối ưu. Các đường còn dao động do nhu cầu, sales, khám phá và cập nhật Q.

Nếu nhìn cả training mean 33.6513 và frozen evaluation mean 34.0997, evaluation cao hơn trong nhóm seed này. Một giải thích phù hợp là tắt khám phá giảm chi phí thử hành động. Tuy nhiên, không nên quy mọi chênh lệch cho epsilon vì evaluation dùng dữ liệu khác và policy cuối chỉ là một snapshot. Các giới hạn về aliasing và thời gian vẫn tồn tại khi epsilon bằng 0.

Để nghiên cứu hội tụ sâu hơn, cần lưu Q qua các checkpoint, đo thay đổi policy, visit counts và regret so với oracle. Bộ bài hiện đã cung cấp bảng Q cuối, learning curve và nhiều

seed; chưa thực hiện chuỗi checkpoint lớn nên không đưa kết luận về tốc độ hội tụ chính xác.

39 Thí nghiệm C — Learning rate

Ba alpha 0.01, 0.10 và 0.50 giữ nguyên gamma, số episode và epsilon schedule. Mean đánh giá lần lượt 33.6484, 34.0997 và 31.9533; SD giữa seed lần lượt 2.1286, 1.3678 và 4.1613. Trong những run đã đo, alpha 0.10 có mean cao và biến động thấp hơn hai lựa chọn còn lại.

Alpha 0.01 tăng training từ 30.9924 ở 500 episode đầu lên 33.7541 ở 500 episode cuối, cho thấy việc cập nhật chậm có thể cần nhiều tương tác. Alpha 0.50 có SD đánh giá lớn, phù hợp với độ nhạy cao trước nhiễu reward và target. Nhưng dữ liệu này không đủ để nói learning rate 0.50 luôn gây bất ổn trong mọi môi trường.

Một điểm kỹ thuật là target terminal giảm mạnh ở cuối mỗi episode. Alpha lớn đưa trọng số lớn hơn cho các mẫu đó, trong khi state không chứa thời gian. Do vậy tác động alpha bị trộn với thiết kế biểu diễn. Muốn tách nguyên nhân, cần thêm ablation Q có chỉ số thời gian trên cùng alpha; đây là thí nghiệm được đề xuất chứ chưa được báo như đã chạy.

40 Thí nghiệm D — Khám phá

Schedule baseline, epsilon cố định 0.05 và cố định 0.30 cho mean evaluation lần lượt 34.0997, 33.3973 và 31.4300; SD lần lượt 1.3678, 1.6126 và 3.9272. Mean training tương ứng 33.6513, 32.9458 và 30.8822. Cấu hình khám phá cao không có lợi trong bộ run hiện tại.

Mức epsilon 0.30 không có nghĩa 30% bước chắc chắn chọn hành động không greedy. Với hai action, nhánh ngẫu nhiên vẫn có xác suất chọn greedy. Ngoài ra, tỷ lệ discount cuối còn phụ thuộc policy đã học, nên discount rate evaluation không bằng epsilon training. Báo cáo ghi riêng các đại lượng trong CSV để tránh đồng nhất chúng.

Tắt khám phá trong evaluation cần thiết để đo policy greedy, nhưng không sửa một bảng Q đã học sai. Đối với seed học ra lựa chọn discount không tốt, epsilon 0 làm lựa chọn đó lặp nhất quán. Vì vậy thiết kế khám phá cần được đánh giá bằng cả hiệu năng trong training, chất lượng cuối và biến động giữa seed.

41 Thí nghiệm E — So sánh policy công bằng

Focal policies gồm Q-learning, rule tham chiếu, luôn không giảm, luôn giảm và DP. Tất cả được đánh giá với cùng rule opponent, cùng seeds và cùng random tapes. Không cập nhật bất kỳ Q nào. Đây là phép so sánh phù hợp hơn việc chỉ đặt reward hai bên của một trận cạnh nhau.

Rule và never-discount đạt cùng hành vi khi cả hai lịch sử khởi đầu bằng 0. DP cũng chọn không giảm ở mọi state/time với tham số mặc định. Do cùng action và tape, ba hàng này có số đo giống hệt nhau từng seed. Mean khoảng 35.1295 không phải 35 đúng tuyệt đối; 35 là kỳ vọng lý thuyết, số đo hữu hạn có nhiễu.

RL baseline đạt 34.0997; mức kém hơn rule trên nhóm seed này khoảng 1.0298 lợi nhuận/step. Đây là kết quả trong môi trường tham chiếu. Nó không chứng minh rule tốt hơn RL ở mọi bài toán; thay đổi conversion, chi phí, đối thủ hoặc state có thể tạo kết quả khác. Điều có thể kết luận là baseline Q-learning hiện tại chưa vượt được một policy xác định rất đơn giản trong mô hình này.

42 Trả lời đầy đủ mười câu hỏi

42.1 Câu 1 — Mô hình tác tử và môi trường

Hai tác tử là hai cửa hàng chọn giảm giá. Môi trường gồm nhu cầu, lịch sử hành động, quá trình sales và quy tắc profit. Observation mỗi agent là (d, c); action gồm 0/1; reward là lợi nhuận đã trừ unit cost sau giảm giá; chuyển tiếp cập nhật lịch sử và sinh nhu cầu mới. Bốn observation kết hợp hai action tạo tám mục Q.

Đây là MAS vì reward của mỗi bên phụ thuộc hành động bên kia qua hệ số $-0.10a_{-i}$ và quan sát tương lai chứa hành động đối thủ. Hai policy có mục tiêu lợi nhuận riêng và quyết định tương tác. Không có thông điệp chat vẫn có thể là MAS: môi trường chung là kênh tương tác. Khi giữ opponent cố định, có thể phân tích learner như bài điều khiển trong một môi trường hiệu dụng, nhưng bản chất hệ gốc vẫn có hai tác tử.

Phân biệt observation với full state là cần thiết. Trạng thái đầy đủ $x = (d, a^A_{t-1}, a^B_{t-1})$ có tám tổ hợp, thêm thời gian nếu hữu hạn chân trời. Observation tham chiếu chỉ là một phép chiếu của full state. Nêu điều này giúp tránh gọi bốn observation là một MDP đầy đủ mà chưa kiểm giả định.

42.2 Câu 2 — Quyết định học và theo luật

RL chọn theo bảng giá trị được điều chỉnh từ reward; rule chọn theo điều kiện do người thiết kế viết. RL cần nhiều mẫu, RNG, bảng Q và công đoạn train; rule không cần tối ưu tham số. Trong inference của bài nhỏ này, cả hai đều rất nhẹ nên chi phí tính toán không phải yếu tố phân biệt lớn nhất. Chi phí thu thập trải nghiệm mới đáng lưu ý.

RL có khả năng thích nghi khi tiếp tục học và được cung cấp thông tin/reward phù hợp, nhưng một policy đóng băng không tự thích nghi chỉ vì tên của nó có chữ learning. Rule có thể phản ứng với đầu vào thay đổi mà không học. Khi môi trường đổi, cả hai đều có thể kém nếu biểu diễn hoặc luật không phù hợp.

Rule phù hợp cho điều kiện nghiệp vụ rõ, ít dữ liệu và cần giải thích, ví dụ không bán dưới giá vốn hoặc ngưỡng khuyến mãi đã được phê duyệt. RL phù hợp hơn khi có tương tác lặp, phản hồi đo được, mô phỏng đủ tin cậy và không biết trước chiến lược tốt. Trong chính môi trường hiện tại, policy không giảm rất mạnh; dùng RL chưa tạo lợi ích chỉ nhờ có học.

42.3 Câu 3 — Tính cập nhật Q-learning

Dữ kiện: $Q(s,a) = 4$, $\alpha = 0.1$, $r = 6$, $\gamma = 0.9$, $\max_{a'} Q(s', a') = 5$.

1. Giá trị tương lai sau chiết khấu: $0.9 \times 5 = 4.5$.
2. TD target: $y = 6 + 4.5 = 10.5$.
3. TD error: $\Delta = y - Q = 10.5 - 4 = 6.5$.
4. Mức thay đổi: $\alpha\Delta = 0.1 \times 6.5 = 0.65$.
5. Giá trị mới: $Q_{new} = 4 + 0.65 = \mathbf{4.65}$.

Agent nâng giá trị vì trải nghiệm mới gợi ý return cao hơn dự đoán. Nó chỉ đi 10% khoảng cách đến target để không tin hoàn toàn một mẫu có nhiễu. Con số này là ví dụ giả định của câu hỏi, không phải đo từ thị trường. Test `test_update_example` xác nhận kết quả.

Nếu transition terminal, không dùng giá trị 5 ở tương lai: target 6 và Q mới 4.2. Test terminal riêng dùng Q tương lai rất lớn để đảm bảo implementation không bootstrap nhầm. Hai phép kiểm tách công thức nonterminal và điều kiện kết thúc.

42.4 Câu 4 — Khám phá và khai thác

Khám phá giúp có dữ liệu về các hành động ít được chọn; khai thác tận dụng hành động có Q cao. Nếu dừng khám phá quá sớm, một reward xấu ban đầu có thể làm agent bỏ một lựa chọn tốt. Nếu khám phá quá nhiều, nó chịu chi phí thử và có thể tác động xấu đến đối thủ trong các bước tiếp theo.

Thực nghiệm schedule so với epsilon cố định 0.30 cho training mean 33.6513 so với 30.8822 và frozen mean 34.0997 so với 31.4300. Chênh lệch khoảng 2.77 ở training và 2.67 ở evaluation hỗ trợ nhận định schedule tốt hơn trong nhóm run này. Fixed 0.05 nằm giữa về mean evaluation 33.3973 nhưng chưa vượt schedule.

Đây không phải bằng chứng epsilon càng thấp càng tốt, vì fixed 0.05 vẫn thấp hơn schedule trong kết quả đo. Đầu kỳ khám phá, cuối kỳ khai thác và nhiễu học tương tác với nhau. Muốn chọn epsilon để triển khai, cần validation riêng; không nên tuning trên chính test đã báo cuối.

42.5 Câu 5 — Reward và mục tiêu kinh doanh

Không giảm: margin $10 - 5 = 5$ mỗi sale. Giảm 20%: giá còn $10(1 - 0.20) = 8$, margin $8 - 5 = 3$. Discount tăng conversion thêm 0.15 nhưng giảm margin 40%. Lợi nhuận tăng hay giảm phụ thuộc tích của số sale kỳ vọng và margin, không chỉ một yếu tố.

Nhu cầu thấp và đối thủ không giảm: không giảm kỳ vọng $10 \times 0.30 \times 5 = 15$; giảm kỳ vọng $10 \times 0.45 \times 3 = 13.5$. Sales kỳ vọng tăng từ 3 lên 4.5, nhưng profit giảm 1.5. Nhu cầu cao và đối thủ không giảm: profit từ 55 xuống 42 dù sales từ 11 lên 14. Hai ví dụ trực tiếp giải thích vì sao tối ưu conversion khác tối ưu profit.

Nếu reward là sales, discount có thể được ưu tiên vì hệ số chuyển đổi cao hơn. Nếu thêm reward cho khách hàng quay lại, agent có thể chấp nhận profit tức thời thấp hơn; muốn mô tả điều này cần thêm trạng thái về khách hàng hoặc mô hình tác động tương lai. Nếu có quy định margin tối thiểu, nên hạn chế hành động hợp lệ thay vì chỉ cộng penalty nhỏ.

42.6 Câu 6 — Hiệu năng và thống kê

Công thức là $\bar{r}_i = \sum_{t=1}^{T} r_i(t)/T$. Với lượt minh họa seed 42, $T = 500 \times 20 = 10000$, RL là $255591/10000 = 25.5591$ và rule đối thủ là $246856/10000 = 24.6856$. Đây là cách tính trên dữ liệu thật, minh họa rõ quan hệ giữa total reward và average reward.

So sánh công bằng tách biệt dùng cùng rule opponent và tape: RL baseline có mean qua năm seed 34.0997, rule khoảng 35.1295, never/DP giống rule trên các run đã đo. Bảng chi tiết từng seed và các cấu hình được tạo tự động từ CSV ở phần bằng chứng. Đơn vị sample để tính SD cấu hình là run mean, tránh coi hàng chục nghìn bước tương quan là độc lập.

Một run không đủ vì training RNG thay đổi hành động khám phá, sales noise thay đổi cập nhật, và policy cuối có thể mắc ở hành vi khác. Seed 42 cho kết quả rất thấp trong khi ba seed khác chọn không giảm trong evaluation. Ngoài biến động random, một mô hình toy không đại diện mọi thị trường; nhiều seed chỉ giải quyết một phần độ tin cậy nội bộ.

42.7 Câu 7 — Learning rate và ổn định

Alpha 0.01, 0.10, 0.50 có frozen mean 33.6484, 34.0997, 31.9533 và SD 2.1286, 1.3678, 4.1613. Những số này được đo với cùng seed set và các thông số khác được giữ. Alpha 0.10 có kết quả tốt nhất trong ba lựa chọn ở thí nghiệm hiện tại, không phải khuyến nghị phổ quát.

Alpha thấp giảm trọng lượng của từng mẫu mới, nên phản ứng chậm; nó tăng đáng kể từ đầu đến cuối training trong nhóm đo. Alpha cao phản ứng nhanh nhưng cũng nhạy với reward ngẫu nhiên và target bị gộp thời gian. Policy cuối alpha 0.50 có biến động lớn giữa seed. Đây là quan sát phù hợp với giả thuyết nhiễu cập nhật, chưa phải thí nghiệm tách nguyên nhân hoàn toàn.

Cần tránh dùng từ "hội tụ" chỉ vì đường được làm trơn tương đối phẳng. Alpha cố định, trạng thái cục bộ thiếu và 20 bước hữu hạn làm các điều kiện lý thuyết không được đáp ứng đầy đủ. Phân tích ổn định nên bổ sung policy flip rate hoặc khoảng cách Q tới mốc trong các thí nghiệm tiếp theo.

42.8 Câu 8 — Tương tác và môi trường không dừng

Discount của đối thủ giảm conversion của agent 0.10, đồng thời trở thành observation cục bộ ở bước tiếp. Hành động của agent cũng ảnh hưởng rule opponent ở tương lai. Vì vậy quyết định hiện tại có thể tác động cả reward tức thời và chuỗi reward sau, dù nhu cầu độc lập.

Khi hai Q learner cập nhật đồng thời, một agent thu thập dữ liệu từ policy đối thủ đang đổi. Đối với cùng trạng thái và hành động cục bộ, phân phối kết quả hiệu dụng có thể thay đổi qua thời gian. Các target trước đây vì thế không còn phản ánh cùng một môi trường. Đây là non-stationarity do đồng học, khác với stochasticity của sales trong một policy cố định.

Báo cáo **đã triển khai** self-play: hai Q learner cùng train 5.000 episode trên năm seed, rồi đóng băng cả hai và đánh giá riêng. Seed 11 có hai mean 30.5514 và 32.5935; các seed khác gần hành vi không giảm và quanh 35. Những số này cho thấy phụ thuộc seed, không

chứng minh nguyên nhân mọi lỗi là non-stationarity hoặc đạt Nash equilibrium.

Thí nghiệm tiếp theo để nghiên cứu sâu hơn là freeze một bên theo từng giai đoạn, lưu policy checkpoint và cross-play các cặp khác seed. Giữ cùng số bước và dữ liệu demand/sales để đối chiếu với simultaneous learning. Chỉ khi đó mới có cơ sở phân tích vai trò của tốc độ thay đổi đối thủ một cách rõ hơn.

42.9 Câu 9 — Giới hạn và biến đổi đã thực hiện

Ít nhất ba giới hạn chính: conversion tuyến tính là giả định minh họa; nhu cầu IID không có mùa vụ; sales hai cửa hàng không cạnh tranh qua ràng buộc thị phần chung. Ngoài ra, không có tồn kho, ngân sách quảng cáo, customer lifetime value, chi phí đổi giá hoặc heterogeneity khách hàng. Observation thiếu own previous action và Q không có thời gian làm hạn chế biểu diễn.

Biến đổi được triển khai là tăng discount fraction từ 20% lên **40%**. Giá giảm còn 6, margin còn 1; giữ nguyên tác động +0.15 lên conversion và các tham số khác để cô lập thay đổi margin trong mô hình. Đây là một stress test đối với cơ chế reward, không phải một khuyến nghị định giá thực tế.

Ba đối chiếu quan trọng là frozen RL từ môi trường cũ, retrained RL trong discount 40% và rule/DP. Frozen policy có discount ở seed 22/33 bị giảm mean từ khoảng 32.5–32.7 xuống 28.9–29.1; các seed không discount giữ nguyên reward trên cùng tape. Retraining không bảo đảm mọi seed sửa tốt: seed 33 còn mean 27.6282. Kết quả tổng hợp và từng seed được lưu trong robustness CSV.

Khi discount 40%, không giảm vẫn có margin 5 nên rule và DP giữ cùng hành vi và số đo baseline trên tape. Điều này giải thích tại sao một số hàng kết quả giống hệt, không phải dấu hiệu chép số. Thay đổi discount trong một môi trường còn đơn giản chưa chứng minh robustness toàn diện trước demand shift hoặc đối thủ mới.

42.10 Câu 10 — Thiết kế agent tốt hơn

Bản cải tiến đã thực thi là **finite-horizon model-based best response** bằng dynamic programming. State là (t, d, u, v), trong đó u và v là hành động trước của mình và đối thủ. Action vẫn 0/1; reward là profit theo đúng mô hình; transition dùng nhu cầu IID và $b = d\,u$ của rule opponent. Điều kiện cuối $V_H = 0$.

Với mỗi thời điểm đi ngược từ $H - 1$ về 0:

$$Q_t(x, a) = \bar{r}(x, a, b(x)) + \gamma \sum_{d' \in \{0,1\}} \frac{1}{2} V_{t+1}(d', a, b(x)), \qquad \pi_t(x) = \arg\max_a Q_t(x, a).$$

DP thêm thông tin cần thiết và xử lý đúng thời gian; với môi trường nhỏ và mô hình biết trước, nó là một mốc mạnh, minh bạch. Không có training samples nên không nên gọi là "Q-learning tốt hơn". So sánh phải ghi rõ lợi thế biết mô hình; đó là oracle trong môi trường mô phỏng đã xác định.

Đánh giá dùng cùng rule opponent, seeds và random tapes như RL/rule. Policy DP chọn không giảm và đạt cùng số đo với never/rule. Độ bền được kiểm thêm ở discount 40%, nhưng vì rule đã đồng hạng với DP trên cả hai chế độ, báo cáo **không tuyên bố agent cải tiến vượt cả hai baseline**. Nó sửa cách mô hình hóa và cung cấp chuẩn kiểm tra; muốn chứng minh vượt rule cần môi trường nơi optimal action thực sự đổi.

Một hướng nghiên cứu tiếp theo là conversion uplift lớn hơn hoặc đối thủ khác, nơi giảm giá đôi khi có lợi; dùng learned model với uncertainty hoặc tabular Q theo state đầy đủ và thời gian. Huấn luyện trên nhiều regime, tách validation để chọn tham số, rồi test zero-shot và retrain trên regime mới. Đo profit, worst-case regret, constraint violations, chi phí dữ liệu và độ ổn định qua seed. Đây là protocol đề xuất, không có số liệu giả cho những thí nghiệm chưa chạy.

43 Phân tích bổ sung: lợi nhuận, oracle và độ nhạy

43.1 Bảng lợi nhuận kỳ vọng

Với các tham số mặc định, clip không hoạt động vì conversion luôn trong miền 0.20–0.70. Bảng dưới tính từ mô hình, không phải sample mean:

Demand	Action đối thủ	Không giảm	Giảm 20%	Chênh giảm trừ không giảm
Thấp	0	15.0	13.5	-1.5
Thấp	1	10.0	10.5	+0.5
Cao	0	55.0	42.0	-13.0
Cao	1	45.0	36.0	-9.0

Hàng thấp/đối thủ 1 cho thấy discount không luôn kém về reward tức thời trong mọi joint action. Nhưng rule opponent không chọn 1 khi demand thấp, nên hàng đó không thuộc hành vi hiện tại của đối thủ. Kết luận best response phải dùng policy đối thủ và động lực tương lai, không chỉ xem một hàng riêng.

43.2 Giải thích mốc 35 và quy hoạch động

Nếu A luôn chọn 0 và lịch sử đầu bằng 0, rule B luôn chọn 0. Reward A kỳ vọng 15 ở demand thấp và 55 ở demand cao; trung bình 35. Ở cả hai mức nhu cầu với B=0, action 0 tốt hơn tức thời và còn tránh kích hoạt B giảm ở bước sau. DP xác nhận policy 0 toàn bộ state/time, làm cơ sở kiểm tra numerical.

Mốc này là expected reward mỗi bước chưa chiết khấu. Giá trị DP đầu kỳ dùng gamma 0.9 sẽ là tổng geometric có chiết khấu, không bằng 20×35 trong biểu thức giá trị. Báo cáo không so trực tiếp V với episode profit. Đây là một điểm nhỏ nhưng quan trọng khi đọc Q-table và đồ thị.

43.3 Ngưỡng discount có lợi tức thời

Gọi $p_0 = 0.30 + 0.25d - 0.10b$, margin gốc $m = P - C = 5$ và uplift $u = 0.15$. Khi chưa clip, discount có lợi tức thời nếu $(p_0 + u)(m - P\delta) > p_0 m$, tức:

$$\delta < \frac{um}{P(p_0 + u)}.$$

Ở demand thấp, b=0, ngưỡng xấp xỉ 0.1667; ở demand cao, b=0, ngưỡng khoảng 0.1071. Discount 0.20 cao hơn cả hai nên kém tức thời trong các tình huống này. Đây là phân tích độ nhạy giữ nguyên conversion uplift; trong thị trường thật uplift có thể phụ thuộc mức giảm, nên không được dùng công thức làm quy tắc kinh doanh trực tiếp.

44 Thảo luận về độ tin cậy, giới hạn và diễn giải

Có ba lớp kết luận cần phân biệt. Lớp tính toán là những hệ quả trực tiếp của công thức, như margin 5 và 3 hoặc kỳ vọng 35. Lớp thực nghiệm là các mean/SD đo từ seed cụ thể. Lớp giải thích là giả thuyết về vì sao alpha cao hoặc khám phá nhiều gây policy kém; muốn xác định nguyên nhân cần ablation thêm. Báo cáo duy trì ranh giới này trong mọi bảng và lời giải.

Multiple seeds cải thiện tính tin cậy nội bộ nhưng không sửa mô hình kinh tế sai. Five-seed means còn chịu cả nhiễu training và evaluation vì seed pair thay cùng nhau; báo cáo không giả rằng đã tách riêng hai nguồn phương sai. Muốn phân rã, cần nhiều evaluation tapes cho mỗi fixed trained policy và nhiều training seeds độc lập.

Common random numbers giảm nhiễu so sánh nhưng không biến policy thành độc lập. Các policy có history khác nhau, nên cùng noise tape có thể dẫn đến state khác nhau. Điều này hợp lệ và là phản ứng của môi trường. So sánh phải giữ đúng opponent, reset và horizon để không trộn các thay đổi ngoài ý muốn.

Tương tác thị trường toy còn thiếu nhiều cơ chế. Hai cửa hàng có thể bán tổng nhiều hơn số khách hàng nếu hiểu khách hàng như một tập độc quyền; do đó nên coi N là số cơ hội mua cho mỗi cửa hàng theo mô hình, không làm kết luận thị phần. Reward không có tiền tệ thật, không có chiến dịch hoặc customer segments; mọi đề xuất triển khai đều cần dữ liệu và thiết kế riêng.

45 Kiểm tra độc lập bằng đánh giá policy chính xác

Ngoài Monte Carlo evaluation, báo cáo đã tính kỳ vọng policy bằng forward occupancy trên full state. Tệp `analyze_exact.py` lan truyền xác suất trạng thái qua 20 bước, cộng reward kỳ vọng thay vì lấy mẫu sales. Phép tính dùng cùng luật opponent, reset và horizon; vì vậy cung cấp mốc kiểm số liệu độc lập với random tape.

Seed 11/55 của baseline có policy 0000 và exact mean 35. Seed 44 có policy 0001 nhưng các trạng thái dẫn tới discount không tới được từ reset, nên exact mean vẫn 35. Seed 22/33

có policy 1000 và exact mean xấp xỉ 32.448. Chuỗi bit theo thứ tự (0,0), (0,1), (1,0), (1,1). Các state không tới được giải thích vì sao bảng policy khác nhau vẫn có cùng hiệu năng.

Lượt seed 42 có exact mean xấp xỉ 25.698, gần sample mean 25.5591. Khoảng cách lớn với 35 vì thế thuộc chất lượng policy, không đơn thuần một evaluation tape quá xấu. Bảng exact của 25 cấu hình/seed được giữ trong `exact_policy_values.csv`; notebook hiển thị toàn bộ và giải thích phương pháp. Phân tích này không tạo thêm training run hoặc thay đổi kết quả đã đo.

46 Kết luận của assignment

Bài lab đã được hoàn thành với mã chạy, tách train/evaluation, thí nghiệm A–E, ba alpha, ba epsilon, năm seed, stress discount 40%, self-play và oracle DP. Mười câu hỏi được trả lời bằng công thức, số đo và giới hạn cụ thể. Các tệp CSV, Q-table, biểu đồ và notebook cho phép kiểm lại kết quả.

Trong môi trường tham chiếu, rule/never-discount là baseline rất mạnh, còn Q-learning có biến động theo seed và chưa vượt được mốc đó. Bài học chính là chất lượng của mô hình hóa, protocol và đối chứng quyết định ý nghĩa kết luận. Có học không bảo đảm tốt hơn luật, cũng như có nhiều agent không bảo đảm phối hợp hiệu quả.

47 Phụ lục kết quả đo và bằng chứng thực thi

Các bảng dưới được dựng trực tiếp từ CSV/JSON của lần chạy đầy đủ. Mọi mean cấu hình là trung bình qua năm seed 11, 22, 33, 44, 55; SD dùng ddof=1. Các ảnh là biểu đồ vẽ từ dữ liệu episode thực thi, không phải ảnh minh họa sinh bằng mô hình.

47.1 Bảng tổng hợp thí nghiệm alpha và epsilon

Cấu hình	Train mean	Eval mean	SD	CI95 ±	Discount (%)
baseline	33.6513	34.0997	1.3678	1.6981	16.16
alpha_001	32.0944	33.6484	2.1286	2.6425	19.91
alpha_050	31.8520	31.9533	4.1613	5.1661	38.08
epsilon_005	32.9458	33.3973	1.6126	2.0020	27.90
epsilon_030	30.8822	31.4300	3.9272	4.8755	33.88

47.2 So sánh công bằng với cùng đối thủ

Focal policy	Mean reward/step	SD giữa seed
RL	34.0997	1.3678
Rule	35.1295	0.1261
Never	35.1295	0.1261
Always	24.9722	0.0701

Focal policy	Mean reward/step	SD giữa seed
DP	35.1295	0.1261

47.3 Thí nghiệm biến đổi discount 40%

Policy	Mean reward/step	SD giữa seed
frozen_RL	32.6415	3.3620
retrained_RL	33.6009	3.3403
Rule	35.1295	0.1261
DP	35.1295	0.1261

47.4 Q-table minh họa seed 42

Demand	Opponent previous	Q(no discount)	Q(discount)	Greedy action
0	0	184.212867	199.449036	1
0	1	221.956047	181.668010	0
1	0	227.279761	235.197513	1
1	1	197.471701	218.162589	1

47.5 Từng seed của baseline

Seed	RL mean	Rule mean cùng tape	RL discount (%)
11	35.1825	35.1825	0.00
22	32.5267	35.0910	40.70
33	32.6863	35.2710	40.12
44	34.9350	34.9350	0.00
55	35.1680	35.1680	0.00

47.6 Cặp learner trong self-play

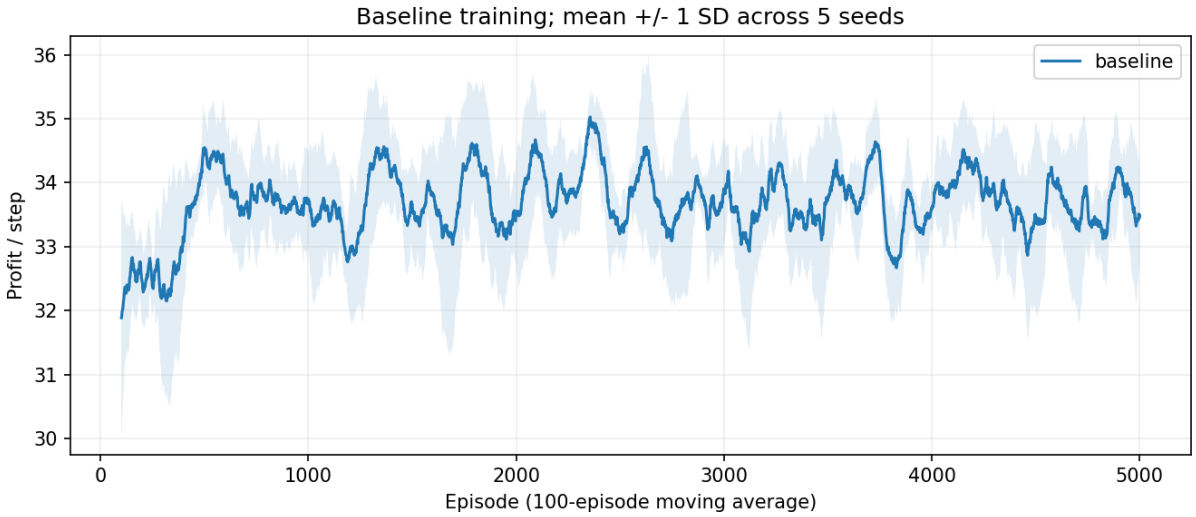
Seed	Learner	Eval mean	Discount (%)
11	0	30.5514	19.64
11	1	32.5935	40.21
22	0	35.0545	0.00
22	1	35.1300	0.00
33	0	35.3565	0.00
33	1	35.4800	0.00
44	0	34.8160	0.00
44	1	34.7835	0.00

Seed	Learner	Eval mean	Discount (%)
55	0	34.8630	0.00
55	1	35.0500	0.00

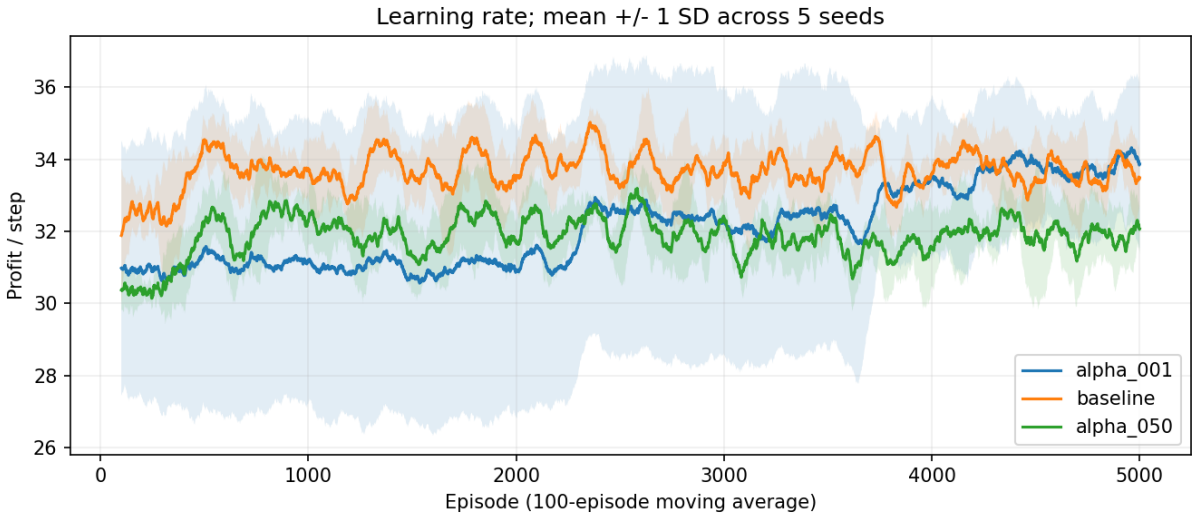
47.7 Đối chiếu bản gốc và bản mở rộng seed 42

Cùng seed số học không tạo cùng chuỗi khi cách dùng RNG khác. Bảng này ghi hai protocol riêng; không lấy chênh lệch làm bằng chứng chất lượng một thuật toán khác.

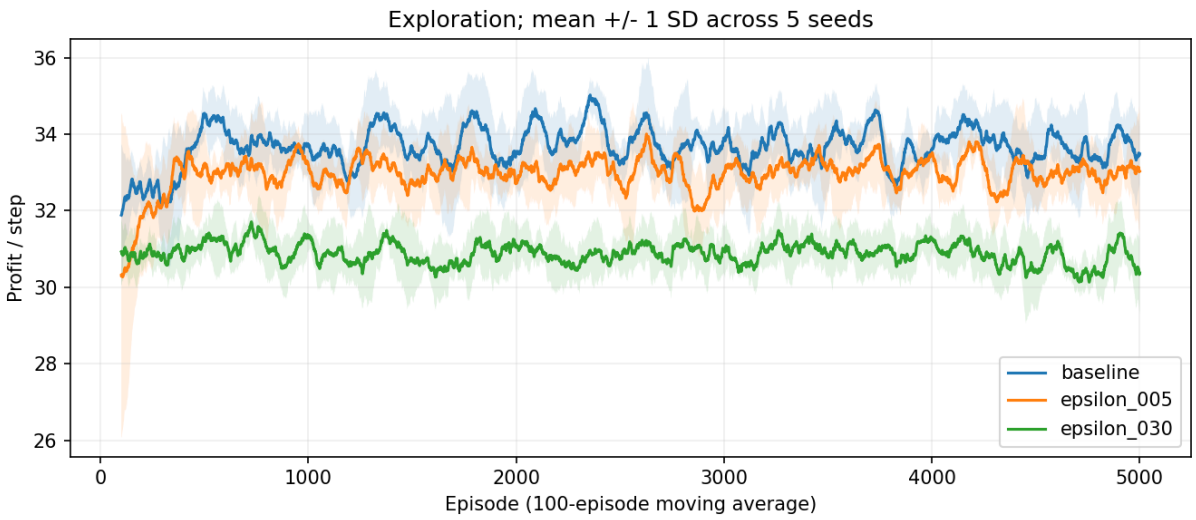
Protocol	Focal mean	Opponent mean
Code gốc nguyên văn	35.1275	34.9685
Bản mở rộng, split RNG	25.5591	24.6856



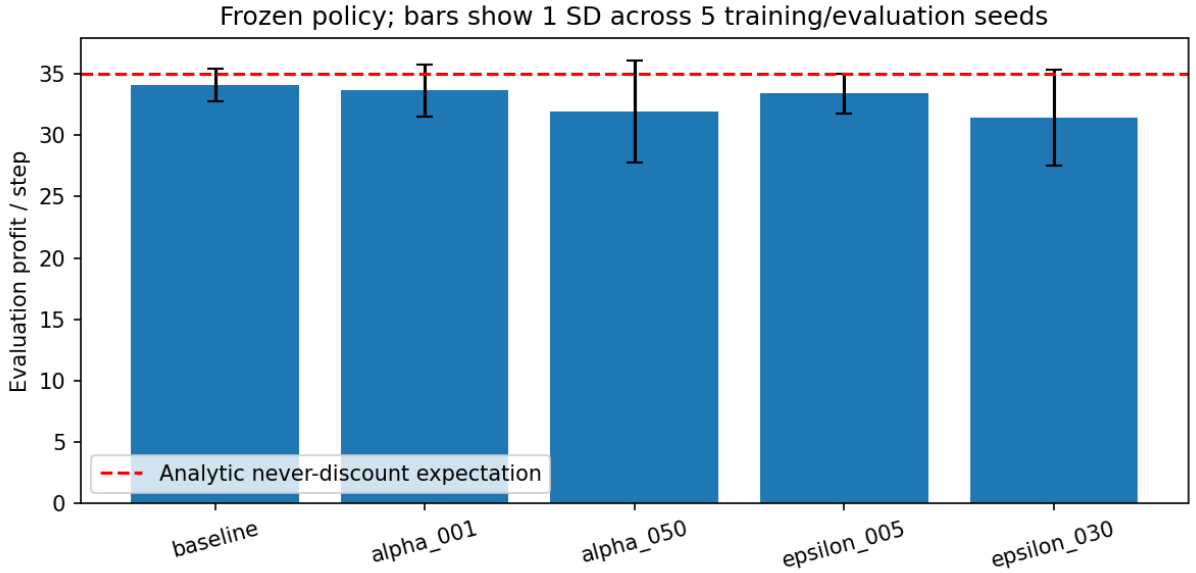
Hình 6: Learning curve baseline: moving average 100 episode, mean và một SD qua năm seed.



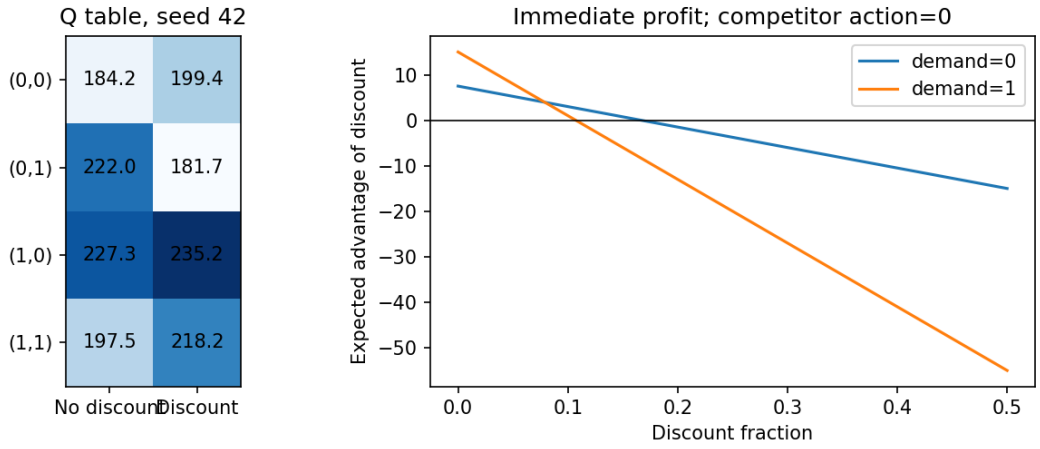
Hình 7: So sánh ba learning rate với cùng epsilon schedule.



Hình 8: So sánh ba cấu hình epsilon; mọi cấu hình đánh giá với epsilon bằng 0.



Hình 9: Policy đóng băng: mean và SD giữa seed; đường 35 là kỳ vọng lý thuyết.



Hình 10: Q-table seed 42 và độ nhạy lợi nhuận tức thời theo discount fraction.

47.8 Mã Python cốt lõi

Đây là các đoạn mã được lấy trực tiếp từ nguồn đã thực thi. Tệp đầy đủ và bản tham chiếu có trong thư mục assignment; các đoạn bên dưới đủ để kiểm cách cập nhật, tách evaluation và tính oracle.

Listing 14: Q-learning và luật, trích từ mã chạy

```
class QLearningAgent:
    def __init__(self, cfg=Config(), seed=10042):
        self.cfg=cfg; self.q=np.zeros((2,2,2)); self.rng=np.random.default_rng(seed)
    def act(self, state, epsilon=0.):
        if epsilon and self.rng.random()<epsilon: return int(self.rng.integers(2))
        q0,q1 = self.q[state]
        if epsilon and q0==q1: return int(self.rng.integers(2))
        return int(q1>q0)
    def learn(self,s,a,r,ns,terminal=False):
        target = r if terminal else r+self.cfg.gamma*float(max(self.q[ns]))
        idx=(*s,a); self.q[idx] += self.cfg.alpha*(target-self.q[idx])

class RuleBasedAgent:
    def act(self,state): return int(state[0]==1 and state[1]==1)
```

Listing 15: Quy hoạch động hữu hạn chân trời

```
def exact_best_response(cfg):
    """Finite-horizon DP on full Markov state (d, own_prev, opponent_prev).
    The fixed opponent's current action is d * own_prev; current actions simultaneous.
    """
    V=np.zeros((cfg.horizon+1,2,2,2)); policy=np.zeros((cfg.horizon,2,2,2),dtype=int)
    for t in range(cfg.horizon-1,-1,-1):
        for d in (0,1):
            for own in (0,1):
                for opp in (0,1):
                    b=d*own
                    values=[expected_profit(cfg,d,a,b)+cfg.gamma*np.mean(V[t+1,:,a,b]) for a in (0,1)]
                    policy[t,d,own,opp]=int(np.argmax(values)); V[t,d,own,opp]=max(values)
    return policy,V
```

Listing 16: Đánh giá đóng băng với common random numbers

```
def evaluate(cfg,policy='rule',opponent='rule',seed=800011):
    """500 episodes vectorized; demand and Bernoulli sales shared across policies.
    Q arrays are read-only inputs; np.argmax breaks exact ties toward action 0.
    """
    sd,ss=np.random.SeedSequence(seed).spawn(2)
    E,H=cfg.evaluation_episodes,cfg.horizon
    d=np.random.default_rng(sd).integers(2,size=(E,H))
    uniforms=np.random.default_rng(ss).random((E,H,2,20))
    prev=np.zeros((2,E),dtype=int); totals=np.zeros((2,E)); counts=np.zeros((2,2),dtype=int)
    def actions(p,idx,t):
        if isinstance(p,str):
            if p=='rule': return d[:,t]*prev[1-idx]
            if p=='never': return np.zeros(E,dtype=int)
            if p=='always': return np.ones(E,dtype=int)
            raise ValueError(p)
        if p.ndim==4: return p[t,d[:,t],prev[idx],prev[1-idx]]
        return np.argmax(p[d[:,t],prev[1-idx]],axis=-1)
    for t in range(H):
        acts=np.stack([actions(policy,0,t),actions(opponent,1,t)])
```

```
        for i in (0,1):
            p=conversion(d[:,t],acts[i],acts[1-i]); n=10+10*d[:,t]
            sales=((uniforms[:,t,i,:]<p[:,None]) & (np.arange(20)[None,:]<n[:,None])).sum(1)
            totals[i]+=sales*(cfg.price*(1-cfg.discount*acts[i])-cfg.cost)
            counts[i,1]+=int(acts[i].sum()); counts[i,0]+=int(E-acts[i].sum())
        prev=acts
    summary=[{'agent':name,'total_reward':float(totals[i].sum()),'mean_reward':float(totals[i].mean()/H),
              'episode_sd':float((totals[i]/H).std(ddof=1)), 'no_discount':int(counts[i,0]),
              'discount':int(counts[i,1]),'discount_rate':float(counts[i,1]/(E*H))} for i,name in enumerate
     (['focal','opponent'])]
    episodes=[{'episode':j+1,'focal_total':totals[0,j],'opponent_total':totals[1,j]} for j in range(E)]
    return summary,episodes
```

47.9 Bằng chứng kiểm thử

Bộ kiểm thử assignment có 12 ca, đã chạy thành công. Các ca kiểm công thức ví dụ 4.65; terminal không bootstrap; observation sau joint action; từ chối action sai; margin; bảng luật; Q không đổi khi đánh giá; tổng action counts; tái lập evaluation; DP chọn không giảm; tái lập training; và sample mean gần kỳ vọng 35 khi số episode lớn.

```
python3 -m unittest discover -s assignment -p 'test_*.py'
............
Ran 12 tests
OK
```

Log thực tế được giữ trong build/assignment_tests.log. Thời lượng đo cụ thể phụ thuộc máy, nên không dùng thời lượng test làm chỉ số thuật toán. Mười hai test không chứng nhận mô hình kinh tế đúng ngoài thực tế; chúng kiểm những bất biến quan trọng của implementation.

Tài liệu tham khảo

[1] PGS.TS. Trần Đình Quế (tài liệu môn học được cung cấp), *Developing Multiagent Systems: A Practical Tutorial with Python and PyTorch*, slide_multiAgent_system_20.10.2026.pdf, 25 trang, 2026. Nguồn cục bộ trong đề bài.

[2] *Laboratory Experiment 02: Reinforcement-Learning-Based Agents versus Rule-Based Agents*, Intelligent System Development, 2026–2027, assignment_Agent02_due_29.10.pdf, 13 trang; agent02_code.tex và agent02_complementary_code.pdf. Tài liệu cục bộ do người dùng cung cấp.

[3] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O'Sullivan, H. D. Nguyen, *Multi-Agent Collaboration Mechanisms: A Survey of LLMs*, arXiv:2501.06322v1, 10/01/2025. https://arxiv.org/abs/2501.06322v1. Phân tích trực tiếp bản PDF 35 trang được cung cấp.

[4] M. Naeem, S. T. H. Rizvi, A. Coronato, *A Gentle Introduction to Reinforcement Learning and Its Application in Different Fields*, IEEE Access, vol. 8, pp. 209320–209344, 2020. DOI: 10.1109/ACCESS.2020.3038605. Phân tích trực tiếp bản PDF 25 trang được cung cấp.

[5] R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018. Tài liệu nền tảng được cả slide và paper RL dẫn chiếu.

[6] C. J. C. H. Watkins, P. Dayan, *Q-learning*, Machine Learning 8, 279–292, 1992. DOI: 10.1007/BF00992698.

[7] Q. Wu et al., *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*, 2023. https://arxiv.org/abs/2308.08155.

[8] S. Hong et al., *MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework*, 2023, bản cập nhật 2024. https://arxiv.org/abs/2308.00352.

[9] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch, *Improving Factuality and Reasoning in Language Models through Multiagent Debate*, 2023. https://arxiv.org/abs/2305.14325.

[10] PyTorch, *CrossEntropyLoss*, tài liệu chính thức. https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html. Đối chiếu giao diện ngày 28/10/2026; môi trường thực nghiệm dùng PyTorch 2.8.0.

[11] Python documentation, *Coroutines and Tasks: asyncio.wait_for*. https://docs.python.org/3/library/asyncio-task.html. Phần timeout được kiểm trực tiếp bằng tests trong môi trường thực thi.

[12] Bộ mã và dữ liệu tự thực thi của báo cáo: tutorial/, assignment/, results/, build/; seed và cấu hình lưu trong summary.json, intent_metrics.json và mas_metrics.json, ngày 28/10/2026. Đây là nguồn kết quả mô phỏng, không phải nguồn dữ liệu thị trường thực tế.